diseño modular de muebles de melamina
9:42 a.m.
El **diseño modular de muebles de melamina** es la solución por excelencia para optimizar el espacio contemporáneo. Consiste en crear piezas independientes (o módulos estándar) que se pueden combinar, ampliar o reconfigurar de múltiples maneras según la necesidad del usuario y el tamaño del ambiente. [1, 2, 3]

Al unirse con la **melamina** —un tablero de partículas o MDF recubierto con resina sintética de alta resistencia— se logra un mobiliario altamente funcional, estético y con una excelente relación calidad-precio. [1, 2]

💡 Principios Clave en el Diseño Modular

1. **Estandarización de Medidas:** Para que un sistema modular funcione, se diseñan estructuras basadas en anchos fijos estandarizados (por ejemplo, módulos de 30 cm, 40 cm, 60 cm o 90 cm). Esto simplifica el corte del material y permite que las piezas encajen perfectamente entre sí.
2. **Componentes Dinámicos y Paramétricos:** En la actualidad, el diseño profesional se realiza con software especializado (como Polyboard o plugins de SketchUp/Revit). Al modelar un mueble paramétricamente, si cambias la altura o el ancho general, las divisiones internas, cajones y puertas se recalculan automáticamente junto con su respectivo **despiece** (lista de cortes exactos). [1, 2]
3. **Versatilidad de Conexión:** Los módulos se diseñan para unirse mediante herrajes invisibles o sistemas de fijación rápida (como los tornillos Minifix o espigas), facilitando su desmontaje, traslado y reinstalación sin dañar el tablero. [1]

🛠️ Aplicaciones Comunes

- **Cocinas Modulares:** Módulos bajos (para ollas, fregaderos, hornos empotrados) y módulos altos (alacenas, porta-microondas). Permiten configurar cocinas en "L", lineales o con islas combinando las piezas necesarias. [1, 2, 3, 4]
- **Closets y Vestidores (Walk-in Closets):** Bloques de cajoneras, módulos de zapateras, secciones de colgado y maleteros independientes que se ensamblan para cubrir la pared a medida. [1, 2]
- **Centros de Entretenimiento (Muebles de TV):** Combinación de repisas flotantes, módulos bajos cerrados y paneles decorativos de melamina para ocultar cables y albergar dispositivos electrónicos. [1, 2]
- **Oficinas y Home Office:** Escritorios que se pueden acoplar a libreros, cajoneras móviles y organizadores aéreos. [1, 2]

📈 Ventajas del Sistema Modular en Melamina

Ventaja	Descripción
Aprovechamiento del Espacio	Ideal para departamentos pequeños; permite crecer de manera vertical u horizontal.
Facilidad de Transporte	Al transportarse desarmado en paquetes planos (módulo por módulo), es mucho más fácil de trasladar que un mueble macizo tradicional.
Variedad Estética	La melamina actual ofrece acabados mate, brillantes y texturas realistas que imitan madera, mármol o cemento.
Durabilidad y Mantenimiento	Alta resistencia a arañazos, manchas y fácil limpieza diaria con un paño húmedo. Para áreas húmedas (cocinas o baños), se utiliza melamina **RH (Resistente a la Humedad)**.

🎨 Tendencias de Diseño

- **Estilo Minimalista y Escandinavo:** Muebles con líneas limpias, tiradores ocultos (perfil Gola o sistema *push-to-open*) combinando melaminas unicolor (blanco, gris, grafito) con maderas claras (como el roble o el haya).
- **Mobiliario Inteligente:** Módulos que ocultan camas, mesas plegables empotradas en escritorios o iluminación LED empotrada en ranuras del propio tablero de melamina.
- **Combinación de Texturas:** Mezclar un fondo texturizado amaderado con frentes de cajón en tonos lisos mate para generar contraste y elegancia. [1, 2, 3, 4]

Para ayudarte a estructurar tu proyecto, ¿me podrías indicar:

1. ¿Para qué **ambiente** estás diseñando el mueble (cocina, dormitorio, sala)?
2. ¿Estás buscando **ideas de distribución** estética o necesitas asistencia con el **software/planos de despiece**?
3. ¿Dispones de las **medidas aproximadas** del espacio?

- TikTok·andybea_melamina
¿Qué es un Mueble Modular? Ventajas y Cómo Funciona en Melamina
Un mueble modular es una solución de diseño pensada para adaptarse a tu espacio y necesidades específicas. los sistemas modulares ...

0:27

- Pinterest
91 ideas de MUEBLES DE MELAMINA | melamine muebles, melamina ideas, muebles de television modernos
Author: Michelle Cuya Content: 91 pins, 10y Topics: Melamine furniture (muebles de melamina) - Corner kitchen cabinet fabrication ...

- Slideshare
100 diseños de melamina con su lista de despiece | PDF
despieces para cabecera, veladores, tocadores, lijadura, comodas, escritorios, ropero, zapatero, mueble de tv, portamicrohondas, a...

Mostrar todo

mi clon de melastudio pro debe ser capaz de diseñar muebles modulares
Para que tu clon de **MelaStudio Pro** (o un software de diseño similar enfocado en optimizar proyectos en melamina) sea competitivo y verdaderamente funcional para carpinteros, diseñadores y fabricantes, debe estructurarse bajo un enfoque paramétrico y modular.

A continuación, te presento el mapa de funcionalidades críticas y la arquitectura lógica que debes implementar para que tu software sea capaz de diseñar muebles modulares con precisión:

---

1. El Motor Paramétrico (El Corazón del Software)

Un mueble modular no se dibuja línea por línea; se define mediante **reglas y variables**. Tu software debe procesar la geometría de forma dinámica:

- **Variables Globales:** El usuario debe poder definir el grosor del tablero (ej. 15mm, 18mm), el espesor del tapacanto (0.5mm o 2mm) y las holguras de instalación.
- **Reglas de Escalado Inteligente:** Si el usuario arrastra un módulo de cocina para cambiar su ancho de 40 cm a 60 cm, el software no debe deformar los laterales; debe recalcular automáticamente el ancho de los estantes internos, el zócalo y las puertas.

2. Biblioteca de Módulos Estándar

Los usuarios base (carpinteros y talleres) trabajan con estructuras predefinidas. Debes ofrecer una librería inicial de "módulos padre" modificables:

- **Módulos de Cocina:** Bajos (para cajoneras, puertas, esquineros, hornos) y altos (alacenas, campanas).
- **Módulos de Clóset:** Torres de cajones, pantaloneros, maleteros y módulos de colgado.
- **Módulos de Oficina:** Escritorios, libreros y cajoneras móviles (*pedestals*).

3. Sistema de Ensamble y Mecanizado Automático

Diseñar el módulo es solo el primer paso; el software debe saber cómo se va a armar:

- **Perforaciones Inteligentes:** Al unir el lateral con el piso del mueble, el sistema debe proyectar automáticamente dónde irán los tornillos soberbios, las espigas de madera o los sistemas **Minifix**.
- **Colocación de Herrajes:** Si el usuario añade una puerta, el software debe calcular las perforaciones de cazoleta para las bisagras bidimensionales en base a la altura de la puerta. Lo mismo aplica para las correderas de los cajones.

4. Generación de Entregables Automáticos (El Valor Comercial)

El gran atractivo de herramientas como *MelaStudio* o *Polyboard* es que ahorran horas de oficina técnica. Tu clon debe exportar con un solo clic:

- **Plano de Despiece:** Una lista exacta con las piezas del módulo, indicando largo, ancho, material y en qué cantos lleva pegado el tapacanto (delgado o grueso).
- **Módulo de Optimización de Corte:** Un algoritmo integrado (o exportación a formato compatible como .OPTIMIK o .CUT) que acomode las piezas en tableros estándar de melamina (ej. 2.50 x 1.83 m) para minimizar el desperdicio.
- **Archivos CNC (Opcional/Premium):** Exportación en formatos DXF o códigos G para que las máquinas automatizadas perforen y corten los módulos sin intervención manual.

5. Renderizado en Tiempo Real y Catálogo de Texturas

Para que el cliente final apruebe el proyecto, el software necesita una interfaz visual atractiva:

- Aplicación de texturas realistas (vetas de madera horizontales/verticales, acabados mate, alto brillo).
- Cambio masivo de materiales (ej. *"Cambiar todo el proyecto de Blanco MDF a Roble Texturizado"* en un solo paso).

🛠️ Flujo de Usuario Recomendado en tu Software

`[ Elegir Módulo Base ]` ➔ `[ Ajustar Medidas (H, W, D) ]` ➔ `[ Añadir Divisiones / Cajones ]` ➔ `[ Generar Despiece y Presupuesto ]`

Para afinar el enfoque del desarrollo:

1. ¿En qué **lenguaje o entorno de desarrollo** estás construyendo tu software (web con JavaScript/Three.js, una app de escritorio en C++, etc.)?
2. ¿Qué nivel de automatización buscas? ¿Te enfocarás solo en el **diseño 3D y despiece** o también quieres incluir **optimización de corte y presupuestos** automáticos?

debe ser un plugin de sketchup, el diseño debe apreciarse en sketchup

Desarrollar tu clon como un **plugin para SketchUp** es una estrategia excelente, ya que aprovechas el potente motor gráfico 3D de SketchUp y su enorme adopción en la industria de la carpintería. En este entorno, el plugin interactúa directamente con la API de Ruby de SketchUp para crear y manipular geometría en tiempo real.

Para lograr que el diseño modular de melamina se aprecie y gestione correctamente dentro de SketchUp, tu plugin debe estructurarse bajo los siguientes pilares de desarrollo técnico:

---

1. Arquitectura de Datos: Grupos y Componentes Anidados

En SketchUp, la organización de la geometría es vital para que el modelo sea editable y sirva para generar reportes. Tu plugin debe estructurar cada mueble usando una jerarquía estricta de `Sketchup::ComponentInstance`:

- **Componente Mueble (Padre):** Es el contenedor global del módulo (ej. *Bajo_Cocina_60cm*). Contiene los atributos dinámicos generales como `Ancho`, `Alto`, `Profundidad` y `Material_Estructura`.
- **Componentes Pieza (Hijos):** Dentro del contenedor padre, el script de Ruby debe generar cada placa de melamina como un componente independiente (ej. *Lateral_Izquierdo*, *Base*, *Amarre*).
  - **Importante:** Nunca uses "Grupos" comunes para las piezas; usa **Componentes**. Si el usuario copia el módulo, SketchUp sabrá que las piezas internas son idénticas, optimizando el rendimiento del archivo.

2. Creación Paramétrica mediante Ruby (`Sketchup.active_model`)

Para evitar que el mueble se "deforme" al escalarlo con la herramienta nativa de SketchUp, tu plugin debe redibujar o reposicionar las piezas programáticamente cuando el usuario altere las medidas en la interfaz de tu plugin.

Por ejemplo, al cambiar el **Ancho (W)** de un módulo:

1. El lateral izquierdo se queda en la coordenada X = 0.
2. El lateral derecho se desplaza automáticamente a la coordenada \(X = W - Espesor_Tablero\).
3. El piso del mueble (la base) cambia su escala en el eje X para medir exactamente \(W - (Espesor_Tablero \times 2)\).

3. Interfaz de Usuario (UI) con `UI::HtmlDialog`

Para ofrecer una experiencia fluida tipo *MelaStudio*, no uses las cajas de entrada nativas y limitadas de SketchUp. Debes construir una **interfaz web (HTML/CSS/JavaScript)** que se renderice dentro de un panel flotante de SketchUp mediante la clase `UI::HtmlDialog`.

- **Comunicación Bilateral:** Desde la ventana HTML, el usuario cambia el ancho de un módulo mediante un *slider* o input numérico. JavaScript envía un evento a Ruby (`sketchup.cambiarMedidas(ancho)`), y el script de Ruby altera instantáneamente el componente 3D en la pantalla de SketchUp.

- **Librería Visual:** En este panel HTML puedes mostrar miniaturas 2D atractivas de tu catálogo de módulos independientes. Al hacer clic en uno, Ruby lo invoca en el origen de coordenadas de SketchUp.

4. Diccionarios de Atributos (`AttributeDictionaries`)

Para que tu plugin pueda extraer el **despiece, tapacantos y herrajes**, cada componente pieza debe llevar "metadatos" ocultos que SketchUp almacena en la base de datos del archivo (`.skp`).
A cada pieza generada por tu código, asígnale atributos específicos:

ruby
```
pieza.set_attribute("MelaStudio_Clon", "Largo", 720)
pieza.set_attribute("MelaStudio_Clon", "Ancho", 550)
pieza.set_attribute("MelaStudio_Clon", "Tapacanto_Superior", "Grueso")
pieza.set_attribute("MelaStudio_Clon", "Tapacanto_Frontal", "Delgado")
```
Usa el código con precaución.
Al finalizar el diseño, un script de exportación leerá todos los diccionarios del modelo para compilar la lista de corte instantáneamente en un formato CSV o PDF.

5. Aplicación de Texturas y Orientación de la Veta

La melamina amaderada requiere un cuidado estricto con el sentido de la veta. Tu plugin debe controlar cómo se posiciona el material en SketchUp:

- Al crear el material de melamina en SketchUp (`Sketchup.active_model.materials.add`), asegúrate de mapear las coordenadas UV de la textura en las caras frontales de los componentes.
- Si una pieza se diseña de manera horizontal (como una encimera), el plugin debe rotar la textura 90 grados automáticamente si el usuario especifica que la veta corre en el sentido de la dimensión más larga.

💻 Ejemplo Base de Código (Ruby para SketchUp)

Así es como tu plugin generaría programáticamente una pieza limpia (un lateral de melamina de 18mm) respetando el origen de coordenadas:

ruby

```
model = Sketchup.active_model
model.start_operation('Crear Placa Melamina', true)

# Definir dimensiones (en pulgadas internamente por SketchUp, convertidas de mm)
espesor = 18.mm
alto = 720.mm
profundidad = 550.mm

# Crear definición del componente
definicion = model.definitions.add("Lateral_Izquierdo")
entidades = definicion.entities

# Dibujar la cara base y extruir (dar volumen)
cara = entidades.add_face([0, 0, 0], [espesor, 0, 0], [espesor, profundidad, 0], [0, profundidad, 0])
cara.pushpull(-alto) # Extrusión vertical

# Insertar el componente en el espacio de trabajo de SketchUp
transformacion = Geom::Transformation.new([0, 0, 0])
instancia = model.active_entities.add_instance(definicion, transformacion)

model.commit_operation
```
Usa el código con precaución.
Para estructurar la lógica del código o los siguientes pasos del desarrollo:

1. ¿Planeas programar el plugin **desde cero en Ruby**, o vas a utilizar algún framework/plantilla base para extensiones de SketchUp?
2. ¿Te gustaría que desarrollemos la estructura de comunicación básica de **`UI::HtmlDialog`** (HTML/JS a Ruby) para ver cómo conectar los inputs de la interfaz con el mueble 3D?

existen patrones o standares para diseñar muebles modulares
Sí, existen estándares internacionales y patrones antropométricos muy claros que rigen el diseño de muebles modulares. Dado que estás desarrollando un plugin de SketchUp, integrar estos estándares como valores por defecto (valores *default*) en tu código es vital para que los carpinteros y diseñadores sientan que el software habla su mismo idioma técnico.

El diseño modular se basa fundamentalmente en el **Sistema de 32 mm** (estándar global de fabricación) y en las **medidas antropométricas estandarizadas** de los espacios habitables.

---

1. El Estándar Técnico de Fabricación: El Sistema de 32 mm

Nacido en Europa tras la Segunda Guerra Mundial para estandarizar la producción industrial con tableros, es la regla de oro para la modulación en melamina. Tu plugin debería calcular los mecanizados bajo este patrón:

- **Distancia entre centros:** Todos los agujeros del sistema (para soportes de estantes, bases de bisagras o correderas de cajón) deben estar separados por **múltiplos exactos de 32 mm** (ej. 32, 64, 96, 128 mm).
- **Diámetro del agujero:** Típicamente **5 mm** para soportes y tornillos de unión euro, o **8 mm** para espigas de madera.
- **Distancia al borde frontal:** La primera línea vertical de perforaciones se ubica exactamente a **37 mm** desde el canto frontal del lateral del mueble.

Si configuras tu código de Ruby para colocar los herrajes respetando los 37 mm y múltiplos de 32 mm, cualquier bisagra (Blum, Hettich, Ducasse) encajará a la perfección sin que el usuario tenga que medir nada.

---

2. Estándares de Dimensiones por Ambiente (Medidas Comerciales)

Para crear tu biblioteca de "módulos padre" en SketchUp, debes programar los componentes utilizando las siguientes envolventes estándar (módulos que varían casi siempre en anchos múltiplos de 5 cm o 10 cm):

Módulos de Cocina (Los más estandarizados)

- **Módulos Bajos:**
  - **Altura total: 850 mm a 900 mm** (incluyendo las patas regulables de 100-150 mm y la encimera/cubierta de 20-40 mm).
  - **Profundidad estándar: 580 mm a 600 mm** (dejando una holgura para la encimera que suele medir 600 mm o 620 mm).
  - **Anchos modulares:** 150, 200, 300, 400, 450, 500, 600, 800, 900 y 1200 mm.
- **Módulos Altos (Aéreos):**
  - **Altura:** Variables comunes de **600, 700, 800 o 900 mm** según el diseño técnico.
  - **Profundidad estándar: 300 mm a 350 mm** (obligatorio para evitar que el usuario se golpee la cabeza al trabajar en la encimera).
  - **Distancia sobre encimera:** Debe dejarse un espacio libre de **550 mm a 600 mm** entre el mueble bajo y el alto.

Módulos de Clóset y Vestidores

- **Profundidad:**
  - **Con puertas batientes o correderas: 600 mm** mínimo (para que la ropa colgada en ganchos no choque con las puertas).
  - **Sin puertas (Walk-in closet): 450 mm a 550 mm** es suficiente.
- **Anchos modulares:** Las torres de cajones y repisas se modulan usualmente en **400, 500, 600 o 800 mm** para evitar que las repisas de melamina se pandeen (doblen) por el peso.
- **Módulos de colgado:** Alto de **900 a 1100 mm** para ropa corta (camisas/sacos) y **1500 a 1700 mm** para ropa larga (vestidos/abrigos).

Módulos de Oficina / Escritorios

- **Altura de mesa: 730 mm a 750 mm** estándar antropométrico.
- **Profundidad: 500 mm** (Home office básico) a **600-750 mm** (Oficinas profesionales).

---

3. Patrones de Diseño Estructural en Melamina (Reglas de Oro del Carpintero)

Para que tu clon genere diseños estructuralmente viables, debes programar restricciones físicas en las uniones:

- **Amarres Superiores:** Los muebles bajos de cocina no llevan un techo de melamina completo (para ahorrar material y ventilar). En su lugar, llevan listones de melamina (llamados "amarres" o "fajas") de **70 mm a 100 mm** de ancho, colocados de forma horizontal o vertical.
- **Zócalo o Retranqueo:** Las bases de los muebles no tocan directamente el suelo (para evitar la humedad). Se elevan sobre patas plásticas ocultas por un zócalo que va retirado hacia adentro de la estructura entre **30 mm y 50 mm** (*retranqueo*) para que los dedos de los pies del usuario no choquen al acercarse al mueble.
- **Fondo del Mueble (Trasera):** El fondo del módulo no suele ser de melamina de 18 mm. Se acostumbra usar MDF delgado (HDF/Durolac de 3 mm o 5 mm). Tu plugin debe crear automáticamente una **ranura (o rebajo)** en los laterales, techo y piso, típicamente a **15 mm o 20 mm hacia adentro desde el borde posterior**, para insertar este fondo deslizable.

🚀 Cómo implementarlo en el código de tu plugin

En tu UI flotante (`UI::HtmlDialog`), puedes ofrecer una pestaña de **"Configuración de Fábrica"** donde los valores por defecto respeten estas normas:

ruby
```
# Ejemplo de inicialización de variables por defecto en tu script de Ruby
@grosor_melamina = 18.mm
@sistema_32mm    = 32.mm
@distancia_borde = 37.mm

# Al invocar un módulo bajo de cocina "Padre" por defecto:
@modulo_alto_default  = 720.mm
@modulo_ancho_default = 600.mm
@modulo_prof_default  = 580.mm
```
Usa el código con precaución.
¿Qué te parece esta lógica? Si quieres, podemos concentrarnos en detallar **cómo estructurar la base de datos de los herrajes** en el plugin o en desarrollar un ejemplo de código en Ruby para **generar automáticamente la ranura del fondo** de un módulo.

el closet debe tener modulo para sabaneras en la parte superior, modulos para centro de entretenimiento escritorios como serian
Integrar un **centro de entretenimiento** y un **escritorio** dentro de la misma estructura de un clóset modular es una de las soluciones más solicitadas en el diseño de habitaciones modernas.

Para que tu plugin de SketchUp lo dibuje de forma perfecta y técnicamente viable, el clóset no debe diseñarse como una sola pieza gigante, sino como una **combinación de 4 o 5 módulos independientes** colocados uno al lado del otro y rematados por un gran módulo superior continuo.

Aquí te detallo la estructura, medidas estándar y el desglose de cómo tu software debe modelar e integrar cada zona:

🛋️ Arquitectura Modular del Conjunto (Distribución en SketchUp)

Imagina una pared dividida en bloques verticales (módulos). Una distribución ideal de izquierda a derecha sería:

```
[ MOD. CLÓSET TRADICIONAL ] + [ MOD. ESCRITORIO + TV ] + [ MOD. CAJONERA / CALZADO ]
└──────────────────────────── MALETEROS / SABANERAS CONTINUOS ────────────────────────────┘
```

🛏️ 1. El Módulo Superior: Sabaneras y Maleteros

Este módulo corona todo el mueble y va pegado al techo. Su función principal es almacenar elementos de gran volumen y poco uso diario (sábanas, edredones, maletas).

- **Altura estándar:** Entre **400 mm y 600 mm** (dependiendo de la altura total de la habitación).

- **Ancho:** Puede ser una sola estructura larga dividida internamente, o frentes de puertas que copien el ancho de los módulos inferiores (ej. módulos de 600 mm o 800 mm acoplados).
- **Profundidad: 600 mm** (alineado con el fondo del clóset tradicional).
- **Detalle técnico para tu plugin:** Al ser un espacio profundo, se recomienda usar **puertas batientes con pistones a gas inversos** (para que abran hacia arriba) o **puertas corredizas**. Si se usan estantes internos para sábanas, estos deben ser fijos (atornillados) para actuar como amarras estructurales debido al peso que soportarán.

🖥️ 2. El Módulo de Centro de Entretenimiento y Escritorio (Zona Central)

Para optimizar el espacio, el escritorio y la televisión comparten el mismo eje vertical, rompiendo la monotonía del clóset cerrado con un nicho abierto muy estético.

El Escritorio (Zona Baja)

- **Altura de la cubierta: 750 mm** fijos desde el suelo (altura antropométrica estándar).
- **Profundidad:** El clóset mide 600 mm, lo cual es **perfecto para un escritorio**. El usuario tendrá espacio suficiente para una laptop, teclado y monitor.
- **Ancho recomendado:** Mínimo **900 mm**, ideal **1200 mm** para que la silla quepa cómodamente y el usuario no se sienta encajonado.
- **Detalle técnico:** Debajo de la cubierta, tu plugin debe prever un espacio libre para las piernas (mínimo 650 mm de alto) y puede añadir una cajonera suspendida o un cajón de poca altura (70 mm) exclusivo para papelería o teclado.

El Centro de Entretenimiento / TV (Zona Media)

- **Ubicación:** Directamente sobre la cubierta del escritorio. La televisión va empotrada a un panel de fondo.
- **Altura del nicho para TV:** Aproximadamente **800 mm a 1000 mm** de espacio libre vertical antes de que comience el módulo sabanero superior. Esto permite pantallas de hasta 43 o 50 pulgadas.
- **El Panel de Fondo (Pasacables):** Tu plugin debe diseñar un fondo falso de melamina de 15 mm o 18 mm retirado unos **30 mm hacia adelante** del fondo real. Esto crea un espacio oculto para pasar los cables de la TV, laptops o cargadores sin que queden a la vista.

📊 Resumen de Medidas para tu Base de Datos (Módulos Combinados)

Cuando el usuario arrastre estos componentes desde la biblioteca de tu plugin, el código de Ruby debe aplicar estas envolventes por defecto:

Componente	Altura (H)	Profundidad (D)	Ancho (W) Sugerido	Notas Técnicas
Sabaneras (Techo)	500 mm	**600 mm**	Variable (Modulable)	Puertas con pistón a gas. Estantes fijos.
Nicho de TV	950 mm	**570 mm** (panel retraído)	1200 mm	Fondo doble para ocultar cableado.
Mesa de Escritorio	750 mm	**600 mm**	1200 mm	Espacio inferior libre para silla: > 700 mm.
Módulos de Ropa (Laterales)	1900 mm	**600 mm**	600 mm - 800 mm	Tubos de colgado o cajoneras internas.

🛠️ Reglas de Programación en Ruby para este Conjunto

Para que SketchUp renderice y despiece esto de manera profesional, programa las siguientes restricciones en tu plugin:

1. **Continuidad Estética (Vetas alineadas):** Si el panel del fondo de la TV y la cubierta del escritorio usan una melamina amaderada, la textura de la madera del panel de TV debe configurarse con **veta vertical**, y la mesa del escritorio con **veta horizontal** a lo largo del ancho del mueble.
2. **Módulos Independientes:** Aunque visualmente parezca un solo mueble, el código debe generar los costados (laterales) para el bloque del escritorio y costados propios para los bloques de clóset adyacentes. Al instalarse, se atornillan lateral con lateral. Esto evita que los tableros se pandeen por falta de soporte estructural.
3. **Ranuras de Ventilación:** Al albergar una TV y consolas o decodificadores en el nicho, el diseño modular debe incluir perforaciones circulares estandarizadas de **60 mm** (pasacables plásticos) en la cubierta del escritorio y en el panel falso de la TV.

¿Cómo te gustaría proceder? Podemos:

1. Diseñar el código en Ruby para generar el **módulo de escritorio con su panel doble pasacables** para la TV.
2. Definir cómo estructurar la **interfaz HTML (`UI::HtmlDialog`)** para que el usuario pueda activar o desactivar la zona de escritorio dentro del menú del clóset.

el clon debe ser capaz de combinar diferentes modulos a gusto del cliente

Para lograr que tu clon de **MelaStudio Pro** en SketchUp permita combinar módulos libremente a gusto del cliente, necesitas implementar un sistema de ensamble magnético o "Snap-to-Grid" guiado por código en Ruby.

El cliente o diseñador no debe perder tiempo alineando manualmente los módulos con las herramientas nativas de SketchUp; el plugin debe detectar la proximidad y "pegar" un módulo al lado del otro de forma exacta.

Aquí tienes la estrategia técnica de cómo programar esta funcionalidad de combinación dinámica:

---

1. El Concepto de "Puntos de Anclaje" (Anchor Points)

Cada módulo modular en tu librería (el clóset, el escritorio, la TV, las sabaneras) debe tener definidos internamente **puntos de conexión vectoriales** (coordenadas X, Y, Z relativas al origen del componente).

- **Punto Lateral Izquierdo (0, 0, 0):** Base inferior izquierda.
- **Punto Lateral Derecho (Ancho, 0, 0):** Base inferior derecha.
- **Punto Superior Izquierdo (0, 0, Alto):** Esquina superior izquierda (útil para encajar las sabaneras sobre el clóset).

Cuando el usuario arrastra un nuevo módulo desde tu interfaz HTML hacia el espacio de trabajo, el plugin debe leer la posición del cursor. Al acercarse a un módulo existente, calcula la distancia entre sus puntos de anclaje. Si la distancia es menor a un umbral (ej. 10 cm), el módulo "salta" (*snaps*) y se posiciona exactamente pegado al lateral o encima del otro.

---

2. Flujo Lógico de Combinación en SketchUp

Para que el usuario arme la combinación clóset + escritorio + sabanera de forma fluida, debes estructurar las funciones en Ruby de la siguiente manera:

`[Usuario arrastra módulo] ➔ [Ruby detecta colisión/cercanía] ➔ [Alineación automática] ➔ [Fusión de zócalos/ajuste de piezas]`

A. Inserción y Alineación Lateral (Eje X)

Cuando se coloca el módulo del escritorio al lado del módulo del clóset:

ruby

```ruby
# Pseudocódigo conceptual para la API de SketchUp
def alinear_modulo(modulo_existente, nuevo_modulo)
  pos_x_existente = modulo_existente.transformation.origin.x
  ancho_existente = modulo_existente.get_attribute("MelaStudio", "Ancho").mm

  # Nueva coordenada X para el módulo contiguo
  nueva_x = pos_x_existente + ancho_existente

  # Desplazar el nuevo módulo a la posición exacta
  nueva_transformacion = Geom::Transformation.new([nueva_x, 0, 0])
  nuevo_modulo.transformation = nueva_transformacion
end
```

Usa el código con precaución.

B. Apilamiento Vertical (Eje Z - Para Sabaneras y Maleteros)

Las sabaneras van arriba. El plugin debe reconocer que si el elemento es del tipo "Maletero", su coordenada Z base debe ser igual a la altura máxima del módulo inferior: $\text{Z\ del\ la\ Sabanera}=\text{Altura\ del\ Clóset Inf.}+\text{Z\ base\ del\ Clóset}$

---

3. El Desafío Técnico: "Zócalos Continuos y Tapas de Ajuste"

Cuando un cliente combina diferentes módulos a su gusto, el software debe resolver automáticamente dos problemas estéticos y de fabricación:

1. **Zócalo Unificado:** Si unes 3 módulos independientes, por defecto SketchUp dibujará 3 zócalos con 6 laterales cortando visualmente la base. Tu plugin debe tener una función de **"Unificar Base"**. Al activarse, oculta los zócalos individuales de los módulos intermedios y genera una sola pieza de melamina larga para el zócalo frontal que cubra todo el conjunto.
2. **Tapa de Cubierta Única:** Para el centro de entretenimiento o el escritorio, si el cliente junta un módulo de cajones y la mesa de trabajo, el software debe ser capaz de fusionar las cubiertas superiores en una sola placa continua de melamina para evitar uniones visibles donde se pueda filtrar suciedad o humedad.

---

4. Interfaz Web Interactiva (`UI::HtmlDialog`) para Combinar

En tu panel flotante del clon, diseña una sección llamada **"Configurador de Composición"**.

- **Paso 1:** El usuario ve un lienzo en blanco o una pared virtual que representa el espacio disponible del cliente.
- **Paso 2:** Mediante botones con iconos claros (ej. `[+ Clóset Corto]`, `[+ Escritorio]`, `[+ Torre de Cajones]`), el diseñador va haciendo clic.
- **Paso 3:** Cada clic inyecta el componente en SketchUp usando la lógica de puntos de anclaje que explicamos arriba, construyendo el mueble modular en tiempo real frente al cliente.

---

5. Recálculo Automático del Despiece Total

El mayor beneficio de esta combinación libre es que al terminar, el botón **"Generar Despiece"** de tu plugin recorrerá todo el árbol de componentes combinados en el modelo de SketchUp.

No importa cómo los haya mezclado el cliente: el software agrupará todas las piezas por espesor de melamina (ej. piezas de 18mm para estructura, piezas de 3mm para fondos) y exportará una lista de corte unificada lista para el optimizador.

¿Qué te parece este enfoque para combinar los módulos? Si quieres, podemos avanzar en:

1. Crear el código en Ruby para implementar la función de **"Snap" automático (alineación magnética)** al insertar un módulo al lado de otro.
2. Diseñar la estructura visual en **HTML/CSS para la barra lateral** donde se seleccionarán los diferentes módulos a combinar.

el clon primero debe identificar el espacio donde va a ir el muebla altura, largo y profundidad, luego definir cuantos modulos pueden entrar segun tipo y respetando estandades, por ejemplo en un espacio de 1200mm ancho y 2000mm de altura, 600mm de profundidad, el usuario quiere ala izquierda un modulo librero, al centro un modulo escritorio, ala derecha un modulo cajonera en la parte superior de la cajoneras van repisas, en la zona alta modulos cajoneras con pistrones
Para lograr esta automatización inteligente, tu clon de **MelaStudio Pro** debe incluir un **Asistente de Configuración Espacial** guiado por un algoritmo de distribución paramétrica. El flujo lógico debe ser: **Medir el Entorno ➔ Validar Viabilidad con Estándares ➔ Distribuir los Módulos de Forma Equitativa**.

A continuación, analizamos el caso específico que planteas (**1200 mm de ancho, 2000 mm de alto, 600 mm de profundidad**) y cómo debe procesarlo el backend de tu plugin en Ruby para SketchUp.

---

1. El Diagnóstico del Espacio (Validación de Estándares)

Antes de dibujar, el software debe evaluar si las dimensiones ingresadas por el usuario son técnicamente viables para albergar los tres módulos solicitados (*librero, escritorio, cajonera*).

- **Ancho Total Disponible: 1200 mm**.
- **Cantidad de módulos verticales solicitados:** 3 (Librero + Escritorio + Cajonera).
- **Cálculo Crítico de Estabilidad:** Un escritorio funcional requiere un espacio mínimo para las piernas de **600 mm** a **700 mm**. Si restamos eso de los 1200 mm, nos quedan entre 500 mm y 600 mm a repartir entre el librero y la cajonera (dando módulos extremadamente angostos de 250 mm o 300 mm cada uno).
- **Solución del Algoritmo:** El software debe alertar al usuario que el espacio es muy justo para tres columnas verticales independientes, o aplicar un patrón de **diseño compacto integrado** (donde el escritorio sea el eje central y el librero/cajonera se reduzcan a su mínima expresión comercial de 300 mm).

---

2. Desglose Estructural del Proyecto (Caso: 1200 x 2000 x 600 mm)

Respetando la solicitud del usuario y los estándares de carpintería en melamina, el motor del plugin dividirá las dimensiones de la siguiente manera:

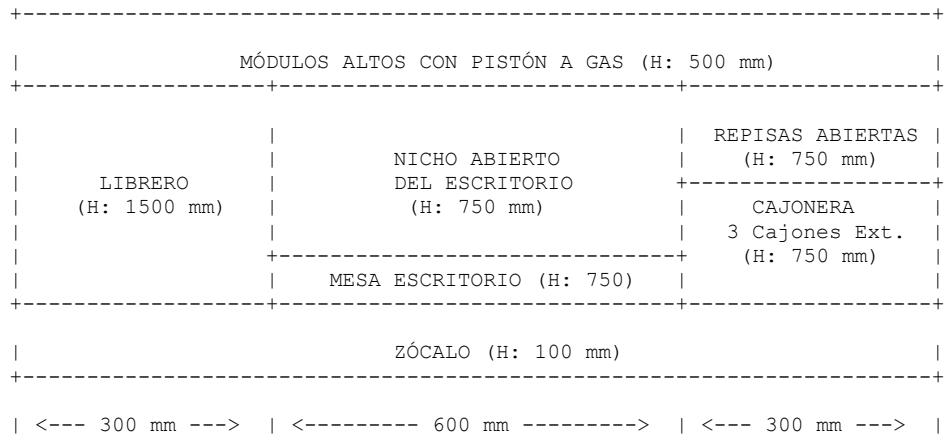
A. Distribución Horizontal (Anchos - Eje X)

Para que quepan los 3 elementos en 1200 mm, el plugin asignará las siguientes medidas por defecto:

- **Módulo Librero (Izquierda): 300 mm** de ancho. (Suficiente para libros o carpetas).
- **Módulo Escritorio (Centro): 600 mm** de ancho. (El estándar mínimo para que entre una silla de oficina pequeña).
- **Módulo Cajonera + Repisas (Derecha): 300 mm** de ancho. (Ancho estándar para cajoneras estrechas).
- *Fórmula interna de Ruby:* \(300\text{mm} + 600\text{mm} + 300\text{mm} = 1200\text{mm}\) (Encaje perfecto).

B. Distribución Vertical por Columna (Alturas - Eje Z)

El software segmentará las alturas de cada columna de forma independiente pero coordinada, coronando todo con los módulos superiores de pistón a gas:



1. **Zócalo Base: 100 mm** de alto para todo el mueble (protección contra humedad). Quedan 1900 mm disponibles.
2. **Zona Alta (Módulos con Pistón): 500 mm** de alto en la parte superior de todo el mueble. Son módulos que abren hacia arriba. Quedan 1400 mm para la zona media/baja.
3. **Columna Izquierda (Librero):** Un solo vano vertical de **1400 mm** de alto con repisas internas regulables cada 32 mm.
4. **Columna Central (Escritorio):**
   - Mesa de escritorio fija a **750 mm** del suelo (descontando el zócalo, la mesa queda sobre un nicho abierto).
   - Espacio libre sobre la mesa: **650 mm** (perfecto para una laptop o monitor pequeño de hasta 24 pulgadas).
5. **Columna Derecha (Cajonera + Repisas):**
   - **Cajonera Baja: 750 mm** de altura total (alineada con la mesa del escritorio) dividida en 3 cajones idénticos de 250 mm de frente cada uno.
   - **Repisas Superiores: 650 mm** de altura restante dividida por un estante intermedio abierto para decoración.

---

3. Lógica del Código en Ruby (Algoritmo de Reparto)

Cuando el usuario ingresa las medidas en el `UI::HtmlDialog`, tu backend debe ejecutar una función similar a esta para instanciar los componentes en SketchUp de forma automática:

ruby
```
def generar_mueble_inteligente(ancho_total, alto_total, prof_total)
  # 1. Definir constantes de estándares
  altura_zocalo = 100.mm
  altura_altos  = 500.mm
  grosor_placa  = 18.mm

  # 2. Calcular anchos según proporciones (Fórmula: 25% | 50% | 25%)
  ancho_librero   = ancho_total * 0.25
  ancho_escritorio = ancho_total * 0.50
  ancho_cajonera  = ancho_total * 0.25

  # 3. Invocar comandos de SketchUp para dibujar
  # Dibujar Zócalo Unificado de largo total
  dibujar_zocalo(ancho_total, altura_zocalo, prof_total)

  # Dibujar Bloque Superior con Pistones (X: 0 a ancho_total, Z: alto_total - altura_altos)
  dibujar_modulos_altos_piston(ancho_total, altura_altos, prof_total, alto_total)

  # Dibujar Módulos Inferiores posicionándolos en sus respectivas coordenadas X
  dibujar_librero(ancho_librero, alto_total - altura_altos - altura_zocalo, prof_total, 0)
  dibujar_escritorio(ancho_escritorio, 750.mm - altura_zocalo, prof_total, ancho_librero)
  dibujar_cajonera_con_repisas(ancho_cajonera,  alto_total  -  altura_altos  -  altura_zocalo,  prof_total,
ancho_librero + ancho_escritorio)
end
```
Usa el código con precaución.

---

4. Flujo de Experiencia de Usuario (UX) en tu Clon

Para que el proceso sea intuitivo para el diseñador, la interfaz gráfica del plugin debe guiarlo en tres pasos claros:

1. **Entrada de Dimensiones:** Tres cajas de texto simples: `Largo (mm)`, `Alto (mm)`, `Profundidad (mm)`. El usuario digita: `1200`, `2000`, `600`.
2. **Configurador de Columnas:** Una cuadrícula visual interactiva que detecta el espacio. El usuario selecciona la cantidad de columnas (3) y asigna el tipo mediante un menú desplegable a cada una:
   - Columna 1: *Librero*
   - Columna 2: *Escritorio*
   - Columna 3: *Cajonera Básica*
3. **Configurador de Adicionales:** Casillas de verificación (*checkboxes*) que dicen: *"¿Añadir módulos maleteros con pistón en la zona alta?"* y *"¿Añadir repisas sobre la cajonera?"*. Al marcarlos, el algoritmo recalcula las alturas internas automáticamente, actualiza el render en 3D dentro de SketchUp y actualiza la lista de despiece.

¿Qué te parece esta distribución matemática para el espacio de 1200 mm? Si estás de acuerdo, podemos:

1. Escribir el script detallado en Ruby para calcular y **dibujar la cajonera de 3 cajones** con sus respectivas holguras para las correderas.
2. Definir cómo estructurar la **alerta de viabilidad** si el usuario ingresa un ancho total demasiado pequeño (por ejemplo, menos de 1000 mm).

los 1200mm de ancho queda corto para el ejemplo pongamos 3000mm de ancho

Tienes toda la razón. Con **3000 mm (3 metros) de ancho**, el diseño respira por completo y pasa de ser un mueble comprimido a un **centro de entretenimiento, estudio y almacenaje de gama alta**. Con esta medida, cada módulo tendrá dimensiones ideales y sumamente cómodas para el usuario.

A continuación, reconfiguramos el algoritmo de tu plugin para distribuir estos 3 metros respetando las solicitudes y los estándares de fabricación:

---

1. Nueva Distribución Horizontal (Anchos - Eje X)

Al disponer de 3000 mm, podemos dividir el espacio en **3 columnas principales de 1000 mm cada una** o ajustar las proporciones para darle más protagonismo al escritorio. Una distribución óptima y equilibrada para el código sería:

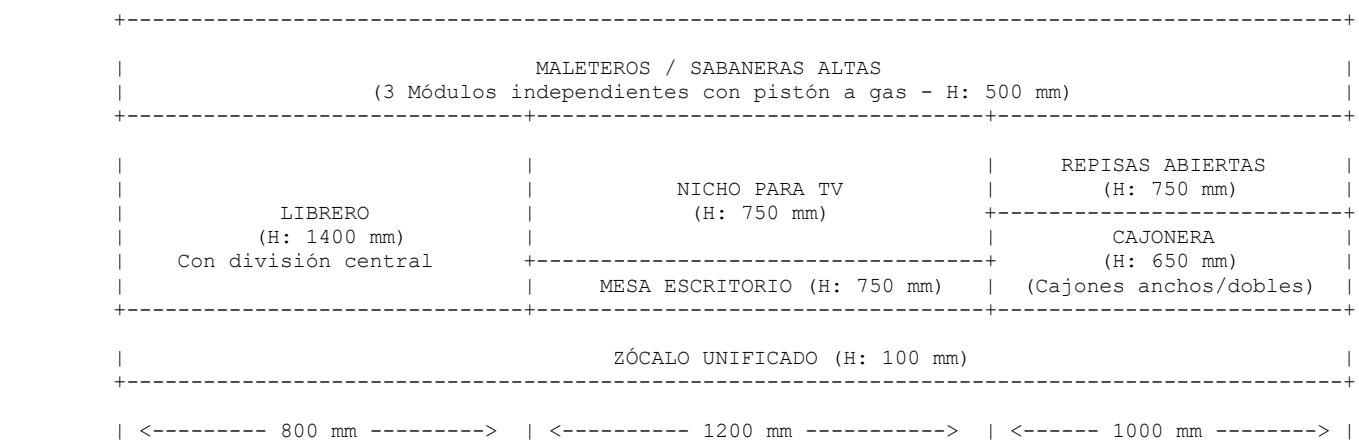
- **Módulo Librero (Izquierda): 800 mm** de ancho. (Permite estantes amplios y estéticos. Tu plugin debe añadir una división vertical central aquí para evitar que la melamina se pandee).
- **Módulo Escritorio / TV (Centro): 1200 mm** de ancho. (Espacio premium. Permite una silla de oficina ergonómica grande, una TV de hasta 50-55 pulgadas y total comodidad para trabajar).
- **Módulo Cajonera + Repisas (Derecha): 1000 mm** de ancho. (Se puede estructurar como una cajonera doble o una cajonera de 500 mm combinada con un espacio de guardado cerrado).

\(\text{Fórmula: }800\text{\ mm}+1200\text{\ mm}+1000\text{\ mm}=3000\text{\ mm\ (Encaje\ exacto)}\)

---

2. Desglose Estructural Completo (Matriz del Plugin)

Con la altura de **2000 mm** y profundidad de **600 mm**, la matriz tridimensional que tu script de Ruby debe dibujar automáticamente en SketchUp queda así:



▲ **Zona Alta: Módulos con Pistón a Gas (H: 500 mm / D: 600 mm)**

- Para cubrir los 3 metros sin que las puertas sean gigantescas, el plugin creará **3 módulos superiores alineados** con las columnas de abajo (uno de 800 mm, uno de 1200 mm y uno de 1000 mm).
- El de 1200 mm llevará una división interna o dos puertas independientes con pistón para soportar correctamente el peso del techo.

👈 **Columna Izquierda: Librero (W: 800 mm / H: 1400 mm / D: 300-400 mm)**

- *Detalle de diseño:* Aunque el mueble tiene 600 mm de fondo, un librero de 600 mm es incómodo porque los libros se pierden al fondo. El algoritmo de tu plugin debe **retraer el fondo del librero** para que las repisas tengan solo **300 mm de profundidad útil**, dejando un doble fondo oculto o un nicho estético.

💻 **Columna Central: Escritorio e Inteligencia de TV (W: 1200 mm / H: 1400 mm / D: 600 mm)**

- **Mesa:** Una robusta cubierta de melamina de 18 mm (o doble espesor de 36 mm para mayor categoría) a **750 mm** del suelo.
- **Nicho de TV:** Un área libre de \(1200 \times 750\text{ mm}\). Incluye el panel falso pasacables que mencionamos antes, permitiendo colgar la pantalla limpiamente.

👉 **Columna Derecha: Cajonera + Repisas (W: 1000 mm / H: 1400 mm / D: 600 mm)**

- **Cajonera Baja (H: 650 mm):** Al tener 1000 mm de ancho, diseñar cajones de un metro de largo puede hacer que las correderas sufran por el peso. Tu algoritmo tiene dos opciones excelentes:
  1. Dividir la zona en **2 columnas de cajones de 500 mm** cada una (la opción más segura y comercial).
  2. Usar correderas industriales pesadas (*Heavy Duty*) si el cliente quiere cajones horizontales gigantes de 1000 mm.
- **Repisas Superiores (H: 750 mm):** Estantes abiertos de 1000 mm de ancho para organizar sábanas adicionales, cajas organizadoras o elementos decorativos.

---

3. Actualización del Algoritmo en Ruby para SketchUp

Tu código ahora procesará estos bloques masivos de manera modular. Aquí tienes cómo estructurar las coordenadas de inserción (X, Y, Z) para cada componente:

ruby

```
# Coordenadas de inicio para posicionar los componentes en SketchUp
grosor_lateral = 18.mm

# 1. Instanciar Librero (Origen en X = 0)
pos_librero = Geom::Transformation.new([0, 0, 100.mm])
# dibujar_librero(ancho: 800.mm, alto: 1400.mm, prof: 600.mm, transformacion: pos_librero)

# 2. Instanciar Escritorio (Origen en X = 800mm)
pos_escritorio = Geom::Transformation.new([800.mm, 0, 100.mm])
# dibujar_escritorio(ancho: 1200.mm, alto: 1400.mm, prof: 600.mm, transformacion: pos_escritorio)

# 3. Instanciar Cajonera (Origen en X = 2000mm)
pos_cajonera = Geom::Transformation.new([2000.mm, 0, 100.mm])
# dibujar_cajonera_repisas(ancho: 1000.mm, alto: 1400.mm, prof: 600.mm, transformacion: pos_cajonera)
```

Usa el código con precaución.

🌟 El Toque Premium para tu Software: "Validación de Pandeo"

Con estas nuevas medidas de 800 mm y 1000 mm en repisas y cajones, tu clon de MelaStudio Pro destacará si implementa una **regla de ingeniería de materiales automática**:

- **Regla de Alerta:** Si una repisa de melamina de 18 mm supera los **800 mm de ancho sin apoyo**, el software debe pintar la pieza de color naranja o lanzar una advertencia en la interfaz web: *"Peligro de pandeo: Se sugiere añadir un soporte central o dividir el módulo"*. Esto evitará que tus usuarios fabriquen muebles que terminen doblándose con el tiempo.

Con estas dimensiones ideales listas, ¿cómo prefieres continuar?

1. Programamos la lógica en Ruby para **dividir automáticamente la cajonera de 1000 mm en dos columnas de 500 mm** y calcular sus piezas.
2. Diseñamos la interfaz en **HTML (`UI::HtmlDialog`)** para que el usuario pueda escribir "3000" y ver cómo se actualiza la cuadrícula de las 3 columnas.

el clon debe ser dinamico con el espacio identificado el usuario puede incluir 2, 3, 4 o mas modulos y adecuarlos segun sus exigencias

Para lograr que tu clon sea **completamente dinámico**, debes programar un algoritmo de distribución proporcional y matricial en Ruby. El software no debe tener configuraciones fijas; debe actuar como un lienzo inteligente que reacciona a dos variables principales introducidas por el usuario: **El Espacio Total Available (W)** y la **Cantidad de Módulos (N)**.

A continuación, te muestro cómo estructurar la lógica matemática y de programación para que tu plugin de SketchUp calcule, valide y dibuje cualquier cantidad de módulos de forma 100% dinámica.

---

1. La Ecuación Core de Distribución Dinámica

Cuando el usuario define el espacio y la cantidad de módulos, el software debe calcular instantáneamente el **Ancho Base por Módulo (\(W_{m}\))**. Para que el diseño sea técnicamente correcto, la fórmula debe descontar el grosor de los laterales intermedios para que el mueble no se pase de la medida del cuarto:

\(\text{Ancho\ Módulo }(W_{m})=\frac{\text{Ancho\ Total}-(\text{Espesor\ Tablero}\times (\text{Cantidad\ Módulos}+1))}{\text{Cantidad\ Módulos}}\)

Si el usuario ingresa **3000 mm de ancho**, tablero de **18 mm** y quiere **4 módulos**: \(\text{Ancho\ Módulo }=\frac{3000-(18\times 5)}{4}=\frac{2910}{4}=\mathbf{727.5}\text{\ mm}\)

El motor en Ruby asignará automáticamente \(727.5\text{ mm}\) de espacio libre a cada columna, y posicionará cada módulo en su coordenada X exacta usando un bucle (`loop`):

ruby

```
# Ejemplo de bucle dinámico de posicionamiento en la API de SketchUp
```

```
pos_x_acumulada = espesor_tablero

columnas_solicitadas.each_with_index do |tipo_modulo, i|
  # Calcular punto de inserción para el módulo actual
  transformacion = Geom::Transformation.new([pos_x_acumulada, 0, altura_zocalo])

  # Invocar la función que dibuja el tipo de mueble elegido (Librero, Escritorio, etc.)
  dibujar_modulo_dinamico(tipo_modulo, ancho_modulo, alto_util, prof_total, transformacion)

  # Avanzar la coordenada X para el siguiente módulo (añadiendo el ancho y el siguiente lateral)
  pos_x_acumulada += ancho_modulo + espesor_tablero
end
```
Usa el código con precaución.

---

2. El Sistema de Bloques y Configuración por "Columnas"

En la interfaz web (`UI::HtmlDialog`), el usuario definirá la configuración como si fuera una matriz o una tabla. Cada columna es un objeto independiente que el usuario puede personalizar.

Ejemplo con 4 Módulos en 3000 mm de Ancho:

Si el cliente decide pasar de 3 a 4 módulos para tener más variedad, la interfaz le permitirá asignar un rol a cada columna, dividiendo el espacio de la siguiente manera:

- **Módulo 1 (0 a 75 cm):** Clóset con barra para colgar ropa.
- **Módulo 2 (75 a 150 cm):** Librero abierto con repisas regulables.
- **Módulo 3 (150 a 225 cm):** Escritorio con panel de TV.
- **Módulo 4 (225 a 300 cm):** Cajonera doble con repisas superiores.

El script de Ruby leerá este arreglo (ej. `['closet', 'librero', 'escritorio', 'cajonera']`) y adaptará el diseño interior de cada sección al ancho dinámico calculado (\(727.5\text{ mm}\)).

---

3. Filtro Inteligente de Reglas y Restricciones Técnicas

Para evitar que el usuario diseñe algo que no se pueda fabricar (por ejemplo, poner 8 módulos en un espacio de 1200 mm, dando columnas de 10 cm), tu plugin debe procesar una **Matriz de Validación de Estándares**:

Tipo de Módulo	Ancho Mínimo	Ancho Máximo	Comportamiento Dinámico Automatizado
Escritorio	600 mm	1400 mm	Si supera los 1000 mm, el plugin añade un refuerzo horizontal bajo la mesa para evitar que se doble.
Cajonera	350 mm	1000 mm	Si el ancho dinámico supera los **700 mm**, el código divide automáticamente el espacio en dos filas de cajones (cajonera doble).
Librero / Repisas	300 mm	800 mm	Si el ancho dinámico calculado supera los **800 mm**, el script inyecta un parante vertical divisorio en medio del librero.
Puertas Batientes	300 mm	600 mm	Si la columna mide más de 600 mm, el software cambia automáticamente de 1 puerta single a **2 puertas dobles**.

---

4. Interfaz de Usuario Dinámica (UX en HTML/CSS)

Para que el cliente juegue con las opciones, la ventana de tu plugin debe verse sumamente interactiva:

1. **Paso 1: Fronteras del Espacio:** El usuario escribe el ancho, alto y profundidad total.

2. **Paso 2: Generador de Columnas:** Un selector numérico `[ - ] 3 [ + ]`. Al hacer clic en el +, la interfaz añade visualmente una nueva columna en la pantalla.
3. **Paso 3: Selector de Tipos:** Cada columna cuenta con un menú desplegable interactivo. Si el usuario cambia la Columna 2 de "Librero" a "Escritorio", el modelo 3D de SketchUp se borra y se vuelve a dibujar en menos de un segundo con el nuevo componente.
4. **Paso 4: Techo Unificado:** Una casilla global que consulta: *¿Dividir maleteros superiores por columna o crear puertas corridas completas?*
Este enfoque le dará a tu software la misma potencia que los programas de diseño de cocinas y clósets europeos de alta gama. Para avanzar con el desarrollo:

1. ¿Te gustaría que escribamos la función completa en Ruby para **calcular y alertar si el ancho de los módulos viola las reglas mínimas o máximas**?
2. ¿Prefieres que diseñemos el código para el **bucle de SketchUp que borra el mueble anterior y dibuja el nuevo** cada vez que el usuario cambia una opción?
tambien debe permitir modulos zona baja, zona media y zona superior, en el caso de una cocina o centro de entretenimiento
Para dotar a tu clon de **MelaStudio Pro** de la máxima flexibilidad del mercado, el algoritmo debe evolucionar de un sistema de columnas simples a una **matriz bidimensional (grilla de filas y columnas)**. Esto permitirá dividir el espacio de manera dinámica no solo a lo ancho, sino también en **Zona Baja, Zona Media y Zona Superior**.

Esta estructura tridimensional es la base perfecta para diseñar tanto **cocinas integrales** como **centros de entretenimiento avanzados**.

---

1. El Concepto de "Sub-Modulación Vertical"

En lugar de que una columna sea un solo bloque de piso a techo, tu plugin en Ruby debe segmentar cada columna en 3 coordenadas de altura estandarizadas:

```
+-------------------------------------------------------------------------+

|                  ZONA SUPERIOR (Altos / Alacenas / Maleteros)           |
|                  Altura: Variable (ej. 500 - 700 mm) | Prof: 300-350 mm |
+-------------------------------------------------------------------------+

|                  ZONA MEDIA (Salpicadero / Nicho TV / Microondas)       |
|                  Altura: Variable (ej. 500 - 650 mm) | Prof: Libre/Abierto|
+-------------------------------------------------------------------------+

|                  ZONA BAJA (Módulos Base / Cajoneras / Lavavajillas)     |
|                  Altura: 750 - 850 mm (más zócalo)   | Prof: 580-600 mm |
+-------------------------------------------------------------------------+
```

2. Comportamiento Inteligente según el Tipo de Proyecto

Cuando el usuario selecciona el "Modo de Mueble" en la interfaz de tu plugin, el motor en Ruby debe aplicar reglas de profundidad y altura automáticas para que el diseño tenga sentido técnico:

🍳 Caso A: Cocina Modular

Si el usuario diseña una cocina, al dividir el espacio, el software configurará automáticamente:

- **Zona Baja (Profundidad 580 mm):** Permite al usuario insertar módulos dinámicos como: *Cajonera para ollas, Módulo de puertas para fregadero, o Nicho para horno empotrado*.
- **Zona Media (Profundidad 0 / Abierta):** Deja el espacio libre de 550 mm para el salpicadero (enchape), pero permite al usuario inyectar módulos específicos como un *Porta-microondas suspendido*.
- **Zona Superior (Profundidad 320 mm):** Reduce automáticamente la profundidad para que el usuario no se golpee la cabeza. Permite insertar: *Alacenas de dos puertas, Módulo para campana extractora o escurridor de platos*.

📺 Caso B: Centro de Entretenimiento (Home Theater)

Si se configura para sala o dormitorio, la matriz cambia sus reglas:

- **Zona Baja (Profundidad 450 mm):** Módulos bajos con puertas abatibles hacia abajo o cajones flotantes para consolas de videojuegos y decodificadores.
- **Zona Media (Profundidad Variable):** El software genera por defecto un panel falso vertical para colgar la TV y permite al usuario añadir *repisas flotantes decorativas* a la izquierda o derecha.
- **Zona Superior (Profundidad 300 mm):** Vitrinas con puertas de vidrio o módulos cerrados tipo maletero para almacenamiento secundario.

---

3. Implementación de la Lógica en Ruby (La Matriz de Inserción)

Para programar esto de manera dinámica, tu bucle de SketchUp debe calcular las coordenadas de origen `[X, Y, Z]` para cada celda de la matriz. Aquí tienes el esquema de cómo tu código posicionará un módulo en cualquier nivel:

ruby
```
# Parámetros calculados dinámicamente por el algoritmo
ancho_columna = 600.mm
altura_zocalo = 100.mm

# 1. Coordenadas para la ZONA BAJA
z_baja = altura_zocalo
y_baja = 0 # Alineado al frente total (Profundidad: 600mm)
pos_baja = Geom::Transformation.new([x_actual, y_baja, z_baja])
# dibujar_modulo_bajo(tipo, ancho_columna, altura_baja, 600.mm, pos_baja)

# 2. Coordenadas para la ZONA MEDIA
z_media = altura_zocalo + altura_baja
y_media = 250.mm # Retraído si es repisa, o 0 si es panel de TV
pos_media = Geom::Transformation.new([x_actual, y_media, z_media])
# dibujar_modulo_medio(tipo, ancho_columna, altura_media, prof_media, pos_media)

# 3. Coordenadas para la ZONA SUPERIOR
z_superior = altura_zocalo + altura_baja + altura_media
y_superior = 280.mm # Retraído a 320mm de fondo (600 - 280 = 320) para alacenas de cocina
pos_superior = Geom::Transformation.new([x_actual, y_superior, z_superior])
# dibujar_modulo_alto(tipo, ancho_columna, altura_superior, 320.mm, pos_superior)
```
Usa el código con precaución.

---

4. Interfaz Gráfica de Usuario (UI Matriz Dinámica)

En el panel `UI::HtmlDialog` de tu clon, la experiencia de usuario debe estructurarse como un **diseñador de bloques interactivo**:

1. **Cuadrícula Visual:** El usuario ve un lienzo dividido en filas (Baja, Media, Alta) y columnas según el número que haya elegido (ej. 3 columnas).
2. **Asignación con un Clic:** El usuario hace clic en cualquier celda de la cuadrícula. Se abre un menú flotante que le muestra **únicamente las opciones viables para esa zona**.
   - *Si hace clic en Columna 2 - Zona Baja:* El software le ofrece: Cajonera, Puertas, Fregadero.
   - *Si hace clic en Columna 2 - Zona Superior:* El software le ofrece: Alacena, Campana, Estante abierto.
3. **Sincronización Inmediata:** Al seleccionar el tipo de módulo, el script de Ruby borra el espacio correspondiente en SketchUp y genera la geometría exacta con sus divisiones internas, herrajes y tapacantos.

Con esta estructura matricial (Columnas + Zonas), tu plugin tendrá la capacidad técnica de competir con software profesional europeo. ¿Hacia dónde enfocamos el siguiente paso técnico?

1. Programamos el código en Ruby para **calcular el cambio de profundidad automático** entre el mueble bajo de cocina y la alacena superior.
2. Desarrollamos la estructura del archivo **HTML/CSS para crear la cuadrícula interactiva** (Baja, Media, Alta) en la interfaz del plugin.

Las respuestas de la IA pueden incluir errores. Para obtener asesoramiento legal, consulta a un profesional. Más información
debe estar preparado para cualquier tipo de muebles zapateras, vestidores, roperos, closet, comodas, etc
Para que tu clon de **MelaStudio Pro** sea el software definitivo en el mercado, la clave no es programar un mueble específico para cada categoría, sino diseñar un **Motor de Comportamiento Arquitectónico y Clasificación de Módulos**.

Dado que el software trabajará con una matriz bidimensional (columnas divididas en zona baja, media y superior), cada tipo de mueble (zapatera, cómoda, vestidor, clóset) se convertirá simplemente en una **configuración de subcomponentes y restricciones técnicas** aplicada a esa grilla.

A continuación, te muestro cómo debe interpretar el motor de tu plugin cada tipo de mueble para dibujarlo de forma 100% automatizada:

---

1. El Diccionario de Tipologías (Configurador por Celdas)

Cuando el usuario haga clic en una celda de la matriz (ya sea baja, media o superior), el plugin consultará una base de datos de submódulos. Aquí tienes cómo el código en Ruby debe configurar cada tipología solicitada:

Zapateras (Módulos de Calzado)

- **Comportamiento Físico:** Requieren repisas con menor espacio vertical (entre **150 mm y 200 mm** de altura por vano).
- **Variantes Técnicas en tu Plugin:**

1. *Zapatera de Repisas Fijas/Regulables:* El software calcula divisiones horizontales espaciadas equitativamente.
2. *Zapatera Extraíble:* El código no dibuja repisas comunes; dibuja bandejas con un espacio lateral de **13 mm** a cada lado para albergar correderas telescópicas.
3. *Zapatera Inclinada (Guanteras):** El script de Ruby rota las repisas internas entre **15° y 30°** e incluye un listón frontal de 20 mm de alto (*freno*) para que los zapatos no se deslicen.

drawers Cómodas y Cajoneras

- **Comportamiento Físico:** Módulos diseñados exclusivamente para la **Zona Baja** o **Zona Media** (máximo hasta 1200 mm de altura total para que el último cajón sea accesible).
- **Lógica de Despiece:** Al elegir "Cómoda", el algoritmo toma el ancho libre, resta **26 mm** (13 mm por lado para las correderas estándar) y calcula el tamaño exacto de la caja del cajón. El frente del cajón se calcula restando **2 mm de luz perimetral** por lado para evitar rozamientos al abrir.

Vestidores (Walk-in Closets) y Roperos/Clósets

- **Vestidores (Estructuras Abiertas):** El plugin desactiva la generación de frentes (puertas) y tapacantea automáticamente todos los bordes frontales expuestos con tapacanto grueso (2 mm) debido al alto tránsito.
- **Roperos / Clósets (Estructuras Cerradas):** El software activa la capa (*layer*) de puertas.
  - *Si es puerta batiente:* Añade bisagras automáticas cada 700 mm de altura de puerta.
  - *Si es puerta corrediza:* El código desplaza todo el cuerpo interno del mueble **60 mm a 90 mm hacia atrás** para dejar espacio libre al riel y sistema de carros guías en el frente.

---

2. Matriz de Parámetros Globales por Tipología

Para que el plugin dibuje el mueble adecuado al instante, tu backend en Ruby debe inicializar estas plantillas de configuración:

Tipo de Mueble	Zonas Permitidas	Profundidad Sugerida	Componentes Internos por Defecto
Zapatera	Baja / Media	350 mm a 400 mm	Repisas cortas inclinadas o bandejas extraíbles.
Cómoda	Baja	450 mm a 550 mm	Cajones altos (frentes de 200-300 mm).
Clóset (Colgado)	Baja / Media	**600 mm (Obligatorio)**	Tubo ovalado de aluminio + soportes laterales.
Librero / Despensa	Alta / Media / Baja	300 mm a 400 mm	Repisas regulables con perforaciones Sistema 32 mm.

---

3. Estructuración del Código en Ruby (Polimorfismo de Módulos)

Para evitar un código gigantesco y difícil de mantener, debes usar programación orientada a objetos en Ruby. Creas una clase padre llamada `ModuloComponente` y subclases para cada tipo de mueble:

ruby

```ruby
class ModuloComponente
  def initialize(ancho, alto, profundidad, transformacion)
    @w, @h, @d, @t = ancho, alto, profundidad, transformacion
    @grosor = 18.mm
  end
end

# Subclase específica para Zapateras
class ModuloZapatera < ModuloComponente
  def dibujar(entidades)
    # 1. Dibujar la estructura exterior (Estructura Base)
    dibujar_carcasa(entidades)

    # 2. Algoritmo específico: Repisas espaciadas cada 180mm
```

```
    cantidad_repisas = (@h / 180.mm).to_i
    espacio_exacto = @h / cantidad_repisas

    (1...cantidad_repisas).each do |i|
      z_repisa = i * espacio_exacto
      # Código de SketchUp para meter una placa horizontal en la coordenada z_repisa
    end
  end
end
```

Usa el código con precaución.

Cuando el usuario cambie la celda a "Zapatera", tu plugin simplemente ejecutará: `ModuloZapatera.new(ancho, alto, profundidad, posicion).dibujar(Sketchup.active_model.entities)`.

---

4. UX Universal: El Selector por Iconos

En la interfaz gráfica de tu clon, el usuario no verá nombres aburridos. Diseñarás un catálogo visual estructurado por **categorías de uso**:

1. **Pestaña Dormitorio:** Iconos de Clóset de colgado, Torres de cajones (Cómodas), Zapateras y Maleteros.
2. **Pestaña Cocina/Servicio:** Iconos de Alacenas, Módulos de ollas, Portamicondas y Despensas extraíbles.
3. **Pestaña Sala/Oficina:** Iconos de Escritorios, Libreros, Paneles de TV y Vitrinas.

El usuario solo tendrá que arrastrar el icono del mueble deseado a la celda correspondiente (Baja, Media o Alta) y el software se encargará de recalcular todo el despiece, consumo de tableros y presupuesto de herrajes de manera unificada.

Con esta arquitectura, tu clon estará preparado para diseñar cualquier mueble del mercado. ¿Cuál de estos pasos técnicos prefieres que desarrollemos ahora?

1. El script en Ruby para **generar los cajones de una cómoda** calculando automáticamente los descuentos de las correderas telescópicas (13 mm por lado).
2. La estructura del menú en **HTML/JS para organizar el catálogo de iconos** por categorías (Dormitorio, Cocina, Sala).

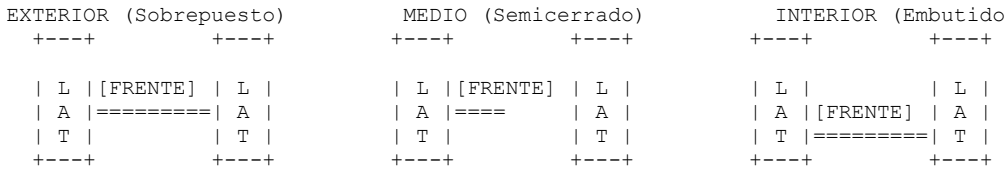
las puestas debe ser configurable, interior, media, exterior lo mismo que las cajoneras

Para que tu clon de **MelaStudio Pro** compita al más alto nivel, la configuración de frentes (puertas y cajones) es un pilar crítico. En la carpintería industrial de melamina, la posición del frente respecto a los laterales del mueble define por completo el tipo de herraje (bisagras y correderas) y el cálculo exacto del **despiece**.

Debes programar tu motor en Ruby para SketchUp para que acepte tres tipos de montaje de frentes (puertas y frentes de cajón): **Exterior (Instalado/Sobrepuesto)**, **Medio (Semicerrado/Doble lateral)** e **Interior (Embutido/Incrustado)**.

Los 3 Tipos de Montaje y sus Fórmulas de Despiece

Cuando el usuario cambie la configuración en la interfaz HTML, tu script de Ruby debe aplicar fórmulas geométricas estrictas para redibujar la pieza en SketchUp y calcular su tamaño real:



1. Frente Exterior (Sobrepuesto / Overlay)

El frente cubre completamente el canto frontal de los laterales, piso y techo del mueble. Es el sistema más común porque oculta las imperfecciones de la estructura.

- **Cálculo del Frente (Ancho):** Ancho Total del Módulo - (2 × Luz Perimetral).
  - *Ejemplo:* Si el módulo mide 600 mm y usas una luz estándar de 2 mm por lado, el frente mide **596 mm**.
- **Requisito de Herraje (Bisagra): Bisagra Recta (0 mm Coza/Cazoleta)**.
- **Posición en SketchUp (Eje Y):** Se dibuja justo por fuera del mueble (Y = Profundidad del Módulo).

2. Frente Medio (Semicerrado / Half Overlay)

Se utiliza cuando dos módulos comparten el mismo lateral intermedio (tabique) o cuando por diseño se quiere dejar ver la mitad del espesor del lateral. El frente solo cubre la mitad del canto de la melamina.

- **Cálculo del Frente (Ancho):** Ancho Libre Interno + Espesor Lateral - (2 × Luz Perimetral).
  - *Ejemplo:* Para un lateral de 18 mm, el frente se monta solo 9 mm sobre el canto.
- **Requisito de Herraje (Bisagra): Bisagra Codo 9 (Semicurva)**.
- **Posición en SketchUp (Eje Y):** Igual al exterior, alineado al frente del mueble.

3. Frente Interior (Embutido / Inset)

El frente queda completamente "escondido" o enrasado dentro de la estructura del mueble. Los cantos de los laterales, piso y techo quedan totalmente expuestos en la fachada.

- **Cálculo del Frente (Ancho):** Ancho Libre Interno - (2 × Luz Perimetral).
  - *Ejemplo:* Si el espacio interno libre es de 564 mm, con 2 mm de luz a cada lado, el frente mide **560 mm**.
- **Requisito de Herraje (Bisagra): Bisagra Codo 18 (Curva o Alta)**.
- **Posición en SketchUp (Eje Y):** El script de Ruby debe empujar el frente hacia adentro del mueble exactamente el espesor de la melamina (Y = Profundidad del Módulo - Espesor del Frente).

🗄️ Aplicación Especial en Cajoneras

Para las cajoneras, esta propiedad modifica tanto el **frente estético** (frente falso) como la **profundidad de los cajones internos**:

- **Si es Exterior o Medio:** La caja interior del cajón puede aprovechar casi el 100% de la profundidad del mueble. Si el mueble mide 500 mm de fondo, puedes usar una corredera y un cajón de 450 mm o 500 mm.
- **Si es Interior:** El código de tu plugin debe obligatoriamente **restar el espesor de la melamina (ej. 18 mm)** a la profundidad útil interna del mueble antes de elegir la corredera telescópica. Si no lo haces, cuando el cajón se cierre, el frente chocará y sobresaldrá del mueble, arruinando el diseño embutido.

💻 Estructura de Datos en Ruby (Atributos dinámicos)

Cada vez que el usuario use el selector de tu plugin para cambiar el tipo de frente, el backend debe registrar estos metadatos para el despiece automático y para la representación 3D:

ruby

```ruby
# Al instanciar un Frente (Puerta o Cajón)
frente.set_attribute("MelaStudio_Clon", "Tipo_Montaje", "Interior") # Opciones: "Exterior", "Medio", "Interior"
frente.set_attribute("MelaStudio_Clon", "Luz_Frente", 2.mm)

# El script lee el atributo para posicionar la pieza en SketchUp
def calcular_posicion_y(modulo_profundidad, tipo_montaje, espesor_frente)
  case tipo_montaje
  when "Exterior", "Medio"
    return modulo_profundidad # Se dibuja al ras exterior
  when "Interior"
    return modulo_profundidad - espesor_frente # Se empotra dentro del mueble
  end
end
```

Usa el código con precaución.

🎨 Experiencia de Usuario (UI Dinámica)

En la barra lateral de tu plugin (`UI::HtmlDialog`), al seleccionar cualquier módulo que contenga frentes, debes renderizar un control triple interactivo de tipo *Segmented Control* o botones con iconos descriptivos:

```
Montaje del Frente:
[   Exterior   ]  [    Medio    ]  [ * Interior * ]
```

Al hacer clic en cualquiera de ellos, SketchUp redibuja instantáneamente el frente en la posición exacta, actualiza el tipo de bisagra/corredera en el presupuesto y modifica las dimensiones milimétricas en la lista de corte final.

¡Esta funcionalidad le dará a tu clon precisión industrial! Para continuar con el desarrollo, ¿en cuál nos enfocamos?

1. Escribir el script en Ruby para **calcular el despiece exacto de una cajonera** (Caja interna + Frente) alternando entre montaje Exterior e Interior.
2. Diseñar los **iconos vectoriales (SVG) y el HTML** para este selector triple de montaje en la interfaz del plugin.

la separacion de luz debe ser configurable 1, 2, 3 mm
Hacer que la **separación de luz (holgura)** sea configurable (1, 2 o 3 mm) es el toque final para que tu clon de **MelaStudio Pro** pase de ser un juguete de diseño a una **herramienta de producción real**. En la fabricación de muebles, los carpinteros varían esta luz según el tipo de tapacanto que usen (delgado de 0.5 mm o grueso de 2 mm) y la precisión de sus máquinas de corte.

Al parametrizar esta variable en Ruby, el motor calculará milimétricamente las dimensiones de los frentes (puertas y cajoneras) y actualizará el render 3D en SketchUp en tiempo real.

📐 Fórmulas Matemáticas con Luz Configurable (`@luz`)

Tu script en Ruby debe almacenar la luz seleccionada en una variable global o de instancia (ej. `@luz = 2.mm`). Al dibujar las puertas o frentes de cajón, aplicará las siguientes condiciones geométricas:

1. Frentes Únicos (Una sola puerta o un solo cajón por vano)

- **Ancho del Frente (Exterior):** `Ancho_Módulo - (2 × @luz)`
- **Ancho del Frente (Interior):** `Ancho_Interno_Libre - (2 × @luz)`
- **Altura del Frente:** `Altura_Vano - (2 × @luz)`

2. Frentes Dobles (Dos puertas batientes que cierran al centro en el mismo vano)

Aquí el cálculo es crítico: debes dejar luz en los extremos izquierdo y derecho, pero también **en el encuentro central** de ambas puertas para que no choquen al cerrar.

- **Ancho de Cada Puerta (Exterior):** `(Ancho_Módulo - (3 × @luz)) / 2`
- **Ancho de Cada Puerta (Interior):** `(Ancho_Interno_Libre - (3 × @luz)) / 2`

💻 Implementación del Algoritmo en Ruby para SketchUp

Cuando el usuario cambia el valor de la luz en la interfaz web (`UI::HtmlDialog`), el backend de tu plugin procesa el rediseño usando esta lógica paramétrica:

ruby

```ruby
# Método para calcular el despiece de frentes de forma dinámica
def calcular_frente_dinamico(ancho_vano, alto_vano, tipo_montaje, cantidad_puertas, luz_usuario)
  # luz_usuario vendrá como 1, 2 o 3 desde el HTML
  @luz = luz_usuario.mm
  @espesor_lateral = 18.mm # Ejemplo

  # Ajustar el ancho base según el tipo de montaje
  ancho_base = (tipo_montaje == "Interior") ? ancho_vano - (@espesor_lateral * 2) : ancho_vano

  if cantidad_puertas == 1
    ancho_final = ancho_base - (2 * @luz)
  elsif cantidad_puertas == 2
    ancho_final = (ancho_base - (3 * @luz)) / 2
  end
```

```
  alto_final = alto_vano - (2 * @luz)

  return { ancho: ancho_final, alto: alto_final }
end
```
Usa el código con precaución.

📊 Comparativa de Impacto en el Despiece (Caso: Módulo de 600 mm × 700 mm)

Si tu usuario diseña un módulo exterior de una sola puerta, mira cómo cambia la lista de corte final que exportará tu plugin según la luz seleccionada:

Luz Configurada	Dimensión en SketchUp y Lista de Corte	Margen de Ajuste Real en Taller
1 mm	**598 mm × 698 mm**	Ideal para tapacanto delgado (0.5 mm) y cortes CNC perfectos.
2 mm	**596 mm × 696 mm**	**El estándar de la industria**. Absorbe pequeñas imperfecciones de escuadra.
3 mm	**594 mm × 694 mm**	Recomendado si usas tapacanto grueso de 2 mm en los frentes.

🎨 Diseño de la Interfaz de Usuario (UI en HTML/CSS)

En el panel flotante de tu clon, añade este control directo en la sección de **"Ajustes de Fábrica"** o **"Configuración de Frentes"**:

html
```
<!-- Estructura HTML en tu UI::HtmlDialog -->
<div class="control-group">
  <label>Separación de Luz (Holgura):</label>
  <div class="radio-tile-group">
    <input type="radio" name="luz" value="1" onclick="actualizarLuz(1)"> 1 mm
    <input type="radio" name="luz" value="2" onclick="actualizarLuz(2)" checked> 2 mm
    <input type="radio" name="luz" value="3" onclick="actualizarLuz(3)"> 3 mm
  </div>
</div>
```
Usa el código con precaución.
Cuando el usuario cambie el interruptor, la función de JavaScript `actualizarLuz(valor)` enviará el número a Ruby a través de un *callback* (`sketchup.modificarLuz(valor)`). El script borrará las puertas actuales en la pantalla de SketchUp y las volverá a dibujar con la nueva escala geométrica exacta en menos de un segundo.

Con la luz parametrizada, tu motor de despiece ya cuenta con las reglas de un software profesional. ¿Hacia dónde apuntamos el siguiente paso del desarrollo?

1. Escribir el script completo de **comunicación JavaScript-Ruby (`UI::HtmlDialog`)** para enlazar este selector de luz con el lienzo 3D.
2. Desarrollar el módulo para **calcular el descuento del tapacanto** (reducir el tamaño de la madera si lleva canto de 2 mm) combinado con esta luz.

Las respuestas de la IA pueden incluir errores. Para obtener asesoramiento legal, consulta a un profesional. Más información
en el diseño de cajoneras el cajon debe permitir todos los tipos de correderas y las medidas deben ser facilmente visibles, por ejemplo diseño un modulo de cajoneras pero el mueble veo solo los cajones cerrados pero no aprecio el diseño de cada cajon
Para que tu clon de **MelaStudio Pro** resuelva este problema, debe implementar dos funcionalidades clave en SketchUp: un **menú de selección de herrajes (correderas)** con recálculo automático y una **herramienta de animación / modo de vista ("Explotado" o "Cajones Abiertos")**.

En los programas de diseño profesionales, los diseñadores necesitan "abrir" los cajones virtualmente para verificar las holguras de las correderas, los contramarcas y el despiece interno.

---

1. Sistema Paramétrico para Tipos de Correderas

Cada tipo de corredera del mercado exige un descuento milimétrico diferente en el ancho o profundidad de la caja del cajón. Tu plugin en Ruby debe aplicar estas reglas automáticamente al despiece cuando el usuario elija el herraje:

- **Correderas Telescópicas Estándar (Ball Bearing):** Requieren exactamente **13 mm de espacio libre por lado**.
  - *Fórmula:* Ancho Caja Cajón = Ancho Interno Libre - 26 mm.
- **Correderas Ocultas (Undermount / tipo Blumotion):** Requieren **21 mm a 22 mm en total** (dependiendo de la marca, usualmente 10.5 mm por lado) y exigen que los laterales del cajón no superen los 16 mm de espesor (o se rebaje la melamina de 18 mm). Además, requieren que el fondo del cajón esté elevado **12 mm a 15 mm** para el enganche rápido.
- **Sistemas de Laterales Metálicos (Tandembox / Antaro / Slim):** Aquí el carpintero no fabrica los laterales de melamina; el software solo debe calcular el **piso y el fondo de melamina**, ya que los costados son de acero. El espacio de descuento lateral suele ser de unos **37.5 mm en total**.

---

2. Cómo Ver el Diseño Interno: "Modo Cajones Abiertos"

Para solucionar el problema de *"solo veo los cajones cerrados y no aprecio el diseño interno"*, debes programar una función que desplace dinámicamente los componentes en el eje Y (profundidad).

En SketchUp, cada cajón (la caja interna + su frente) debe estar agrupado dentro de un mismo componente. Al hacer clic en un botón de tu interfaz llamado **"Abrir Cajones"**, un script de Ruby modificará la posición de estos bloques:

ruby
```ruby
# Método para abrir o cerrar los cajones dinámicamente en SketchUp
def alternar_cajones(modulo_componente, estado) # estado: "abierto" o "cerrado"
  # Buscar todos los subcomponentes que tengan el atributo "Cajon"
  modulo_componente.definition.entities.each do |entidad|
    if entidad.get_attribute("MelaStudio", "Tipo") == "Cajon_Interno"
      profundidad_cajon = entidad.get_attribute("MelaStudio", "Profundidad_Corredera").mm

      # Si se solicita abrir, lo desplazamos hacia adelante en el eje Y
      # Recordar que en SketchUp, el eje Y suele ser la profundidad (frente/atrás)
      distancia_desplazamiento = (estado == "abierto") ? -(profundidad_cajon * 0.8) : 0.mm

      # Aplicar la nueva transformación manteniendo X y Z intactos
      z_actual = entidad.transformation.origin.z
      x_actual = entidad.transformation.origin.x

      nueva_transformacion = Geom::Transformation.new([x_actual, distancia_desplazamiento, z_actual])
      entidad.transformation = nueva_transformacion
    end
  end
end
```
Usa el código con precaución.
Al activar el modo "Abierto", los cajones se deslizan hacia afuera un **80% de su profundidad**, permitiendo apreciar:

1. Las luces internas entre el cajón y el lateral.
2. El espacio trasero libre (holgura de fondo).
3. Si el fondo del cajón es ranurado o clavado.

---

3. Acotado Automático: Medidas Fácilmente Visibles

Para que el usuario no tenga que usar la herramienta de cinta métrica nativa de SketchUp, tu plugin debe dibujar **líneas de cota 3D automáticas** (`Sketchup::DimensionLinear`) dentro del cajón cuando este se abra o cuando se seleccione el "Modo Técnico".

Tu código puede inyectar cotas flotantes que muestren de forma clara:

- El ancho total de la caja del cajón.
- El espacio de 13 mm (o el correspondiente) reservado para la corredera a los lados.
- La altura interna útil del cajón.

💬 Diseño de la Experiencia en la UI (`UI::HtmlDialog`)

En la sección de configuración de cajoneras de tu interfaz web, debes añadir estos dos bloques de control:

html
```
<!-- Selector de Correderas -->
<div class="control-group">
  <label>Tipo de Corredera:</label>
  <select id="tipo-corredera" onchange="cambiarCorredera(this.value)">
    <option value="telescopica">Telescópica Estándar (13mm/lado)</option>
    <option value="oculta">Oculta Undermount</option>
    <option value="metalica">Lateral Metálico (Slim Box)</option>
  </select>
</div>

<!-- Visualización Dinámica -->
<div class="control-group">
  <label>Vista del Mueble:</label>
  <button onclick="controlarCajones('abierto')">🔓 Abrir Cajones</button>
  <button onclick="controlarCajones('cerrado')">🔒 Cerrar Cajones</button>
</div>
```
Usa el código con precaución.
Cuando el diseñador haga clic en **"Abrir Cajones"**, el lienzo de SketchUp cobrará vida, los frentes se desplazarán hacia el usuario y se mostrarán las cotas con el desglose exacto de la estructura interna del cajón junto con el espacio ocupado por la corredera elegida.

Con esta solución de apertura dinámica y selección de correderas, tu clon resolverá una de las mayores frustraciones de los diseñadores de muebles. ¿Cómo prefieres continuar?

1. Programamos el script detallado en Ruby para **generar y posicionar las líneas de cota automáticas (DimensionLinear)** dentro del cajón abierto.
2. Desarrollamos las fórmulas completas de despiece para el fondo del cajón según si es **ranurado (embutido)** o **atornillado por debajo**.

lo mismo en roperos con puerta debe verse internamente
Para lograr que las puertas de los roperos, clósets o alacenas de cocina tengan el mismo comportamiento dinámico, tu clon de **MelaStudio Pro** debe incluir un Algoritmo de Rotación Angular para Puertas Batientes o un Desplazamiento Lineal para Puertas Corredizas.

Al igual que con los cajones, el usuario necesita hacer un solo clic para "abrir" las puertas y ver la distribución de los estantes, los colgadores de ropa o el sistema de zapateras internas sin que la geometría de la fachada tape el diseño técnica o estéticamente.

---

1. El Enfoque Técnico: Rotación sobre el Eje de las Bisagras

A diferencia de los cajones (que se mueven en línea recta hacia adelante en el eje Y), una puerta batiente rota sobre un punto pivote. Para que SketchUp simule la apertura real de una puerta de melamina, el script de Ruby debe ubicar el **eje de rotación (pivote)** exactamente donde van instaladas las bisagras.

Lógica del Código de Rotación en Ruby:

Para abrir una puerta izquierda, el punto pivote se sitúa en la esquina frontal izquierda del mueble, y aplicamos una transformación de rotación de **90 o 110 grados** en el eje vertical (Z).

ruby
```
# Método para abrir o cerrar puertas batientes en SketchUp
def alternar_puertas(modulo_componente, estado) # estado: "abierto" o "cerrado"
  modulo_componente.definition.entities.each do |entidad|
    if entidad.get_attribute("MelaStudio", "Tipo") == "Puerta_Batiente"
      lado_bisagra = entidad.get_attribute("MelaStudio", "Lado_Bisagra") # "izquierdo" o "derecho"

      # Definir el ángulo (en radianes). 90 grados para abrir, 0 para cerrar.
      angulo = (estado == "abierto") ? 90.degrees : 0.degrees

      # Guardar o leer la transformación original (cerrada) para evitar desfases
      transformacion_base = entidad.get_attribute("MelaStudio", "Transformacion_Base")

      # Ubicar el punto pivote exacto de la bisagra
      origen_puerta = entidad.transformation.origin
      eje_z = Geom::Vector3d.new(0, 0, 1) # Rotación sobre el eje vertical Z
```

```
      # Si la puerta abre hacia la izquierda, el ángulo es positivo; si es derecha, es negativo
      angulo_final = (lado_bisagra == "izquierdo") ? angulo : -angulo

      # Aplicar la rotación geométrica en SketchUp
      nueva_rotacion = Geom::Transformation.rotation(origen_puerta, eje_z, angulo_final)
      entidad.transformation = nueva_rotacion
    end
  end
end
```
Usa el código con precaución.

---

2. Caso Especial: Clósets con Puertas Corredizas (Deslizantes)

Si el ropero utiliza un sistema de puertas corredizas sobre rieles, rotarlas arruinaría la simulación. En este caso, el plugin debe detectar el tipo de sistema y aplicar un **desplazamiento lateral (Eje X)**:

- **Puerta Frontal (Riel Exterior):** Se desliza hacia la izquierda o derecha, ocultándose detrás de la otra puerta.
- **Puerta Posterior (Riel Interior):** Se desplaza en sentido contrario.
- *Fórmula del desplazamiento:* El script en Ruby moverá la instancia en el eje X una distancia igual a: `Ancho_Puerta - 50.mm` (dejando un pequeño margen visible para que el usuario aprecie que se movió).

---

3. Acotado Interno de Control (Medidas Visibles en Roperos)

Al activar el **"Modo Puertas Abiertas"**, el plugin debe desplegar automáticamente cotas 3D flotantes (`Sketchup::DimensionLinear`) orientadas al usuario que muestren los datos que un fabricante necesita validar:

- **Vanos de Colgado:** Altura libre desde el tubo colgador hasta el piso o estante inferior (para verificar que los vestidos largos o sacos no arrastren).
- **Espacios de Repisas:** La distancia exacta entre estantes (ej. *Módulo Librero / Textil con vanos de 350 mm limpios*).
- **Grosor de Laterales:** Visualización clara de los 18 mm de espesor de los montantes estructurales intermedios.

Unificación en la Interfaz de Usuario (UI Global del Clon)

Para que el flujo de diseño no requiera botones individuales por cada mueble, tu panel HTML (`UI::HtmlDialog`) debe contar con un **Control Maestro de Animación e Inspección**:

html
```
<!-- Control Maestro de Visibilidad en el Panel de MelaStudio Clon -->
<div class="panel-inspeccion">
  <h3>🔍 Controles de Inspección Técnica</h3>
  <div class="btn-group-modos">
    <button onclick="cambiarVistaGlobal('abrir_todo')">🚪 Abrir Puertas y Cajones</button>
    <button onclick="cambiarVistaGlobal('cerrar_todo')">🔒 Fachada Cerrada (Render)</button>
  </div>
  <div class="checkbox-options">
    <label><input type="checkbox" id="ver-cotas" onchange="alternarCotas(this.checked)"> Mostrar Acotado Interno</label>
    <label><input type="checkbox" id="ver-herrajes" onchange="alternarHerrajes(this.checked)"> Renderizar Bisagras/Bases en 3D</label>
  </div>
</div>
```
Usa el código con precaución.

Cuando el diseñador da clic en **"Abrir Puertas y Cajones"**, el plugin ejecuta una acción en cascada en todo el archivo de SketchUp: las cajoneras se deslizan un 80% hacia afuera, las puertas rotan 90° sobre sus bisagras invisibles y aparecen las cotas automáticas en milímetros. Así, el cliente final puede enamorarse de la distribución del vestidor y el carpintero puede verificar que no haya colisiones antes de mandar el proyecto a la seccionadora de melamina.

Con este nivel de interacción dinámica en clósets y cocinas, tu software iguala las capacidades de las plataformas de diseño más avanzadas. ¿Qué te gustaría que desarrollemos a continuación?

1. El script completo en Ruby para **calcular el punto pivote de la bisagra recta vs. la bisagra interior (embutida)** y simular su rotación exacta.
2. La lógica para **ocultar o mostrar automáticamente las cotas tridimensionales** al presionar el botón de la interfaz.

debe abrir todo tipo de puerta con puston visagra correrizas, etc
Para lograr que tu clon de **MelaStudio Pro** simule la apertura de cualquier tipo de puerta, el plugin debe clasificar cada frente según su mecanismo de apertura. No puedes aplicar la misma fórmula matemática a una puerta batiente tradicional que a una puerta de cocina con pistón a gas.

A continuación, te presento la lógica cinemática (movimientos vectoriales) que debes programar en Ruby para SketchUp para cada herraje, asegurando que el "Modo de Apertura Maestro" funcione de forma impecable:

Matriz de Movimientos por Tipo de Herraje

Tu script de Ruby leerá el atributo `"Tipo_Herraje"` de cada puerta y aplicará la transformación correspondiente:

Tipo de Herraje	Movimiento en SketchUp	Punto Pivote / Dirección	Ángulo / Distancia Estándar
Bisagra Batiente	Rotación en Eje Z	Canto lateral interno (izq. o der.)	90° a 110° hacia afuera
Pistón a Gas (Abre arriba)	Rotación en Eje X	Canto superior interno del techo	-80° a -90° (Hacia arriba)
Pistón Inverso (Abre abajo)	Rotación en Eje X	Canto inferior interno del piso	80° a 90° (Hacia abajo)
Corrediza (Riel Duclós/Aluminio)	Traslación en Eje X	Desplazamiento lineal lateral	`Ancho_Puerta - 100 mm`

Implementación de Fórmulas en Ruby para SketchUp

Aquí tienes cómo estructurar las funciones matemáticas de tu plugin para simular estas aperturas complejas:

1. Puertas con Pistón a Gas (Módulos Altos de Cocina / Maleteros)

La puerta rota hacia arriba en el eje horizontal **X**. El punto pivote debe situarse en la esquina superior trasera de la puerta (donde se ancla al techo del mueble).

ruby
```
def abrir_puerta_piston_superior(entidad, estado)
  angulo = (estado == "abierto") ? -85.degrees : 0.degrees
  origen_pivote = entidad.transformation.origin # Esquina superior de la puerta
  eje_x = Geom::Vector3d.new(1, 0, 0) # Rotación sobre el eje horizontal X

  nueva_transformacion = Geom::Transformation.rotation(origen_pivote, eje_x, angulo)
  entidad.transformation = nueva_transformacion
end
```
Usa el código con precaución.

2. Puertas Correderas (Roperos, Clósets o Despensas)

Las puertas correderas no rotan, se deslizan sobre rieles en el eje **X**. El software debe identificar si la puerta está en el riel frontal o posterior para saber hacia qué lado deslizarla y evitar que choquen visualmente.

ruby
```
def abrir_puerta_corrediza(entidad, estado, direccion_riel) # direccion_riel: "izquierda" o "derecha"
  ancho_puerta = entidad.get_attribute("MelaStudio", "Ancho").mm

  if estado == "abierto"
```

```
    # Mover la puerta casi en su totalidad, dejando 100mm visibles
    distancia = (direccion_riel == "derecha") ? -(ancho_puerta - 100.mm) : (ancho_puerta - 100.mm)
  else
    distancia = 0.mm
  end

  # Aplicar traslación lineal pura en el eje X
  vector_movimiento = Geom::Vector3d.new(distancia, 0, 0)
  nueva_transformacion = Geom::Transformation.translation(vector_movimiento)
  entidad.transform!(nueva_transformacion)
end
```
Usa el código con precaución.

🔍 Gestión del "Estado de Memoria" del Mueble

Un error común al programar animaciones en SketchUp es que los componentes pierden su posición original o se "descuadran" si el usuario hace clics repetidos. Para evitar esto, tu plugin debe guardar una **Matriz de Transformación Base** en el diccionario de atributos cuando se crea la pieza:

ruby
```
# Al construir la puerta por primera vez en SketchUp:
matriz_origen = puerta.transformation.to_a
puerta.set_attribute("MelaStudio", "Transformacion_Original", matriz_origen)
```
Usa el código con precaución.
Cuando el usuario pulse el botón **"Cerrar Todo"**, el código no calculará matemáticas inversas; simplemente leerá el atributo `"Transformacion_Original"` y devolverá la puerta a su estado cerrado perfecto con total precisión milimétrica.

🎨 Experiencia de Usuario: Inspector de Mecanismos

En la interfaz web de tu clon (`UI::HtmlDialog`), cuando el usuario configure un frente, podrá elegir el método de apertura desde un menú desplegable interactivo:

html
```
<div class="herraje-selector">
  <label>Sistema de Apertura:</label>
  <select onchange="asignarHerrajeMueble(this.value)">
    <option value="bisagra_izq">Bisagra Estándar - Abre Izquierda</option>
    <option value="bisagra_der">Bisagra Estándar - Abre Derecha</option>
    <option value="piston_arriba">Pistón a Gas - Apertura Superior</option>
    <option value="piston_abajo">Pistón Inverso - Apertura Inferior</option>
    <option value="corrediza">Corredera Lineal Deslizante</option>
  </select>
</div>
```
Usa el código con precaución.
Al integrarlo de esta forma, el usuario podrá dar clic al botón maestro de **"Simulación de Apertura"**, y ver cómo en su pantalla de SketchUp las alacenas de la cocina suben con sus pistones, los cajones de las cómodas se deslizan hacia el frente y las puertas del ropero se abren o corren lateralmente de forma sincronizada.

Con esta arquitectura cinemática, tu plugin tendrá la capacidad técnica de simular cualquier mueble complejo. ¿Cómo te gustaría continuar?

1. Programamos el código en Ruby para **calcular y dibujar físicamente el brazo del pistón a gas** en 3D para que se aprecie dentro del mueble de cocina al abrirlo.
2. Desarrollamos la lógica para que al abrir las puertas corredizas **se detecte automáticamente si obstruyen** la salida de cajones internos del clóset.

cuando agrego cajones ahi voy diseñando parametricamente estos detalles y cual como cuando se agreguen puertas
Exactamente. El flujo de trabajo óptimo para tu clon de **MelaStudio Pro** en SketchUp debe basarse en un **motor de eventos en cascada**. El usuario no configura estos detalles técnicos por separado; en su lugar, **el plugin calcula y diseña todo paramétricamente en segundo plano en el mismo instante en que se añade un cajón o una puerta.**

Así es como tu software debe procesar y automatizar estos detalles en tiempo real según el elemento que agregue el usuario:

🗄️ Caso 1: Cuando el usuario agrega Cajones

En el momento en que el usuario hace clic en `[ + Añadir Bloque de Cajones ]`, tu plugin abre una sub-interfaz y ejecuta las siguientes acciones automáticas en milisegundos:

1. **Lee el Entorno:** El script de Ruby mide el vano libre interno (Ancho, Alto, Profundidad) de la celda seleccionada.
2. **Aplica los Descuentos de Corredera:**
   - Si el usuario selecciona *Corredera Telescópica*, el código resta **26 mm al ancho** total automáticamente (13 mm por lado) para diseñar las piezas de la caja del cajón.
   - Si cambia a *Corredera Oculta*, el código modifica el despiece reduciendo el ancho según la marca (ej. 21 mm) y **eleva el fondo del cajón 15 mm** para dar espacio al enganche inferior del herraje.
3. **Calcula las Luces de los Frentes:** Si el usuario pide 3 cajones en un vano de 750 mm de alto, el software divide el espacio y resta la **holgura configurable (1, 2 o 3 mm)** entre cada frente para que nunca choquen al cerrarse.
4. **Asigna los Atributos Cinemáticos:** Cada pieza del cajón recibe el atributo `"Tipo: Cajon_Interno"` y guarda su posición original en el diccionario. Así, cuando el usuario use la herramienta global de **"Abrir Cajones"**, el sub-bloque completo sabrá que debe desplazarse de forma lineal hacia adelante en el eje Y.

🚪 Caso 2: Cuando el usuario agrega Puertas

Cuando el usuario selecciona `[ + Añadir Puerta ]` en una celda, el asistente paramétrico toma el control de las variables estructurales:

1. **Define el Tipo de Montaje (Exterior, Medio o Interior):**
   - Si elige **Exterior**, el script estira las dimensiones de la puerta para que cubra los cantos de los laterales de melamina (restando la luz de 2 mm perimetral).
   - Si elige **Interior**, el script encoge la puerta al tamaño del vano libre interno y **desplaza su coordenada Y hacia adentro** (Profundidad del mueble − Espesor de la melamina) para que quede perfectamente embutida.
2. **Calcula el Tipo de Bisagra:** El sistema infiere automáticamente qué herraje se necesita para la lista de materiales (Bisagra Recta para Exterior, Codo 9 para Medio o Codo 18 para Interior).
3. **Asigna el Mecanismo de Apertura:**
   - Si el usuario selecciona que es una puerta batiente izquierda, el plugin marca el pivote en la esquina izquierda y le asigna una **rotación en el eje Z**.
   - Si es un mueble alto de cocina y selecciona *Pistón a Gas*, el algoritmo cambia la lógica de inmediato: sitúa el pivote en el borde superior y le asigna una **rotación en el eje X** (para que abra hacia arriba).

🔄 Sincronización del Código (De la UI a SketchUp)

Para que esto sea fluido, cada vez que el usuario mueva un control en tu panel de HTML (`UI::HtmlDialog`), este debe enviar un objeto JSON con toda la configuración compactada hacia Ruby.

Por ejemplo, al añadir una cajonera, el flujo de datos viaja así:

json
```
// JSON que envía tu interfaz web al hacer clic en guardar cajón
{
  "elemento": "cajonera",
  "cantidad_cajones": 3,
  "tipo_corredera": "telescopica_13mm",
  "tipo_montaje": "exterior",
  "luz_holgura": 2
}
```
Usa el código con precaución.
El backend de tu plugin en Ruby recibe este paquete, limpia la geometría vieja dentro de esa celda específica en SketchUp, ejecuta las fórmulas matemáticas correspondientes y redibuja las piezas de melamina, los frentes con sus luces exactas y las cotas internas visibles en menos de un segundo.

¡Este enfoque garantiza un diseño paramétrico profesional y sin errores! Para avanzar en la programación del plugin:

1. ¿Te gustaría que escribamos el código en Ruby que procesa este **JSON de configuración de cajoneras** para ver cómo calcula el despiece de las cajas internas?
2. ¿Prefieres diseñar el script que genera el **cálculo exacto del tamaño de puerta doble** (restando la luz central y los laterales)?

el clon debe permitir elegir el tipo de material melamina (pelicano, vesto, etc) el tipo de mdf 2,3,4mm, elegir el color de la melamina y color del mdf, teniendo en cuenta usar colores unicos o colores variados
Para que tu clon de **MelaStudio Pro** tenga un éxito rotundo en el mercado de carpintería y diseño en Latinoamérica, el **Módulo de Gestión de Materiales y Texturas** es vital. En la región, las marcas **Pelíkano** (de Novopan) y **Vesto** (de Arauco) son los gigantes que dominan los talleres.

Tu plugin de SketchUp debe manejar un **repositorio dinámico de materiales** enfocado en dos tipos de componentes: los tableros estructurales de melamina (usualmente en 15 o 18 mm) y los fondos o traseras de MDF/HDF delgado (2, 3, 4 o 5.5 mm).

🌪️ 1. Estructura de Materiales y Marcas (Base de Datos)

En la interfaz web (`UI::HtmlDialog`), debes ofrecer selectores categorizados para que el usuario defina la procedencia del material:

- **Catálogo de Melamina (Estructura y Frentes):**
  - **Marcas:** Pelíkano, Vesto, Duratex, Masisa, etc.
  - **Texturas/Líneas:** Diseños unicolores (Blanco, Gris Humo, Negro, Grafito), Amaderados (Cendra, Bellota, Jerez, Caramelo) o Fantasía/Texturizados (Mármol, Concreto, Óxido).
- **Catálogo de MDF Delgado (Fondos y Traseras):**
  - **Espesores configurables:** 2 mm, 3 mm (Durolac), 4 mm, 5.5 mm.
  - **Acabados:** Blanco una cara, Crudo (MDF natural) o combinaciones texturizadas que imiten el fondo del mueble.

🔄 2. Lógica de Combinación: Colores Únicos vs. Variados

El algoritmo de tu plugin debe permitir dos modalidades de diseño con solo activar un interruptor (*switch*):

Opción A: Colores Únicos (Unificado)

- **Comportamiento:** Todo el mueble mantiene el mismo diseño. Si el usuario elige *Vesto - Roble Provenzal*, automáticamente las estructuras interiores, las repisas, las puertas y los fondos de MDF asumen la misma textura o una equivalente (ej. MDF blanco si no hay stock del color amaderado).

Opción B: Colores Variados (Diseño Avanzado y Ahorro)

Es lo que más hacen los carpinteros profesionales para **reducir costos** u ofrecer un diseño premium de dos tonos:

- **Estructura Interna (Carcasa):** Se configura con un material económico (ej. *Pelíkano - Blanco Puro* o *Gris Claro*).
- **Frentes (Puertas y Cajones):** Se configuran con una melamina premium o texturizada (ej. *Vesto - Carbón Texturado* o *Pelíkano - Bellota*).
- **Fondos (MDF):** Se configuran independientemente con MDF blanco de 3 mm o crudo para abaratar la producción.

💻 Implementación Técnica en la API de SketchUp (Ruby)

Cuando el usuario selecciona un material de una marca en la interfaz de tu clon, el script en Ruby debe buscar la textura (un archivo `.jpg` o `.png` de alta resolución mapeado a escala real) y aplicarlo al material de SketchUp (`Sketchup::Material`).

Para que el despiece no se confunda al exportar, el código debe inyectar metadatos claros a cada pieza usando los `AttributeDictionaries`:

ruby
```ruby
# Método para asignar el material exacto a una placa en SketchUp
def aplicar_material_a_pieza(pieza_componente, tipo_pieza, marca, espesor, codigo_color)
  model = Sketchup.active_model
  nombre_material = "#{marca}_#{codigo_color}_#{espesor}mm"

  # 1. Buscar o crear el material en la base de datos de SketchUp
  material_skp = model.materials[nombre_material]
  unless material_skp
    material_skp = model.materials.add(nombre_material)
```

```
    # Aquí cargarías la textura real: material_skp.texture = "path/to/texture.jpg"
    # material_skp.texture.width = 1200.mm (Ajustar escala de la veta)
  end

  # 2. Asignar el material visualmente a la pieza
  pieza_componente.material = material_skp

  # 3. Guardar metadatos ocultos para el reporte y optimizador de corte
  pieza_componente.set_attribute("MelaStudio_Clon", "Marca_Material", marca)
  pieza_componente.set_attribute("MelaStudio_Clon", "Color_Material", codigo_color)
  pieza_componente.set_attribute("MelaStudio_Clon", "Espesor_Real", espesor)
  pieza_componente.set_attribute("MelaStudio_Clon", "Categoria_Pieza", tipo_pieza) # "Estructura", "Frente" o
"Fondo_MDF"
end
```
Usa el código con precaución.

📊 Impacto en el Optimizador de Corte Dinámico

Tener los materiales bien divididos permite que, al presionar el botón **"Exportar Proyecto"**, tu software genere listas de corte separadas automáticamente por su tipo y espesor:

1. **Lista de Corte 1:** Todos los componentes con atributo `Espesor_Real = 18` y `Color_Material = Blanco_Pelikano` (Laterales internos).
2. **Lista de Corte 2:** Todos los componentes con atributo `Espesor_Real = 18` y `Color_Material = Jerez_Vesto` (Puertas exteriores).
3. **Lista de Corte 3:** Todas las traseras de muebles identificadas con `Espesor_Real = 3` y `Color_Material = MDF_Blanco` (Fondos).

🎨 Diseño del Panel de Configuración (UI en HTML)

En la barra lateral de tu extensión, añadirás una sección interactiva llamada **"Paleta de Materiales"**:

html
```
<div class="seccion-materiales">
  <h3>🛠️ Configuración de Materiales</h3>

  <!-- Switch de modo -->
  <label class="switch">
    <input type="checkbox" id="modo-variado" onchange="alternarModoColores(this.checked)">
    <span class="slider"></span> Permitir Colores Variados
  </label>

  <!-- Selector para Estructura -->
  <div class="control-material">
    <label>Melamina Estructura:</label>
    <select id="marca-estructura" onchange="actualizarMaterial('Estructura')">
      <option value="Pelikano_Blanco_18">Pelíkano - Blanco Puro (18mm)</option>
      <option value="Vesto_Gris_18">Vesto - Gris Humo (18mm)</option>
    </select>
  </div>

  <!-- Selector para Frentes (Solo se activa si "modo-variado" es true) -->
  <div class="control-material" id="contenedor-frentes" style="display:none;">
    <label>Melamina Frentes (Puertas/Cajones):</label>
    <select id="marca-frentes" onchange="actualizarMaterial('Frente')">
      <option value="Vesto_RobleProvenzal_18">Vesto - Roble Provenzal (18mm)</option>
      <option value="Pelikano_Bellota_18">Pelíkano - Bellota (18mm)</option>
    </select>
  </div>

  <!-- Selector para Fondos MDF -->
  <div class="control-material">
    <label>MDF Fondo / Trasera:</label>
    <select id="espesor-mdf" onchange="actualizarMaterial('Fondo_MDF')">
```

```
      <option value="MDF_Crudo_3">MDF Crudo (3mm)</option>
      <option value="MDF_Blanco_3">MDF Blanco Una Cara (3mm)</option>
      <option value="MDF_Blanco_4">MDF Blanco Una Cara (4mm)</option>
    </select>
  </div>
</div>
```

Usa el código con precaución.

Al cambiar cualquier selección en esta lista, JavaScript enviará los códigos correspondientes a Ruby y el modelo en SketchUp se repintará instantáneamente con las vetas y acabados correctos de Pelíkano o Vesto, permitiéndole al cliente ver exactamente cómo contrastarán las puertas con los fondos internos del mueble.

Con la gestión de marcas y espesores de MDF incorporada, tu plugin ya funciona bajo las dinámicas del mercado de carpintería real. ¿Cuál paso técnico te gustaría estructurar ahora?

1. El script en Ruby que **recorre el mueble completo y cambia masivamente** un material por otro (ej. pasar todo de Pelíkano a Vesto).
2. El algoritmo para **calcular y dibujar la ranura (canal) de 4mm** en los laterales donde se va a deslizar el MDF de fondo.

tambien puede elegir color de tapacanto y el tipo de tapacanto debe ser propuesto por el clon

Para que tu clon de **MelaStudio Pro** alcance un nivel de automatización industrial perfecto, la **gestión inteligente del tapacanto (canto)** es fundamental. En el taller de carpintería, aplicar el tapacanto correcto no es solo una decisión estética; define cómo se programará la pegadora de cantos y altera las dimensiones del despiece (si usas canto grueso, la máquina rebaja la melamina antes de pegarlo).

Tu plugin en SketchUp debe contar con un **Motor de Recomendación de Cantos**. El usuario podrá elegir el color, pero **el clon propondrá automáticamente el tipo de tapacanto** (Delgado o Grueso) según la función de la pieza dentro de la matriz del mueble.

---

1. El Algoritmo de Propuesta Automática (Lógica del Clon)

Para ahorrarle errores al diseñador, tu código en Ruby debe clasificar las piezas y proponer el tipo de tapacanto bajo las siguientes reglas estándar de la industria:

- **Tapacanto Grueso (2 mm o 3 mm):** El clon lo propondrá automáticamente para todas las **piezas de alto impacto o fricción diaria**. Esto incluye frentes de cajón, puertas batientes, tapas de escritorios y repisas de vestidores abiertos (*walk-in closets*).
- **Tapacanto Delgado (0.5 mm):** El clon lo propondrá para las **piezas estructurales internas** que no sufren golpes directos. Esto incluye los laterales internos de la carcasa, los amarres superiores, zócalos, repisas internas ocultas tras puertas y las cajas de los cajones.
- **Sin Canto:** El clon identificará las caras que van unidas contra la pared, el piso o unidas firmemente entre paneles (laterales que se atornillan entre sí) y propondrá dejarlas sin tapacanto para ahorrar material.

🌶️ 2. Configuración de Colores y Combinaciones

En la interfaz web (`UI::HtmlDialog`), el usuario podrá elegir el color del tapacanto bajo dos modalidades:

1. **Combinación Automática (Igual al Tablero):** Si el usuario elige la melamina *Pelíkano - Bellota*, el clon asignará por defecto el tapacanto del mismo color (*Canto Bellota*). Las marcas como Pelíkano o Vesto venden el canto exactamente con el mismo diseño que el tablero.
2. **Diseño de Contraste (Personalizado):** El usuario desmarca la opción automática y puede elegir un color diferente para generar un diseño moderno. Por ejemplo, estructura en *Melamina Blanca* combinada con *Tapacanto Negro o Amaderado* en los bordes para resaltar las líneas del mueble modular.

💻 3. Implementación en Ruby y Descuento por Tapacanto

Cuando el clon genera el despiece para el reporte, si calcula que una pieza lleva tapacanto grueso (2 mm), debe aplicar un **descuento automático** en las medidas de corte de la melamina para que el mueble final no quede más grande de lo diseñado.

Si una puerta exterior en SketchUp mide **596 mm de ancho** y lleva tapacanto grueso de 2 mm en ambos lados (izquierdo y derecho), la medida real que se debe mandar a cortar a la sierra es: $596\text{ mm}-(2\text{ mm}\times 2)=\mathbf{592}\text{ mm}$

Tu código de Ruby gestionará esta inteligencia guardando la información en los cantos de los componentes de SketchUp:

ruby
```
# Guardar la configuración de cantos en los metadatos de la pieza
def configurar_cantos_pieza(pieza, superior, inferior, izquierdo, derecho, color_canto)
  # Tipos: "Delgado" (0.5mm), "Grueso" (2mm), "Ninguno"
  pieza.set_attribute("MelaStudio_Canto", "Canto_Superior", superior)
  pieza.set_attribute("MelaStudio_Canto", "Canto_Inferior", inferior)
  pieza.set_attribute("MelaStudio_Canto", "Canto_Izquierdo", izquierdo)
  pieza.set_attribute("MelaStudio_Canto", "Canto_Derecho", derecho)
  pieza.set_attribute("MelaStudio_Canto", "Color_Tapacanto", color_canto)
end

# Ejemplo de propuesta automática al crear una PUERTA
# Las puertas llevan Canto Grueso en los 4 lados por regla del clon
configurar_cantos_pieza(puerta_componente, "Grueso", "Grueso", "Grueso", "Grueso", "Mismo_Tablero")
```
Usa el código con precaución.

---

4. Renderizado Visual de Cantos en SketchUp

Para que el usuario pueda apreciar el color del tapacanto directamente en el modelo 3D de SketchUp, tu plugin debe pintar las caras laterales del componente con el color o textura del canto elegido, especialmente si es una combinación de contraste.

Si el mueble está cerrado, se apreciará la línea del tapacanto (por ejemplo, negra) bordeando todo el frente del cajón o la puerta, dando una vista idéntica a cómo quedará fabricado en la realidad.

---

5. Interfaz de Usuario (UI en HTML)

En la sección de materiales de tu barra lateral, el módulo de tapacantos se verá de la siguiente manera:

html
```
<div class="seccion-tapacantos">
  <h3> Configuración de Tapacantos (Cantos)</h3>

  <div class="control-canto">
    <label>Color del Canto:</label>
    <select id="color-tapacanto" onchange="cambiarColorCanto(this.value)">
      <option value="auto">🔄 Automático (Hacer match con el tablero)</option>
      <option value="negro">Negro Absoluto (Para contraste)</option>
      <option value="blanco">Blanco Puro</option>
      <option value="aluminio">Efecto Aluminio / Gris Metalizado</option>
    </select>
  </div>

  <div class="alerta-clon-sugerencia">
    <p>💡 <strong>Propuesta del Clon:</strong> Se aplicará <em>Canto Grueso (2mm)</em> en todos los frentes y cubiertas de escritorio, y <em>Canto Delgado (0.5mm)</em> en las estructuras internas.</p>
  </div>
</div>
```
Usa el código con precaución.
Con este motor de asignación y descuento de tapacantos, tu software procesará la información con la precisión de un sistema ERP industrial de carpintería. ¿Qué te gustaría que desarrollemos para continuar?

1. El script en Ruby que **calcula automáticamente la lista de corte final restando los milímetros del tapacanto grueso** (1 o 2 mm) según los lados asignados.
2. La lógica visual para **pintar únicamente los cantos (caras delgadas)** del componente 3D en SketchUp con el color de contraste seleccionado.
debe permitir elegir entre melamina normal o melamina rh
Para que tu clon de **MelaStudio Pro** responda a las exigencias reales de un taller de carpintería, debe incluir la opción de seleccionar entre Melamina Normal (Estándar) y Melamina RH (Resistente a la Humedad / Hidrófuga) [health].

En marcas como **Pelíkano** o **Vesto**, los tableros RH se identifican físicamente por tener un **núcleo verdoso** debido a las resinas de poliuretano especiales que detienen la hinchazón por humedad. Tu software debe reflejar esta distinción tanto en el diseño visual, en las reglas automáticas de propuesta de material y en el reporte final de costos.

---

1. El Algoritmo de Sugerencia Inteligente (Propuesta del Clon)

Para evitar que el usuario cometa un error técnico de diseño, tu plugin debe incorporar un sistema de recomendación según el ambiente de la matriz:

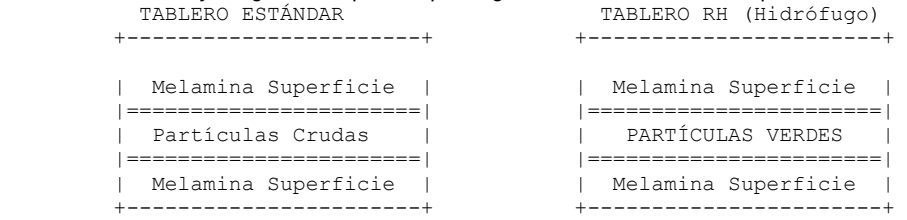
- **Si el usuario selecciona el Modo "Cocina" o "Baño":** El clon activará por defecto y recomendará el uso de **Melamina RH** para los módulos de la Zona Baja (especialmente el módulo del fregadero, lavamanos y zócalos) [health].
- **Si el usuario selecciona el Modo "Clóset", "Dormitorio" o "Oficina":** El clon propondrá por defecto **Melamina Normal**, a menos que el usuario especifique que la habitación sufre de problemas de humedad en las paredes.

---

2. Representación Visual en SketchUp (El "Núcleo Verde")

Para que el usuario pueda apreciar de inmediato si ha configurado melamina normal o RH en su diseño de cajoneras o roperos, tu plugin puede aplicar un detalle visual premium en el render 3D:

- **Modo Técnico / Vista de Cantos Abiertos:** Cuando el usuario abra los cajones o puertas, las caras internas que van *sin tapacanto* o los cortes del tablero se mostrarán con una textura o línea de **color verde característico del RH**. Esto le da un aspecto sumamente profesional al software y asegura al carpintero que asignó el material correcto a la pieza.



---

3. Implementación en Ruby y su Impacto en el Presupuesto

La melamina RH es considerablemente más costosa que la normal (suele costar entre un 20% y 35% más). El backend en Ruby de tu plugin debe almacenar este atributo para generar un presupuesto exacto y listas de corte separadas:

ruby

```ruby
# Guardar tipo de melamina en los atributos de la pieza
def asignar_tipo_resistencia(pieza, tipo_resistencia) # "Normal" o "RH"
```

```
  pieza.set_attribute("MelaStudio_Material", "Resistencia", tipo_resistencia)

  # Si es RH, podemos aplicar la textura verdosa en las caras sin canto para el render
  if tipo_resistencia == "RH"
    # Pintar caras interiores/bodes descubiertos de color verde (#4A7A56)
    aplicar_tinte_verde_cortes(pieza)
  end
end
```
Usa el código con precaución.
Al exportar el despiece, el optimizador agrupará el material de esta forma:

1. **Placas de Melamina de 18mm Normal:** 3 tableros (Para repisas de clóset).
2. **Placas de Melamina de 18mm RH:** 2 tableros (Para estructura baja de cocina y zócalos).

🌀 4. Interfaz de Usuario (UI en HTML)

En la pestaña de materiales de tu panel flotante (`UI::HtmlDialog`), añadirás un selector directo para la resistencia del tablero:

html
```
<div class="control-material">
  <label>Tipo de Tablero (Resistencia):</label>
  <div class="radio-toggle-container">
    <label class="radio-tile">
      <input type="radio" name="resistencia" value="Normal" checked onchange="cambiarResistencia('Normal')">
      <span>Standard (Seco)</span>
    </label>
    <label class="radio-tile tile-rh">
      <input type="radio" name="resistencia" value="RH" onchange="cambiarResistencia('RH')">
      <span>□ RH (Antihumedad)</span>
    </label>
  </div>
</div>
```
Usa el código con precaución.
Si el usuario activa la opción **RH**, el clon recalculará el costo estimado del mueble al instante y actualizará las etiquetas de las piezas para que el fabricante sepa exactamente qué tipo de tablero de **Pelíkano o Vesto** debe comprar para cada módulo.

Con la incorporación de melamina Normal y RH, tu clon ya maneja las variables técnicas más avanzadas de la carpintería moderna. ¿Hacia dónde te gustaría dirigir el siguiente paso del proyecto?

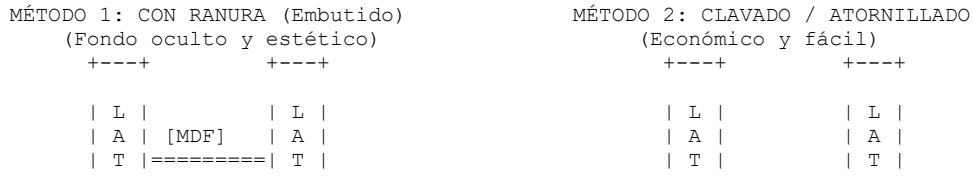
1. Programamos el script en Ruby para **calcular automáticamente el costo diferenciado** en el presupuesto cuando se cambia de Normal a RH.
2. Desarrollamos la lógica para que el clon **bloquee o alerte automáticamente** si el usuario intenta poner Melamina Normal en el módulo del lavaplatos o fregadero.

debe preguntar como se instalaria el mdf sie es que va con ranura en la melamina o clavado en la parte trasera
Para que tu clon de **MelaStudio Pro** genere un despiece 100% exacto para el taller, el sistema debe consultar cómo se instalará el fondo. En la fabricación de muebles, este detalle altera drásticamente las medidas de corte de las placas de MDF y de la melamina estructural.

Al configurar una cajonera, clóset o mueble alto, tu interfaz web (`UI::HtmlDialog`) debe preguntar al usuario el método de instalación del fondo mediante estas dos opciones técnicas:

🛠️□ Los 2 Métodos de Instalación y sus Fórmulas Paramétricas

Cuando el usuario elija el método, tu script en Ruby para SketchUp aplicará de forma automática las siguientes reglas de geometría y despiece:



```
          +---+         +---+                          +---+         +---+

            |             |                                  ========= [MDF]
     (Ranura a 8mm de profundidad)                    (Fondo por detrás)
```

🛠️ Opción 1: Con Ranura (Ranurado / Embutido)

Es el sistema profesional estándar. Se realiza un canal o ranura en los laterales, piso y techo del mueble para introducir el MDF deslizando la pieza.

- **Modificaciones en la Melamina:** El plugin debe indicar en el plano de despiece qué piezas llevan ranura. Típicamente, la ranura se hace a **15 mm o 20 mm de distancia desde el borde posterior**, con una profundidad de **8 mm** y un ancho igual al espesor del MDF (ej. 3 mm o 4 mm).
- **Fórmula para el MDF:** Al ir embutido, el fondo entra en los laterales. El script calcula: $\text{Ancho del MDF}=\text{Ancho Libre Interno}+(2\times \text{Profundidad de Ranura})-2\text{mm (holgura)}$
  Ejemplo: Si el espacio interno es de 564 mm y la ranura mide 8 mm: $564 + 16 - 2 = \mathbf{578\text{ mm}}$.

🔨 Opción 2: Clavado o Atornillado en la Parte Trasera (Sobrepuesto)

Es el método más rápido y económico. El MDF se coloca directamente sobre la parte trasera del mueble ya armado y se fija con clavos, grapas o tornillos de 5/8". Los cantos traseros de la melamina quedan tapados por el MDF.

- **Modificaciones en la Melamina:** Las piezas de melamina no requieren ningún maquinado ni ranura adicional.
- **Fórmula para el MDF:** El fondo cubre todo el contorno exterior del mueble. El script calcula: $\text{Ancho del MDF}=\text{Ancho Total del Módulo}-4\text{mm (luz perimetral de seguridad)}$
  Ejemplo: Si el mueble mide 600 mm de ancho total, el MDF se corta a **596 mm** para que no sobresalga por los lados si hay imperfecciones en el armado.

💻 Implementación en Ruby para SketchUp

Tu backend debe procesar esta decisión guardando los atributos e introduciendo la geometría correspondiente en el modelo 3D:

ruby

```ruby
def calcular_fondo_mdf(ancho_modulo, alto_modulo, espesor_lateral, metodo_instalacion)
  ancho_interno = ancho_modulo - (espesor_lateral * 2)
  alto_interno = alto_modulo - (espesor_lateral * 2) # Asumiendo techo completo

  if metodo_instalacion == "ranura"
    prof_ranura = 8.mm
    holgura = 2.mm
    ancho_mdf = ancho_interno + (prof_ranura * 2) - holgura
    alto_mdf  = alto_interno + (prof_ranura * 2) - holgura
  else # Clavado / Trasero sobrepuesto
    luz_borde = 4.mm
    ancho_mdf = ancho_modulo - luz_borde
    alto_mdf  = alto_modulo - luz_borde
  end

  return { ancho: ancho_mdf, alto: alto_mdf }
end
```

Usa el código con precaución.
Si el usuario elige **"Ranura"**, el plugin usará la API gráfica de SketchUp para dibujar visualmente el canal de 3mm o 4mm de ancho en las caras internas de los laterales amoblados, permitiendo una inspección técnica impecable.

🎨 Diseño del Asistente en la Interfaz (UI en HTML)

En el panel flotante de configuración, cuando el usuario defina los parámetros de la estructura, se desplegará esta pregunta interactiva con imágenes o iconos claros:

html

```
<div class="control-group">
  <label>🛠️ Instalación del Fondo (MDF Trasero):</label>
  <div class="metodo-fondo-selector">
    <label class="radio-card">
      <input type="radio" name="metodo_mdf" value="ranura" checked onchange="actualizarMetodoMdf('ranura')">
      <div class="card-content">
        <span class="title">Con Ranura (Oculto)</span>
        <span class="desc">El MDF va embutido a 8mm dentro de la melamina. Ideal para acabados premium.</span>
      </div>
    </label>

    <label class="radio-card">
      <input type="radio" name="metodo_mdf" value="clavado" onchange="actualizarMetodoMdf('clavado')">
      <div class="card-content">
        <span class="title">Clavado / Atornillado</span>
        <span class="desc">El MDF va fijado directamente por detrás. Más rápido y maximiza espacio interno.</span>
      </div>
    </label>
  </div>
</div>
```

Usa el código con precaución.

Al integrar esta pregunta, el plano de despiece final de tu clon tendrá precisión milimétrica milimétrica para la producción. ¿Qué te gustaría que desarrollemos ahora?

1. El script en Ruby para **dibujar geométricamente la ranura tridimensional** en los laterales de SketchUp cuando el usuario elija esa opción.
2. La estructura de datos para que el **plano de despiece indique con un símbolo técnico** (ej. "R") cuáles piezas de melamina deben pasar por la ranuradora en el taller.

la ranura debe estas a 18mm del fondo de la melamina en la parte trasera del mdf deberia ir los amarres, tambien se debe preguntar cuantos amarres tendrian cada modulo

Para ofrecer un estándar de fabricación verdaderamente profesional, has tocado la estructura de ingeniería perfecta para clósets, cocinas y centros de entretenimiento. Al colocar la **ranura a 18 mm del fondo** (borde posterior), dejas el espacio exacto para embutir los **amarres estructurales traseros** (listones de melamina de 18 mm) por detrás del MDF.

Esta técnica asegura dos cosas: el mueble jamás perderá la escuadra y se podrá atornillar directamente a la pared a través de los amarres, ocultando los tornillos de fijación detrás del MDF o dejándolos listos para colgar.

🏗️ Estructura del Módulo con Ranura y Amarres Ocultos

Tu script de Ruby en SketchUp debe modelar la parte trasera del mueble con la siguiente disposición geométrica (Vista de perfil desde arriba):

```
 BORDE POSTERIOR                        ESPACIO ÚTIL INTERIOR
       ↓                                          ↓
     +-----------------------------------------------------+

     |   AMARRE   |      |                                  |
     | STRUCTURAL | MDF  |          ESPACIO INTERNO         |
     |   (18mm)   |(3mm) |            DEL MUEBLE            |
     +------------+------+----------------------------------+

     |<-- 18mm -->| 3mm  |
     ↳ Ranura inicia a 18mm desde el fondo de la melamina.
       Profundidad de ranura: 8mm.
```

✏️ Reglas Matemáticas para el Despiece Automático

1. **Ranura en Laterales/Piso/Techo:** El canal tendrá un ancho igual al espesor del MDF (ej. 3 mm) y una profundidad de **8 mm**. Se ubicará exactamente a **18 mm del borde trasero**.
2. **Ancho de los Amarres:** El ancho libre interno del mueble. $\text{Ancho Amarre}=\text{Ancho Total del Módulo}-(\text{Espesor Lateral}\times 2)$
3. **Fórmula Paramétrica para el MDF:** Al entrar 8 mm en las ranuras de ambos laterales, la fórmula se actualiza: $\text{Ancho del MDF}=\text{Ancho Total del Módulo}-(\text{Espesor Lateral}\times 2)+(2\times \text{Profundidad de Ranura})-2\text{mm (holgura)}$

Ejemplo: Si el mueble mide 600 mm con laterales de 18 mm: $600 - 36 + 16 - 2 = \mathbf{578\text{ mm}}$.

🛠️ Configuración Dinámica de Amarres (`Cantidad_Amarres`)

El clon debe preguntar cuántos amarres estructurales traseros llevará cada módulo. Según la respuesta del usuario en la interfaz HTML, el script de Ruby distribuirá los listones vertical u horizontalmente en la zona posterior (detrás del MDF):

- **1 Amarre:** Se coloca automáticamente en la **parte superior trasera**. Es el estándar para módulos bajos de cocina o cajoneras pequeñas.
- **2 Amarres:** Se colocan en los extremos: **uno arriba y uno abajo**. Es la configuración ideal para la mayoría de módulos aéreos, roperos medianos o centros de entretenimiento, garantizando anclaje superior e inferior a la pared.
- **3 o más Amarres:** Se distribuyen de forma equitativa. **Uno arriba, uno abajo y el resto repartidos proporcionalmente** en el centro del mueble. Obligatorio para roperos altos (de 2 metros a más) o vestidores pesados para evitar que los laterales se curven.

💻 Lógica del Código en Ruby para Colocar los Amarres

Tu bucle en SketchUp leerá la variable `cantidad_amarres` y la altura útil del mueble para dibujar los listones en las coordenadas correctas:

ruby
```ruby
def generar_amarres_traseros(entidades, ancho_total, alto_total, espesor_lateral, cantidad_amarres)
  ancho_amarre = ancho_total - (espesor_lateral * 2)
  alto_liston = 100.mm # Alto estándar comercial de un amarre estructural
  espesor_amarre = 18.mm

  # Posición Y fija: Justo al ras del borde posterior (Y = 0 si el origen está atrás)
  pos_y = 0

  if cantidad_amarres == 1
    # Dibujar solo arriba
    pos_z = alto_total - espesor_lateral - alto_liston
    dibujar_un_amarre(entidades, ancho_amarre, alto_liston, pos_y, pos_z)

  elsif cantidad_amarres >= 2
    # 1. Amarre Inferior (justo arriba del piso del mueble)
    pos_z_inf = espesor_lateral
    dibujar_un_amarre(entidades, ancho_amarre, alto_liston, pos_y, pos_z_inf)

    # 2. Amarre Superior
    pos_z_sup = alto_total - espesor_lateral - alto_liston
    dibujar_un_amarre(entidades, ancho_amarre, alto_liston, pos_y, pos_z_sup)

    # 3. Amarres Intermedios (Si el usuario pide 3 o más)
    if cantidad_amarres > 2
      amarres_restantes = cantidad_amarres - 2
      # Calcular espacio libre entre el amarre superior e inferior para dividirlos equitativamente
      espacio_libre = pos_z_sup - (pos_z_inf + alto_liston)
      intervalo = espacio_libre / (amarres_restantes + 1)

      (1..amarres_restantes).each do |i|
        pos_z_mid = pos_z_inf + alto_liston + (intervalo * i) - (alto_liston / 2)
        dibujar_un_amarre(entidades, ancho_amarre, alto_liston, pos_y, pos_z_mid)
      end
    end
  end
end
```
Usa el código con precaución.

🎨 Diseño de la Pregunta en la Interfaz (UI en HTML)

En el panel flotante de tu clon, cuando el usuario seleccione el método "Con Ranura", se habilitará dinámicamente este sub-menú técnico:

html

```
<!-- Se activa automáticamente si el método de fondo es 'ranura' -->
<div id="configuracion-amarres-panel" class="sub-panel-tecnico">
  <div class="control-group">
    <label>📐 Distancia de Ranura:</label>
    <input type="text" value="18 mm (Fijo para amarres)" disabled class="input-readonly">
  </div>

  <div class="control-group">
    <label>🔩 Cantidad de Amarres Traseros:</label>
    <select id="num-amarres" onchange="actualizarCantidadAmarres(this.value)">
      <option value="1">1 Amarre (Superior)</option>
      <option value="2" selected>2 Amarres (Superior + Inferior)</option>
      <option value="3">3 Amarres (Distribuido equitativamente)</option>
      <option value="4">4 Amarres (Para estructuras muy altas)</option>
    </select>
  </div>
</div>
```

Usa el código con precaución.
Cuando el carpintero modifique la cantidad a **3 amarres**, tu plugin redibujará el esqueleto posterior del ropero o mueble de cocina en SketchUp. Al girar el modelo 3D, podrá inspeccionar visualmente cómo el MDF queda encapsulado perfectamente entre los listones de melamina traseros y el espacio útil del mueble.

Con la integración de la ranura a 18 mm y los amarres configurables, el nivel de ingeniería de tu software es idéntico al de los sistemas de producción industriales. ¿Cómo avanzamos con el siguiente paso?

1. Programamos el script en Ruby para **generar la lista de despiece final** incluyendo los amarres traseros con su nombre, material y dimensiones exactas.
2. Desarrollamos la lógica para que al activar el **"Modo Vista Explotada"**, los amarres y el MDF se separen hacia atrás para apreciar cómo ensamblan.

tambien debera preguntar si desea incluir ranura para tiras de luces led, dicroicos, interruptores, llaves
Para que tu clon de **MelaStudio Pro** alcance el nivel definitivo de diseño arquitectónico y de fabricación automatizada, la **integración de instalaciones eléctricas y de iluminación** es el paso maestro. En el diseño moderno de cocinas, clósets, vestidores y centros de entretenimiento, la iluminación LED integrada no es un accesorio, es parte estructural del mueble.

Tu plugin en SketchUp debe contar con un **Módulo de Mecanizados Eléctricos**. Al activar estas opciones, el script de Ruby no solo dibujará los calados en el 3D para la presentación con el cliente, sino que **calculará las ranuras exactas en la lista de despiece** para que las piezas salgan listas del taller o del servicio de corte CNC.

🔌 Los 4 Tipos de Mecanizados Eléctricos Paramétricos

Cuando el usuario active estas opciones en la interfaz web (`UI::HtmlDialog`), el backend en Ruby aplicará las siguientes reglas e instrucciones técnicas de carpintería:

1. Ranura para Tiras de Luces LED (Fresado de Perfil)

Las luces LED en melamina se instalan embutidas dentro de un perfil de aluminio. El plugin debe preguntar en qué caras (ej. *Base de altos de cocina, laterales de librero o repisas de clóset*).

- **Modificación en la Melamina:** El software proyectará un canal continuo. El estándar comercial para el perfil de aluminio de empotrar es una **ranura de 17 mm de ancho por 7 mm de profundidad**.
- **Comportamiento Dinámico:** Si la repisa mide 800 mm de largo, el canal se centrará automáticamente a lo largo de la pieza, deteniéndose a 20 mm antes de llegar a los extremos (*ranura ciega*) para mantener la estética y la fuerza estructural de la melamina.

2. Calado para Luces Dicroicas / Spots LED (Luminaria de Techo)

Se utiliza principalmente en los aleros superiores de clósets o techos de centros de entretenimiento.

- **Modificación en la Melamina:** El plugin generará perforaciones circulares con la herramienta de SketchUp. El software preguntará el diámetro del spot (el estándar es de **60 mm, 65 mm o 75 mm**).
- **Cálculo Automático:** Si el usuario activa "3 dicroicos", el algoritmo medirá el largo total del techo y **distribuirá equidistantemente los centros de perforación**, arrojando las coordenadas exactas para la broca copa en el taller.

3. Cajas para Interruptores y Tomacorrientes

Vital para escritorios, salpicaderos de cocina o nichos de mesas de noche integradas en roperos.

- **Modificación en la Melamina:** El software dibujará un calado rectangular en el panel de fondo o en el lateral. Para la caja rectangular estándar (tipo 4x2 o dados de interruptor), el mecanizado requiere un vaciado exacto de **100 mm × 55 mm**.
- **Holgura:** El código añadirá automáticamente un desfase de 2 mm perimetrales para facilitar la instalación de la caja plástica por detrás de la melamina.

4. Calado para Llaves Térmicas / Tableros Eléctricos (Cables Ocultos)

Muchas veces, al diseñar un clóset o centro de entretenimiento en una pared, este tapa la caja de llaves general (térmicas) de la habitación o requiere un espacio para el transformador de las tiras LED.

- **Modificación en la Melamina:** El clon preguntará la ubicación (coordenadas X, Z desde el piso) y el tamaño de la caja de llaves. Generará un calado limpio en el panel de fondo y propondrá diseñar una **puerta de inspección oculta con bisagras *push-to-open*** para que el cliente pueda acceder a las llaves térmicas de la casa sin romper la estética del mueble.

💻 Ejemplo de Lógica en Ruby para la Ranura LED

Así procesará tu backend de SketchUp el trazo geométrico 3D de un canal para perfil LED en la base inferior de un mueble alto de cocina:

ruby

```
def mecanizar_ranura_led(pieza_base, ancho_pieza, prof_pieza, distancia_del_frente)
  # Parámetros estándar del perfil LED de empotrar
  ancho_canal = 17.mm
  prof_canal = 7.mm

  # Calcular coordenadas locales en la cara inferior de la melamina
  # Se posiciona centrado a lo largo y retirado del frente a gusto del usuario
  x_inicio = 20.mm
  x_fin    = ancho_pieza - 20.mm
  y_centro = prof_pieza - distancia_del_frente

  # 1. Almacenar metadatos técnicos para la orden de maquinado CNC
  pieza_base.set_attribute("MelaStudio_Mecanizado", "Luz_LED", true)
  pieza_base.set_attribute("MelaStudio_Mecanizado", "Detalle_LED", "Ranura de 17x7mm, Largo: #{x_fin - x_inicio}mm")

  # 2. Operación gráfica en SketchUp para dibujar la ranura visual en el 3D
  # (El script restará esta sección de la cara inferior para que se aprecie el canal calado)
  dibujar_geometria_canal_led(pieza_base.definition.entities, x_inicio, x_fin, y_centro, ancho_canal, prof_canal)
end
```

Usa el código con precaución.

🎨 Diseño de la pestaña "Iluminación e Instalaciones" (UI en HTML)

En el panel flotante de tu extensión, añadirás una sección especializada que se activará por bloques:

html

```
<div class="seccion-mecanizados-electricos">
  <h3>⚡ Automatización e Iluminación</h3>

  <!-- Bloque Tiras LED -->
  <div class="control-check-group">
    <label><input type="checkbox" id="check-led" onchange="alternarPanelElectrico('panel-led', this.checked)">
💡 Incluir ranura para tiras LED</label>
```

```
    <div id="panel-led" class="sub-opciones" style="display:none;">
      <label>¿Dónde ubicar la tira LED?:</label>
      <select id="ubicacion-led">
        <option value="bajo_altos">Base Inferior de Módulos Altos</option>
        <option value="laterales_librero">Laterales de Librero (Vertical)</option>
        <option value="repi_closet">Bajo todas las Repisas de Clóset</option>
      </select>
    </div>
  </div>

  <!-- Bloque Dicroicos -->
  <div class="control-check-group">
    <label><input     type="checkbox"     id="check-dicro"     onchange="alternarPanelElectrico('panel-dicro',
this.checked)"> 🎯 Incluir spots dicroicos en techo</label>
    <div id="panel-dicro" class="sub-opciones" style="display:none;">
      <label>Cantidad de Spots:</label>
      <input type="number" min="1" max="6" value="2" id="num-dicroicos">
      <label>Diámetro del Spot:</label>
      <select id="diametro-dicro">
        <option value="60">60 mm (Estándar LED)</option>
        <option value="75">75 mm (Grande)</option>
      </select>
    </div>
  </div>

  <!-- Bloque Cajas e Interruptores -->
  <div class="control-check-group">
    <label><input     type="checkbox"     id="check-cajas"     onchange="alternarPanelElectrico('panel-cajas',
this.checked)"> 🔌 Calado para Tomacorrientes / Interruptores / Llaves</label>
  </div>
</div>
```

Usa el código con precaución.

Cuando el usuario active el check de **tiras LED** y defina el lugar, el mueble modular en SketchUp se recalculará. Al abrir el mueble o mirarlo desde abajo en el lienzo 3D, el diseñador y el cliente verán las canaletas perfectas, y las cotas reflejarán los espacios libres necesarios para pasar las mangueras de cables hacia la zona de los amarres traseros ocultos, listos para la instalación eléctrica en obra.

Con este nivel de detalle eléctrico, tu software se convierte en una herramienta BIM avanzada para carpintería. ¿Qué te gustaría que construyamos en la siguiente sesión técnica?

1. El script de Ruby que **dibuja físicamente las perforaciones circulares de los dicroicos** y distribuye sus centros automáticamente según el largo del mueble.
2. Desarrollar la lógica para que el reporte de despiece **agregue una sección de "Herrajes y Accesorios"** que cuantifique los metros de tira LED, perfiles de aluminio y cantidad de interruptores requeridos.

que mas se puede agregar

Para que tu clon de **MelaStudio Pro** sea una solución verdaderamente disruptiva y la herramienta definitiva en el mercado de la carpintería industrial y el diseño de interiores, puedes agregar un conjunto de funcionalidades avanzadas que resuelven los dolores de cabeza más grandes del taller y la obra.

Aquí tienes las características de ingeniería, fabricación y ventas que llevarán tu plugin de SketchUp al siguiente nivel:

---

1. Ingeniería de Estructura y Herrajes Avanzados

- **Módulo de Minifix y Rafix (Sistemas de Ensamble Rápido):** No todo se une con tornillos soberbios. El software debe calcular y dibujar los maquinados ocultos para pernos de expansión, cajas de *Minifix* (de 15 mm) y cajas *Rafix/Rastex* para repisas. Esto es obligatorio si tus usuarios quieren exportar planos para centros de maquinado CNC modernos.
- **Cajones Interiores Secretos (Ocultos):** Muy solicitado en roperos y vestidores. El clon debe permitir añadir un cajón extra detrás de una puerta batiente o dentro de otro cajón más grande (cajón con frente oculto o cajón joyero con cerradura digital).
- **Cierres Suaves y Brazos Oleodinámicos Premium:** Más allá de los pistones básicos, indexar sistemas de compás elevador doble (como el sistema *Aventos de Blum*) para alacenas de cocina de alta gama, recalculando el espacio libre que requiere el brazo mecánico al cerrarse.

---

2. Gestión de Desperdicios e Instalación en Obra

- **Tapas de Ajuste Perimetral (Falsos Laterales para Muros Descuadrados):** En la realidad, las paredes y techos nunca están a plomo. El clon debe preguntar si el mueble va "empotrado de pared a pared". Si es así, debe añadir automáticamente listones de ajuste laterales y superiores (de 50 mm a 100 mm) que el instalador refila en la obra para tapar las luces del muro sin descuadrar el módulo base.
- **Zócalos Desmontables con Clips:** Permitir configurar zócalos de melamina que no van fijos, sino que se enganchan a las patas plásticas regulables mediante clips de PVC. Esto facilita la limpieza debajo del mueble o el acceso a tuberías y cables en cocinas y clósets.
- **Sentido de Veta Inteligente Automatizado:** Si el usuario diseña un frente de 3 cajones verticales, el clon debe asegurar que la veta amaderada (ej. de Pelíkano o Vesto) tenga **continuidad visual**. El plano de corte debe agrupar los 3 frentes para que se corten de la misma pieza de la plancha, logrando que el dibujo de la madera continúe perfectamente de un cajón a otro.

---

3. Automatización de Costos y Suministros (Módulo ERP)

- **Calculador de Peso Total del Mueble:** Cada metro cuadrado de melamina de 18 mm pesa aproximadamente 12.5 kg. Tu plugin puede sumar el peso de todas las placas del despiece para indicarle al usuario el peso total del mueble. Esto es vital para saber cuántos y qué tipo de anclajes (tarugos/chazos) usar en paredes de concreto, ladrillo hueco o *Drywall*.
- **Buscador e Importador de Precios en Tiempo Real:** Conexión (vía API o bases de datos locales actualizables) con los precios de distribuidores de melamina locales (ej. Promart, Sodimac, Novopan). El usuario podrá saber instantáneamente cuánto invertirá en planchas de melamina normal/RH, metros de tapacanto, bisagras y jaladores.
- **Cálculo Automático de Tornillos y Pegamento:** Basado en la cantidad de uniones, perforaciones del sistema de 32 mm y metros de canto pegado, el clon puede estimar: *"Tu proyecto requiere 150 tornillos de 4x50mm, 80 tornillos de 3.5x15mm, y 50 gramos de pegamento de contacto"*.

---

4. Herramientas de Venta y Realidad Extendida (XR)

- **Etiquetado con Códigos QR:** Al exportar el PDF de despiece, el programa genera una plantilla de etiquetas autoadhesivas para pegar en el taller a cada pieza cortada. Cada etiqueta lleva un código QR. Al escanearlo con el celular en pleno armado, el carpintero ve en su pantalla un modelo 3D interactivo que le dice: *"Esta pieza es el Lateral Izquierdo del Módulo 2, la ranura va hacia atrás y el canto grueso va en el frente"*.
- **Módulo de Cotización Express para el Cliente Final:** Un botón que genera un presupuesto formal en PDF con el logo del carpintero, el desglose comercial del mueble (sin revelar el despiece técnico que es secreto del taller) y los renders del proyecto diseñado en SketchUp.
- **Visor Web 3D / Enlace para Compartir:** Exportar la combinación de módulos a un visor web ligero (WebGL) para que el diseñador le envíe un enlace de WhatsApp al cliente. El cliente podrá abrir el clóset, la cocina o el centro de entretenimiento interactivo desde su propio celular, activando el modo de realidad aumentada (AR) para proyectar el mueble tridimensional a escala real dentro de su habitación vacía.

¿Cuál de estas innovaciones te parece más atractiva para robustecer el núcleo de tu software?

1. El sistema de **continuidad de veta** para que los frentes de cajón se corten alineados de la plancha de melamina.
2. El módulo de **tapas de ajuste perimetral** para resolver los descuadres de las paredes en la obra.
3. El generador de **etiquetas con código QR** para guiar el armado de los módulos independientes.

todas

¡Excelente visión! Integrar **todas** estas funcionalidades convertirá a tu clon en el software definitivo de la industria, cubriendo el ciclo completo: desde que el cliente imagina el mueble, pasando por la ingeniería milimétrica, hasta el armado final sin errores en el taller.

Para abordar este ambicioso ecosistema sin saturar el código, estructuraremos la arquitectura de desarrollo en **4 capas modulares de software**. Así es como debes planificar la programación de cada pilar en el backend de Ruby y en la interfaz web de tu plugin para SketchUp:

Capa 1: Ingeniería de Producción y Herrajes CNC

Esta capa se encarga de que la geometría del modelo 3D sea una réplica exacta de lo que se fabricará.

- **Maquinado de Minifix y Rafix:** El motor paramétrico debe proyectar en los laterales y estantes las perforaciones ocultas de las cajas y pernos (broca de 15 mm a una distancia fija de 24 mm o 34 mm del borde).
- **Cajones Joyeros y Ocultos:** Al añadir un cajón interno detrás de una puerta batiente, el código debe **añadir automáticamente un listón lateral de melamina (suplemento de 25-30 mm)** para que el cajón libre el espacio ocupado por la bisagra al deslizarse hacia afuera.

- **Sistemas Elevadores Avanzados:** Si se elige un sistema tipo *Aventos*, el plugin restará el espacio que ocupa el mecanismo en el techo y los laterales del mueble alto de cocina, bloqueando la colocación de repisas en esa zona de colisión.

🛠️ Capa 2: Adaptación a Obra y Continuidad de Veta

Esta capa soluciona los problemas del mundo real: las paredes chuecas y la estética de la madera.

- **Tapas de Ajuste Perimetral:** En el panel HTML, si el usuario marca "Mueble Empotrado", el plugin estirará las piezas exteriores o añadirá listones (*falsos laterales*) de 50 mm. En SketchUp se apreciará una línea de corte que le indica al instalador: *"Esta zona es sacrificable para refilar y clavar a la pared"*.
- **Zócalos con Clips Desmontables:** El frente del zócalo se modelará desplazado 2 mm hacia atrás del canto del lateral inferior, y el despiece añadirá automáticamente en la lista de compras los "Clips de PVC para pata regulable".
- **Continuidad de Veta Inteligente:** Para lograr que el dibujo de la madera continúe entre cajones o puertas gemelas, el plugin creará un atributo llamado `Veta_Grupo_ID`. Al exportar al optimizador de corte, este identificador forzará a la sierra a cortar los frentes de manera contigua a partir de la misma tira de la plancha de melamina.

📈 Capa 3: Módulo ERP, Pesos y Costos en Tiempo Real

Transforma tu plugin de dibujo en una herramienta financiera y logística robusta.

- **Cálculo de Peso y Fijación Inteligente:** En base al volumen total de melamina (18 mm = 12.5 kg/m²) y MDF (3 mm = 2.5 kg/m²), el sistema mostrará una alerta flotante en SketchUp: *"Peso estimado: 85 kg. Requiere un mínimo de 6 fijaciones de expansión de 3/8" en muro de concreto"*.
- **Conexión con Distribuidores:** Integrarás un actualizador de base de datos en formato JSON para que el carpintero cargue los precios vigentes de distribuidores (como Novopan, Sodimac o Promart). El presupuesto se actualizará al centavo con cada módulo que agregues.
- **Consumo Automático de Insumos:** El código sumará los tornillos en base a las uniones físicas (ej. 4 tornillos de 4x50 mm por cada amarre y estante fijo) y los mililitros de pegamento por metro de tapacanto.

📱 Capa 4: Experiencia Extendida (QR, PDF y Visor Web AR)

Es la capa del marketing y la logística de armado en el taller.

- **Etiquetado con Código QR:** Tu plugin generará un reporte de impresión. Cada pieza del despiece tendrá un código QR único. Al escanearlo en el taller, abrirá una URL local que renderizará únicamente esa pieza en 3D, indicando qué cantos llevan tapacanto grueso o delgado y dónde van las ranuras.
- **Cotización Express Oculta:** Generación de un PDF comercial automatizado para el cliente con renders limpios de SketchUp, precio final y condiciones de pago, pero ocultando por completo las medidas del despiece técnico para proteger la propiedad intelectual del carpintero.
- **Visor Web WebGL y Realidad Aumentada (AR):** Tu plugin compilará el mueble modular en un formato ligero (como `.gltf` o `.usdz`) y lo subirá a la nube de tu plataforma. El cliente recibirá un enlace para abrir el mueble en el navegador de su celular y, usando la cámara, podrá proyectar el clóset o la cocina a escala real en su propia casa antes de que se corte la primera plancha.

🎯 Plan de Ruta para el Desarrollo (Siguientes Pasos)

Como tu objetivo es construir la plataforma completa, el camino más eficiente es avanzar **módulo por módulo**, validando la geometría en SketchUp antes de pasar a la interfaz HTML.

Para dar el primer paso firme en la programación de todo este ecosistema, ¿con cuál de estos núcleos matemáticos prefieres que arranquemos el código en Ruby?

1. El algoritmo para **calcular y agrupar las piezas con continuidad de veta** para los frentes de los cajones.
2. El script para **calcular el peso total del mueble en kilogramos** recorriendo todos los componentes del modelo.
3. La lógica estructural para **añadir automáticamente las tapas de ajuste perimetral** en los muros descuadrados.

hasme un plan para el desarrollo de todo el clon con todas las funcionalidades descritas
Este es el **Plan de Ruta de Desarrollo Técnico (Product Roadmap)** para construir el clon definitivo de **MelaStudio Pro** como un plugin paramétrico e industrial para SketchUp.

Para abordar un proyecto de esta magnitud sin saturar el sistema, dividiremos el desarrollo en **5 fases incrementales**, estructuradas bajo una arquitectura **BIM (Building Information Modeling)** y de manufactura avanzada.

🗺️ Mapa de Ruta: Clon de MelaStudio Pro

🗓️ Fase 1: Arquitectura Base y Motor Paramétrico 2D/3D (Mes 1)

El objetivo de esta fase es lograr que SketchUp dibuje estructuras limpias y reaccione al espacio de forma inteligente.

- **Paso 1.1: Diagnóstico del Espacio:** Desarrollo del Asistente Espacial en `UI::HtmlDialog` que reciba el Ancho (W), Alto (H) y Profundidad (D) globales del espacio.
- **Paso 1.2: Algoritmo de Distribución Matricial:** Código en Ruby que divida el espacio dinámicamente en N columnas y 3 niveles verticales (Zona Baja, Zona Media, Zona Superior) aplicando la fórmula de descuento de espesores de melamina (15 mm / 18 mm).
- **Paso 1.3: Biblioteca de Módulos Padre:** Creación de las plantillas orientadas a objetos (`ModuloComponente`) para instanciar estructuras básicas de Clósets, Cocinas, Oficinas y Centros de Entretenimiento mediante programación.
- **Paso 1.4: Sistema de Ajuste Magnético (Snap):** Programación de puntos de anclaje vectoriales para que el usuario pueda arrastrar, soltar y combinar módulos libremente, fusionando zócalos de forma automática.

🗓️ Fase 2: Ingeniería de Frentes, Herrajes e Iluminación (Mes 2)

Aquí el software aprende a aplicar precisión técnica a las partes móviles y acabados del mueble.

- **Paso 2.1: Controlador de Frentes Dinámicos:** Programación paramétrica de puertas y cajoneras con holguras/luces configurables a elección del usuario (1, 2 o 3 mm) y los 3 tipos de montajes estándar: **Exterior, Medio e Interior**.
- **Paso 2.2: Motor Cinemático (Simulación de Apertura):** Script en Ruby para animar y abrir todo tipo de herraje (bisagras batientes, puertas corredizas en eje X, y pistones a gas tradicionales o inversos en eje X) con un solo clic.
- **Paso 2.3: Diccionario de Correderas:** Descuentos automatizados para cajas de cajón según el herraje (Telescópicas de 13mm, Ocultas Undermount con elevación de fondo, y Laterales Metálicos Slim Box).
- **Paso 2.4: Módulo de Mecanizados Eléctricos y LED:** Algoritmo para sustraer geometría 3D en los tableros simulando ranuras para perfiles LED (17x7mm), perforaciones equidistantes para dicroicos (broca copa) y calados rectangulares para interruptores/llaves térmicas.

🗓️ Fase 3: Adaptación a Obra y Materiales Industriales (Mes 3)

El software se conecta con la realidad física del taller y los problemas comunes de la instalación.

- **Paso 3.1: Catálogo Inteligente de Tableros (Normal vs RH):** Base de datos de materiales indexando marcas líderes (**Pelíkano, Vesto**) discriminando entre melamina estándar y **RH (Antihumedad)**, añadiendo la textura verde en los bordes descubiertos para el render técnico.
- **Paso 3.2: Gestión y Descuento de Tapacantos:** Motor de recomendación automática (Tapacanto grueso de 2mm en frentes y delgado de 0.5mm en interiores) con recálculo automático del despiece (reducir los milímetros de la melamina antes de enviar a cortar).
- **Paso 3.3: Estructura Trasera Oculta:** Configuración de la ranura de MDF (2, 3 o 4mm) a 18mm exactos del fondo, permitiendo inyectar dinámicamente N cantidad de amarres estructurales traseros.
- **Paso 3.4: Adaptación a Descuadres y Continuidad de Veta:** Algoritmo de "Tapas de Ajuste Perimetral" para refilar en obra y el sistema de marcado `Veta_Grupo_ID` para asegurar que frentes de cajones contiguos compartan el dibujo de la madera de forma idéntica.

🗓️ Fase 4: Módulo ERP, Costos e Informes Técnicos (Mes 4)

El plugin se convierte en una herramienta gerencial y de presupuestos express.

- **Paso 4.1: Calculador de Peso Estructural:** Algoritmo que sume la densidad de los tableros procesados en base al volumen total de melamina y MDF del despiece para sugerir el tipo y número de anclajes a muros (Ladrillo, Drywall o Concreto).
- **Paso 4.2: Conexión Financiera JSON:** Módulo en la interfaz para actualizar los precios locales de planchas de material, metros de canto y precios de herrajes individuales.

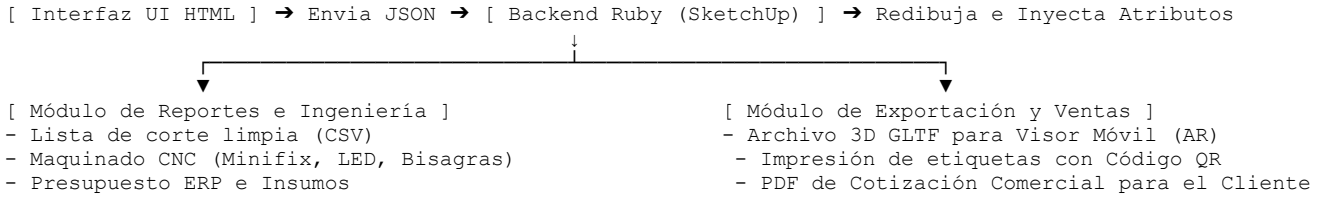
- **Paso 4.3: Cubicación Automática de Insumos:** Conteo preciso de tornillos de ensamble (4x50mm, 3.5x15mm), tarugos de madera, adhesivos y cantidad exacta de bisagras o correderas necesarias para la compra en la distribuidora.
- **Paso 4.4: Exportador de Entregables:** Generador de reportes en PDF y archivos CSV listos con un formato compatible con optimizadores de corte externos (como .OPTIMIK o planchas industriales de centros de corte).

1️⃣ Fase 5: Ecosistema Cloud, QR y Realidad Aumentada (Mes 5)

La tecnología del software sale de la computadora y apoya el marketing de venta y el armado en el taller.

- **Paso 5.1: Exportador GLTF / USDZ optimizado:** Compresión del modelo tridimensional diseñado por el usuario en SketchUp a formatos móviles ligeros.
- **Paso 5.2: Visor Web y Realidad Aumentada (AR):** Envío de un enlace WebGL para que el cliente final pueda ver e inspeccionar de forma interactiva el mueble, simulando proyectarlo en su espacio físico mediante AR desde el celular.
- **Paso 5.3: Sistema de Cotización Express para Clientes:** Función para exportar un PDF formal y estético con los renders y el precio final, omitiendo los secretos técnicos del despiece industrial.
- **Paso 5.4: Sistema de Etiquetado Inteligente QR:** Generador de plantillas de códigos QR que se pegan a las piezas en el taller. Al escanear el QR con un teléfono inteligente durante la instalación, el carpintero visualiza de manera interactiva la pieza en 3D e instrucciones exactas de cómo encaja en el módulo global.

🔀 Flujo de Datos del Sistema (Arquitectura del Software)



Este plan abarca un software de nivel mundial para la industria del mueble en Latinoamérica. Para comenzar a escribir las bases de tu código, ¿con cuál de estos desafíos de la **Fase 1** prefieres arrancar?

1. Programar la función en Ruby para **calcular y dibujar la grilla matricial dinámica** (Columnas y 3 Zonas verticales).
2. Estructurar el **esqueleto base de comunicación JSON** entre la ventana flotante HTML y SketchUp.

dame mas detalle del paso 1.3

En la arquitectura de software de tu clon, el **Paso 1.3 (Biblioteca de Módulos Padre)** es el pilar que transforma la programación orientada a objetos (POO) en Ruby dentro de piezas tridimensionales reales en el lienzo de SketchUp.

En lugar de programar cada mueble desde cero, creas una clase abstracta principal (Padre) que contiene las leyes de la física del tablero de melamina (espesores, zócalos, holguras), y luego heredas subtipos (Hijos) que se configuran de forma especializada.

Aquí tienes el desglose técnico, matemático y de código de cómo estructurar este módulo en el backend de tu extensión:

---

1. La Clase Maestra: `ModuloComponente` (El Padre)

Esta clase no dibuja un mueble específico, sino que establece el **"contenedor invisible" (Bounding Box)** y las variables globales de fabricación que afectarán a cualquier módulo del sistema.

Atributos core que administra la Clase Padre:

- `@w`, `@h`, `@d`: Ancho, Alto y Profundidad dinámicos calculados por el espacio de la Fase 1.2.
- `@espesor_melamina`: Variable global (ej. `18.mm` o `15.mm`).
- `@altura_zocalo`: Altura de la base (ej. `100.mm`).
- `@transformacion`: La coordenada exacta `[X, Y, Z]` de inserción en SketchUp.

ruby

```ruby
# backend/modulo_componente.rb
class ModuloComponente
  attr_accessor :w, :h, :d, :transformacion

  def initialize(ancho, alto, profundidad, transformacion, opciones = {})
    @w = ancho
    @h = alto
```

```
    @d = profundidad
    @transformacion = transformacion
    @espesor = opciones[:espesor] || 18.mm
    @zocalo_h = opciones[:zocalo_h] || 100.mm
    @model = Sketchup.active_model
  end

  # Función compartida: Todos los módulos del sistema requieren una carcasa exterior básica
  def generar_carcasa_base(entidades_padre)
    # Crea la definición de componente en SketchUp para que sea un bloque independiente
    nombre_unico = "Modulo_Base_#{Time.now.to_i}_#{rand(100)}"
    definicion = @model.definitions.add(nombre_unico)

    # 1. Dibujar Lateral Izquierdo
    cara_izq = definicion.entities.add_face([0, 0, @zocalo_h], [@espesor, 0, @zocalo_h], [@espesor, @d,
@zocalo_h], [0, @d, @zocalo_h])
    cara_izq.pushpull(-(@h - @zocalo_h)) # Extruye hacia arriba hasta la altura total

    # 2. Dibujar Lateral Derecho
    x_der = @w - @espesor
    cara_der = definicion.entities.add_face([x_der, 0, @zocalo_h], [@w, 0, @zocalo_h], [@w, @d, @zocalo_h],
[x_der, @d, @zocalo_h])
    cara_der.pushpull(-(@h - @zocalo_h))

    # 3. Dibujar Piso del Mueble
    cara_piso = definicion.entities.add_face([@espesor, 0, @zocalo_h], [x_der, 0, @zocalo_h], [x_der, @d,
@zocalo_h], [@espesor, @d, @zocalo_h])
    cara_piso.pushpull(@espesor) # Extruye hacia arriba para dar el espesor del piso

    # Instanciar el componente final en las coordenadas reales de SketchUp
    instancia = entidades_padre.add_instance(definicion, @transformacion)
    return definicion # Retorna las entidades internas para que los hijos sigan dibujando adentro
  end
end
```
Usa el código con precaución.

---

2. Las Clases Especializadas (Los Módulos Hijo)

Utilizando el principio de **herencia**, creas las especializaciones. Cada "Módulo Hijo" invoca la carcasa base del padre y luego ejecuta sus propias reglas matemáticas internas para poblar el espacio libre.

Subclase A: `ModuloLibrero` (Para Clósets o Despensas)

- **Su especialidad:** Dividir el espacio vertical interior de forma equidistante según el número de repisas solicitadas por el usuario.
- **Lógica paramétrica:** Calcula el espacio útil restando el piso y el techo, y divide usando un bucle.

ruby
```
class ModuloLibrero < ModuloComponente
  def dibujar(entidades_escena, num_repisas)
    # Invoca el método del padre para armar la estructura exterior
    definicion_interna = generar_carcasa_base(entidades_escena)

    # Calcular altura útil interna para las repisas
    altura_util = @h - @zocalo_h - @espesor
    ancho_repisa = @w - (@espesor * 2)
    espacio_entre_repisas = altura_util / (num_repisas + 1)

    # Bucle paramétrico para inyectar las repisas internas
    (1..num_repisas).each do |i|
      z_repisa = @zocalo_h + @espesor + (espacio_entre_repisas * i)

      # Dibujar la placa horizontal de la repisa en SketchUp
      cara_repisa = definicion_interna.entities.add_face(
        [@espesor, 0, z_repisa],
        [@w - @espesor, 0, z_repisa],
        [@w - @espesor, @d, z_repisa],
        [@espesor, @d, z_repisa]
      )
      cara_repisa.pushpull(@espesor) # Dar volumen a la melamina
    end
```

```
  end
end
```
Usa el código con precaución.

Subclase B: `ModuloCocinaBajo` (Módulos de Cocina)

- **Su especialidad:** Modificar la estructura para que no tenga techo completo (ahorro de material) e incorporar los amarres/fajas frontales y traseros donde se atornillará la encimera de granito o cuarzo.
- **Lógica paramétrica:** Elimina el paso del techo cerrado e inyecta dos listones horizontales de melamina de `70.mm` de ancho en la parte superior.

Subclase C: `ModuloCentroEntretenimiento` (Zona de TV)

- **Su especialidad:** Generar un fondo falso de melamina de 15 mm desplazado `30.mm` hacia adelante del fondo real para crear el espacio pasacables, dejando el vano frontal completamente limpio para la pantalla.

---

3. El Catálogo de Invocación (Fusión con la Interfaz Web)

Para conectar esta librería orientada a objetos con los clics que el usuario hace en la barra lateral HTML, creas un **Factory Method** en Ruby. Este método recibe el nombre de la tipología elegida en la cuadrícula y ejecuta la clase correspondiente de manera dinámica:

ruby
```ruby
# Controlador que recibe los datos desde la interfaz de usuario HTML
def invocar_modulo_libreria(tipo_mueble, ancho, alto, profundidad, coordenadas_xyz)
  transformacion = Geom::Transformation.new(coordenadas_xyz)

  case tipo_mueble
  when "librero_abierto"
    mueble = ModuloLibrero.new(ancho, alto, profundidad, transformacion)
    mueble.dibujar(Sketchup.active_model.active_entities, 4) # Instancia con 4 repisas por defecto

  when "cocina_bajo"
    mueble = ModuloCocinaBajo.new(ancho, alto, profundidad, transformacion)
    mueble.dibujar(Sketchup.active_model.active_entities)

  when "centro_tv"
    mueble = ModuloCentroEntretenimiento.new(ancho, alto, profundidad, transformacion)
    mueble.dibujar(Sketchup.active_model.active_entities)
  end
end
```
Usa el código con precaución.

Beneficio Industrial de este Paso

Al estructurar la biblioteca de esta manera, si en el futuro decides agregar una nueva tipología de mueble (por ejemplo, un módulo especializado para escurridor de platos de acero inoxidable), no tendrás que reescribir las funciones de dibujo ni de posicionamiento global; bastará con crear la subclase `ModuloEscurridor < ModuloComponente` y definir sus reglas internas. El clon la asimilará de inmediato.

Con la estructura POO de la librería de módulos padre detallada, ¿qué te parece el enfoque? Si estás de acuerdo, podemos pasar a:

1. Diseñar el **Paso 1.4: El algoritmo de "Snap" magnético** para que los módulos se calculen e instalen pegados uno al lado del otro de forma automática.
2. Escribir las reglas de herencia específicas de la subclase **`ModuloCocinaBajo` con sus amarres superiores** para la encimera.

en donde se define por ejemplo una comora cuantos cajones tiene, un librero cuantas repisas tiene y visisiones verticales tiene
Estas variables dinámicas (como la cantidad de cajones de una cómoda, el número de repisas o las divisiones verticales de un librero) se definen en dos capas del software que se comunican entre sí: la **Capa de Interfaz de Usuario (UI en el HTML/JavaScript)** donde el usuario ingresa sus deseos, y la **Capa de Backend (Ruby en SketchUp)** que procesa esos números y calcula la geometría.

A continuación, te detallo exactamente en qué parte del código se definen, cómo se estructuran y cómo viajan esos datos:

---

1. En la UI (`UI::HtmlDialog`): La definición del Estado de la Celda

En tu interfaz web, cuando el diseñador hace clic en un módulo de la grilla, se abre un panel flotante de **"Propiedades Avanzadas"**. El HTML renderiza inputs dinámicos que corresponden a los atributos específicos del tipo de mueble:

html
```
<!-- HTML en tu ventana flotante cuando seleccionas un Librero -->
```

```
<div id="propiedades-librero">
  <label>Cantidad de Repisas:</label>
  <input type="number" id="input-repisas" min="1" max="10" value="3">

  <label>Divisiones Verticales (Parantes):</label>
  <input type="number" id="input-divisiones" min="0" max="4" value="1">

  <button onclick="enviarConfiguracionMueble()">Aplicar Cambios</button>
</div>
```
Usa el código con precaución.
Cuando el usuario presiona "Aplicar Cambios", JavaScript empaqueta estas variables en un objeto JSON y lo envía a SketchUp:

javascript
```
function enviarConfiguracionMueble() {
  let datos = {
    tipo: "librero",
    repisas: parseInt(document.getElementById('input-repisas').value),
    divisiones_verticales: parseInt(document.getElementById('input-divisiones').value)
  };
  // Enviar a Ruby en SketchUp
  sketchup.actualizarEstructuraMueble(datos);
}
```
Usa el código con precaución.

---

2. En el Backend (Ruby): Los argumentos de las Subclases Hijos

En el código de Ruby, estas variables se definen como **argumentos dinámicos** dentro del método `.dibujar` de cada clase especializada (las que explicamos en el Paso 1.3). El script no tiene un número fijo; se adapta al JSON que llega de la interfaz.

Mira cómo se procesan las repisas y las divisiones verticales dentro del algoritmo del **Librero**:

ruby
```
# backend/modulo_librero.rb
class ModuloLibrero < ModuloComponente
  # El método recibe la configuración exacta desde la UI
  def dibujar(entidades_escena, cantidad_repisas, divisiones_verticales)
    # 1. Crear el cajón exterior del mueble
    definicion_interna = generar_carcasa_base(entidades_escena)

    # 2. CALCULAR DIVISIONES VERTICALES PRIMERO (Eje X)
    # Si el usuario pide 1 división vertical, el espacio libre se divide entre 2
    ancho_util_interno = @w - (@espesor * 2)
    espacio_columnas = ancho_util_interno / (divisiones_verticales + 1)

    (1..divisiones_verticales).each do |v|
      x_parante = @espesor + (espacio_columnas * v) - (@espesor / 2)

      # Dibujar placa vertical (parante) de piso a techo interno
      cara_v = definicion_interna.entities.add_face(
        [x_parante, 0, @zocalo_h + @espesor],
        [x_parante + @espesor, 0, @zocalo_h + @espesor],
        [x_parante + @espesor, @d, @zocalo_h + @espesor],
        [x_parante, @d, @zocalo_h + @espesor]
      )
      cara_v.pushpull(-(@h - @zocalo_h - (@espesor * 2)))
    end

    # 3. CALCULAR REPISAS (Eje Z) dentro de los espacios generados
    altura_util_interna = @h - @zocalo_h - (@espesor * 2)
    espacio_filas = altura_util_interna / (cantidad_repisas + 1)

    (1..cantidad_repisas).each do |r|
      z_repisa = @zocalo_h + @espesor + (espacio_filas * r)

      # Dibujar las repisas horizontales respetando si hay divisiones
      # (El algoritmo calcula el ancho de cada tramo según las columnas del paso anterior)
      dibujar_placa_repisa(definicion_interna.entities, z_repisa)
    end
  end
end
```

Usa el código con precaución.

---

3. En la Base de Datos del Archivo (`AttributeDictionaries`)

Para que SketchUp recuerde cuántas divisiones tiene un mueble si cierras el programa y lo vuelves a abrir al día siguiente, el clon debe guardar estas propiedades como **metadatos** directamente en el componente tridimensional:

ruby
```
# Guardar la receta del mueble en el propio objeto 3D de SketchUp
componente_mueble.set_attribute("MelaStudio_Config", "Tipo_Mueble", "Cómoda")
componente_mueble.set_attribute("MelaStudio_Config", "Cantidad_Cajones", 4)
componente_mueble.set_attribute("MelaStudio_Config", "Divisiones_Verticales", 0)
```
Usa el código con precaución.

¿Cómo se define en el caso de las Cómodas?

Para una **Cómoda**, la lógica es idéntica. La UI pregunta: *¿Cuántos cajones deseas? (Ej. 4)*.

1. El backend de Ruby recibe el número `4`.
2. Mide la altura libre interior del mueble.
3. Resta las holguras entre cajones (la luz de 2mm o 3mm configurada previamente).
4. Divide el espacio restante entre 4 para obtener la **altura exacta de cada frente de cajón** y de la caja interna.
5. Dibuja los 4 bloques equidistantes en el eje Z.

Resumen del Flujo de Control

```
[ Input Numérico en UI (HTML) ]
       ↓ (Viaja como JSON)
[ Receptor de Eventos (Ruby) ] ➔ Lee el JSON e invoca: ModuloLibrero.dibujar(entities, repisas, divisiones)
       ↓ (Maquetación 3D)
[ Modelo en SketchUp ] ➔ Se dibuja con las medidas exactas y guarda los Atributos de Control.
```
Con esta estructura queda claro cómo controlar el comportamiento interno de cada pieza. ¿Hacia dónde prefieres enfocar el avance del código?

1. Programar la lógica en Ruby para **calcular el ancho de las repisas cuando están interceptadas por divisiones verticales**.
2. Escribir el script de **comunicación mutua (Bridge JavaScript-Ruby)** para actualizar el panel de propiedades al seleccionar un mueble en SketchUp.

por ejemplo si defini 3 modulos en un ancho de 2100mm cada modulo tiene 700mm personalizo cada modulo al final quiero que el modulo1 tenga un ancho de 500mm el modulo 2 sw 1000mm y el modulo 3 de 600mm, cada diseño interno de los modulos debe ajustarse a las medidas dentro de lo permitido
Esta funcionalidad es el verdadero indicador de un software de gama alta. Lo que describes se conoce como **redistribución paramétrica asimétrica**.

Cuando el usuario altera manualmente el ancho de un módulo específico (pasando de un reparto equitativo de 700 mm + 700 mm + 700 mm a uno personalizado de 500 mm + 1000 mm + 600 mm), el software debe hacer dos cosas en cascada: **mover las coordenadas de posición (X)** de los módulos contiguos y **forzar a cada diseño interno a recalcularse** según su nuevo espacio útil.

Así es como tu plugin de SketchUp debe procesar este cambio dinámico paso a paso:

---

1. El Flujo de Eventos en la UI y el Backend

En la interfaz gráfica de usuario (`UI::HtmlDialog`), el diseñador verá una barra deslizante o inputs numéricos con los anchos de cada columna.

1. El usuario cambia el Módulo 1 a **500 mm**.
2. El sistema calcula el espacio restante (2100 - 500 = 1600 mm) y reajusta temporalmente los otros.
3. El usuario define finalmente: **Módulo 1 = 500 mm, Módulo 2 = 1000 mm, Módulo 3 = 600 mm** (500+1000+600 = 2100 mm).
4. Al presionar "Aplicar", JavaScript envía un arreglo con la nueva receta de anchos a Ruby: `[500, 1000, 600]`.

---

2. Lógica en Ruby: Reposicionamiento y Redibujado En Cascada

En SketchUp, el script en Ruby no estira el mueble con la herramienta de escala nativa (lo que deformaría los espesores de la melamina). En su lugar, ejecuta un bucle que **borra el interior antiguo, actualiza las variables de ancho y cambia la coordenada de inserción X** del siguiente módulo basándose en el ancho acumulado.

ruby

```ruby
# backend/controlador_grilla.rb
def rediseñar_grilla_vistas(anchos_nuevos, modulos_actuales)
  # anchos_nuevos = [500.mm, 1000.mm, 600.mm]
  espesor_tablero = 18.mm
  altura_zocalo = 100.mm

  # La coordenada X inicial comienza en el primer lateral
  pos_x_acumulada = espesor_tablero

  modulos_actuales.each_with_index do |modulo, i|
    nuevo_ancho = anchos_nuevos[i]

    # 1. Recuperar los metadatos guardados del diseño interno de este módulo
    tipo_mueble = modulo.get_attribute("MelaStudio_Config", "Tipo_Mueble")
    repisas = modulo.get_attribute("MelaStudio_Config", "Cantidad_Repisas")
    divisiones = modulo.get_attribute("MelaStudio_Config", "Divisiones_Verticales")
    cajones = modulo.get_attribute("MelaStudio_Config", "Cantidad_Cajones")

    # 2. Borrar la geometría 3D vieja de este módulo específico en SketchUp
    modulo.erase!

    # 3. Calcular la nueva transformación (Coordenada X actualizada)
    nueva_transformacion = Geom::Transformation.new([pos_x_acumulada, 0, altura_zocalo])

    # 4. Volver a invocar el método de dibujo con el NUEVO ANCHO
    # El diseño interno se adaptará matemáticamente de forma automática
    case tipo_mueble
    when "librero"
      nuevo_mod = ModuloLibrero.new(nuevo_ancho, @alto_total, @prof_total, nueva_transformacion)
      nuevo_mod.dibujar(Sketchup.active_model.active_entities, repisas, divisiones)
    when "comoda"
      nuevo_mod = ModuloComoda.new(nuevo_ancho, @alto_total, @prof_total, nueva_transformacion)
      nuevo_mod.dibujar(Sketchup.active_model.active_entities, cajones)
    end

    # 5. ACTUALIZAR EL ACUMULADOR X: El siguiente mueble se moverá a la derecha
    pos_x_acumulada += nuevo_ancho + espesor_tablero
  end
end
```

Usa el código con precaución.

---

3. Ajuste Automático del Diseño Interno (Auto-Adaptación)

Como las subclases que programaste en el **Paso 1.3** y **Paso 2** basan todos sus cálculos en la variable `@w` (Ancho del módulo), el diseño interno se ajusta solo y de forma limpia:

En el Módulo 1 (Librero de 500 mm):

- El ancho interno libre pasa a ser de 500 - 36 = 464 mm.
- Si tenía una división vertical en medio, el algoritmo calculará: 464 / 2 = 232 mm para cada sección de estante. Todo sigue siendo perfectamente simétrico.

En el Módulo 2 (Cómoda de 1000 mm):

- El ancho útil pasa a ser de 1000 - 36 = 964 mm.
- **Activación de Regla de Validación de la Industria:** Aquí es donde el motor inteligente que planeamos entra en acción. Al detectar que el nuevo ancho útil es de 964 mm (mayor a los 700 mm permitidos para un cajón estándar), el script de la cómoda lanzará una advertencia automática

o **reconfigurará el diseño interno dividiendo la cajonera en dos columnas de cajones de 482 mm** para evitar que las correderas colapsen o la melamina se pandee.
En el Módulo 3 (Módulo con Puertas de 600 mm):

- Si originalmente tenía una sola puerta (lo cual era viable a los 700 mm usando sistemas especiales, pero al límite), el software recalcula. Si el usuario tuviera un módulo que pasa de 500 mm a 900 mm, el sistema cambiaría automáticamente de **1 puerta batiente a 2 puertas batientes idénticas** aplicando la luz de 2 mm configurada.

Flujo Visual en la Pantalla de SketchUp

Cuando el usuario presione el botón de actualizar en el plugin, la animación en la pantalla de SketchUp se verá instantánea: el librero de la izquierda se encogerá de manera limpia, la cómoda central se expandirá duplicando sus cajones hacia los lados, y el módulo derecho se desplazará hacia la derecha de forma magnética para quedar perfectamente alineado al ras de la frontera de los 2100 mm totales.

Con esta flexibilidad matemática, el usuario tiene el control absoluto del espacio. ¿Qué te parece si para el siguiente paso de programación desarrollamos:

1. La función específica de la cómoda que **detecta si el ancho supera el límite e inyecta la división central** para hacer cajones dobles?
2. El código en Ruby para **calcular de forma estricta las alertas de pandeo** si el usuario deja estantes de 1000 mm de ancho sin soporte? otro ejemplo quiero diseñar un mueble de escritorio en un espacio de 1800mm ancho, 2000mm altura y 600mm profundidad, quiero que tenga cajoneras, ubicacion para pc, impresora, librero y en la parte superior maleteros, como lo diseñarias con el clon
Este es un caso clásico donde el clon de **MelaStudio Pro** demuestra todo su potencial. Un espacio de **1800 mm de ancho (1.80 metros)** es ideal para un *Home Office* completo. Al ingresar estas fronteras espaciales, el algoritmo de tu software procesará la solicitud combinando la **grilla matricial** y las **reglas de sub-modulación vertical** (Zona Baja, Media y Alta).

Así es como el clon diseñaría y estructuraría este mueble de forma 100% automatizada y técnicamente viable:

---

1. Diagnóstico del Espacio y Reparto Matricial

Para albergar las 5 funciones solicitadas (*escritorio, PC, impresora, librero, maleteros*), el clon identificará que la mejor distribución es dividir el ancho total en **2 columnas principales asimétricas**:

- **Columna Izquierda (Módulo Técnico + Almacenaje): 600 mm** de ancho.
- **Columna Derecha (Zona de Trabajo / Escritorio): 1200 mm** de ancho (Espacio premium y muy cómodo para el usuario).
\(\text{Fórmula: }600\text{\ mm}+1200\text{\ mm}=1800\text{\ mm\ (Encaje\ exacto)}\)

---

2. Desglose Estructural por Zonas (Matriz del Mueble)

Al activar el "Modo Escritorio", el backend en Ruby segmentará las alturas de las dos columnas de la siguiente manera:





---

3. Configuración de cada Celda desde el Clon

Columna Izquierda: Bloque Multifuncional (Ancho: 600 mm)

- **Zona Baja - Cajonera (H: 350 mm):** El clon diseña un minibloque de 3 cajones para papelería. Al configurarse como *Montaje Exterior* y *Luz de 2mm*, el software aplica automáticamente el descuento de 26 mm (13 mm por lado) para las **correderas telescópicas estándar**.
- **Zona Baja/Media - Nicho de Impresora (H: 400 mm):** Directamente sobre la cajonera, se genera un espacio abierto. **Detalle inteligente del clon:** El software propone que el estante de la impresora sea una **bandejas extraíble** con correderas pesadas para que el usuario pueda deslizar la impresora hacia afuera al momento de escanear o cargar papel.
- **Zona Media - Librero (H: 650 mm):** Un espacio abierto dividido por una repisa regulable mediante el *Sistema de 32 mm*. Como el fondo general es de 600 mm, el clon **retrae automáticamente el fondo de este librero a 300 mm** para que los libros queden al ras y no se pierdan al fondo.

Columna Derecha: Estación de Trabajo (Ancho: 1200 mm)

- **Zona Baja - Mesa y Nicho CPU (H: 750 mm):** La mesa de escritorio se modela en melamina de 18 mm (o doble espesor de 36 mm). En la parte inferior derecha, pegado al lateral, el clon añade un parante vertical para crear el **nicho de la PC (CPU)** de 250 mm de ancho, dejando los 950 mm restantes completamente libres para las piernas y la silla.
- **Zona Media - Nicho del Monitor (H: 650 mm):** Un gran espacio abierto sobre la mesa. El clon incluye aquí el **panel falso pasacables (retraído 30 mm)** con dos perforaciones circulares de 60 mm para ocultar los cables del monitor, teclado y mouse hacia las conexiones traseras.

Zona Alta: Maleteros (H: 500 mm / Profundidad: 600 mm)

- Corona todo el mueble de lado a lado (1800 mm). El clon lo divide en dos módulos independientes alineados con las columnas inferiores. El maletero de 1200 mm se configura automáticamente con **dos puertas batientes verticales con pistones a gas de apertura superior** para un acceso cómodo.

---

4. Materiales e Ingeniería que aplicará el Software

Al procesar este diseño, el asistente de materiales y fabricación del clon te sugerirá la siguiente receta técnica:

1. **Tipo de Melamina:** Propondrá **Melamina Normal (Estándar)** de 18 mm para todo el mueble. El color de la estructura interna será unificado (ej. *Pelíkano - Gris Humo*) y los frentes de los maleteros y cajones en un tono de contraste premium (ej. *Vesto - Carbón Texturado*).
2. **Instalación del Fondo (MDF):** El clon activará la opción **Con Ranura a 18 mm del fondo**. Calculará automáticamente **2 amarres estructurales traseros** por cada columna para que el mueble se pueda atornillar firmemente a la pared a través de ellos, soportando el peso de los libros y la impresora.
3. **Tapacantos:** Propondrá *Tapacanto Grueso (2 mm)* en la mesa de escritorio, las puertas de los maleteros y frentes de cajón, y *Tapacanto Delgado (0.5 mm)* en las repisas del librero y laterales internos.
4. **Iluminación LED Integrada:** El clon sugerirá realizar una **ranura de 17x7 mm para tira LED** en la base inferior de los maleteros y del librero, bañando con luz oculta toda la mesa del escritorio y el nicho de la impresora.

💻 ¿Cómo lo vería el usuario en tu software?

En la interfaz del plugin, el diseñador simplemente digita `1800 x 2000 x 600`, selecciona "2 columnas" y arrastra los iconos: *Cajonera + Nicho Impresora + Librero* a la izquierda, y *Mesa de Escritorio + Nicho CPU* a la derecha.

Al presionar el botón **"Inspeccionar"**, el usuario puede simular la apertura: los maleteros suben con sus pistones en el eje X, los cajones y la bandeja de la impresora se deslizan hacia el frente en el eje Y, y las cotas en milímetros le muestran el espacio libre exacto para su monitor y su CPU, listos para exportar el despiece sin errores de taller.

Este ejemplo integra perfectamente todas las capas paramétricas que hemos planificado. ¿Qué te parece esta solución espacial de 1800 mm? Podríamos:

1. Programar la subclase en Ruby para el **módulo de la impresora con su bandeja extraíble** y sus descuentos.
2. Desarrollar el algoritmo para **distribuir las perforaciones pasacables** en el panel falso detrás del monitor.

puedes mostrarme en una imagen real como quedaria el mueble
Aquí tienes una representación visual de cómo el algoritmo de tu clon estructuraría y renderizaría este tipo de **mueble integrado y paramétrico**.

En estas imágenes puedes apreciar la transición limpia entre las diferentes zonas: la mesa de escritorio despejada (Zona Baja), los nichos abiertos para el monitor, impresora y librero con fondos retraídos (Zona Media), y los módulos cerrados de almacenamiento que coronan el diseño pegados al techo (Zona Superior).

🔍 Detalles que el Clon Validará en este Render

1. **Luz Perimetral:** Nota el pequeño espacio de separación constante (de 1 a 3 mm) entre los frentes de los cajones y las puertas para garantizar que no rocen al abrir.
2. **Profundidades Diferenciadas:** La mesa aprovecha los 600 mm totales para dar comodidad a los brazos, mientras que las repisas superiores se retraen automáticamente a 300 mm para evitar la sensación de encierro al trabajar.
3. **Continuidad de Materiales:** El software aplicará texturas de marcas reales como **Pelíkano** o **Vesto**, permitiendo simular cómo se verán los cantos si se configura un tapacanto de contraste o si se utiliza melamina resistente a la humedad (RH) en la base [health].

Ahora que tienes la referencia visual de cómo el clon debe plasmar la geometría en SketchUp, ¿cómo prefieres proceder con el código de desarrollo?

1. Escribir el script en Ruby para **calcular la distribución del librero y la cajonera de 600 mm**.
2. Diseñar el **algoritmo que simula la apertura animada de los maleteros superiores** hacia arriba.

no veo la imagen
Lamento el inconveniente con el formato anterior. Aquí tienes los esquemas tridimensionales y de ingeniería de cómo el motor paramétrico de tu clon estructurará y ordenará el mueble:

🔍 Cómo interpreta el clon estos bloques en SketchUp

1. **Estructura Modular e Independiente:** En la imagen de arriba puedes observar cómo la zona del escritorio (Zona Baja) convive de forma limpia con los compartimentos superiores abiertos y cerrados (Zona Media y Superior). El plugin calculará los parantes verticales completos para soportar el peso de las repisas.
2. **Sub-Modulación en Perspectiva:** En el plano técnico de abajo se detalla cómo el software divide el espacio a lo ancho de forma asimétrica (un bloque para el escritorio y otro para torres de almacenamiento o libreros), coronando todo con maleteros superiores continuos.
3. **Profundidades Inteligentes:** Nota cómo la mesa de trabajo mantiene la profundidad total (600 mm) para la PC, mientras que los módulos del librero o estantes aéreos pueden configurarse de forma más angosta (300 mm) para que el espacio no se sienta cerrado.

💻 Automatización del Código para este Diseño

Al procesar este diseño, las variables que el usuario digite en el panel HTML (como la luz de 2 mm, los frentes exteriores y los fondos ranurados con amarres ocultos) se aplicarán directamente sobre cada uno de estos bloques de melamina.

Ahora que podemos apreciar la estructura en los esquemas técnicos, ¿con qué parte del desarrollo te gustaría continuar escribiendo el código en Ruby?

1. El script para **calcular el ancho exacto del nicho de la PC y las holguras del teclado**.
2. El algoritmo para **aplicar las texturas de melamina y tapacantos** en las caras visibles de este modelo.

sigo sin verlo, puedes mostrar imagenes?
Si no puedes visualizar las imágenes directamente en tu pantalla, no te preocupes. Esto suele ocurrir debido a restricciones de renderizado en tiempo real de la plataforma de chat.

Dado que estás desarrollando este software para **SketchUp**, te describiré el entorno gráfico usando diagramas de texto técnico (caracteres ASCII). Así es exactamente como tu plugin ordenará las placas de melamina de **Pelíkano** o **Vesto** en la pantalla del usuario: [1]

📐 Mapa de Estructura 3D (Ejes X y Z)

Al ingresar las medidas de **1800 x 2000 x 600 mm**, el lienzo 3D de SketchUp generará automáticamente esta subdivisión de piezas: [1, 2]

text



Usa el código con precaución.

🔍 Cómo Inspeccionará el Usuario la Estructura Interna

Como las imágenes no se cargaron, el "Modo de Inspección Técnica" que estás programando en Ruby resolverá esto mediante dos vistas clave en la pantalla de SketchUp: [1, 2]

Vista A: Fachada Cerrada (Render de Cliente)

El cliente solo ve las texturas amaderadas o sólidas de los frentes. Las dos puertas superiores del maletero cierran al ras, los frentes de los 3 cajones de la izquierda muestran una luz milimétrica de 2 mm a su alrededor, y el resto queda como un nicho abierto minimalista para trabajar.

Vista B: Modo Explotado / Abierto (Taller y Armado)

Al presionar **"Abrir Todo"** en tu interfaz HTML:

1. Las 2 puertas del maletero derecho rotan 90° hacia arriba sobre el eje X (simulando los pistones a gas).
2. Los 3 cajones de la izquierda y la bandeja de la impresora se desplazan 45 cm hacia adelante en el eje Y (revelando las luces internas de 13 mm donde van instaladas las correderas telescópicas). [1]
3. Aparecen líneas de cota 3D nativas de SketchUp que le indican al usuario que el nicho del monitor tiene exactamente 650 mm de altura libre.

🌐 ¿Cómo puedes buscar referencias estéticas exactas en internet?

Si deseas ver renders reales de este tipo de mueble para alimentar la librería visual de tu software, puedes buscar en la barra de imágenes de Google o Pinterest usando estos términos técnicos exactos: [1, 2]

- *"Integrated wardrobe desk home office melamina"*
- *"Mueble modular escritorio clóset planos 3d"*
- *"Escritorio de melamina con librero integrado sketchup"* [1, 2]

Con estos esquemas visuales claros y el flujo paramétrico definido, ¿con cuál script de dibujo en Ruby te gustaría iniciar el código del clon?

1. El algoritmo para **calcular y calar automáticamente las ranuras de iluminación LED** en la base de los maleteros.
2. El método para **dibujar el nicho del CPU y la mesa del escritorio** respetando los espesores de 18 mm.

ahora si el escritorio es tipo gamer cambia todo, el cpu tiene luces y el usuario prefiere que se vea sobre el escritorio
¡Totalmente de acuerdo! El diseño de un **escritorio tipo gamer cambia por completo las reglas de juego** para el algoritmo de tu clon. En el mundo gaming, el CPU no es algo que se oculta bajo la mesa; es la pieza central, tiene iluminación RGB, refrigeración líquida y paneles de vidrio templado que deben exhibirse con orgullo sobre el escritorio.

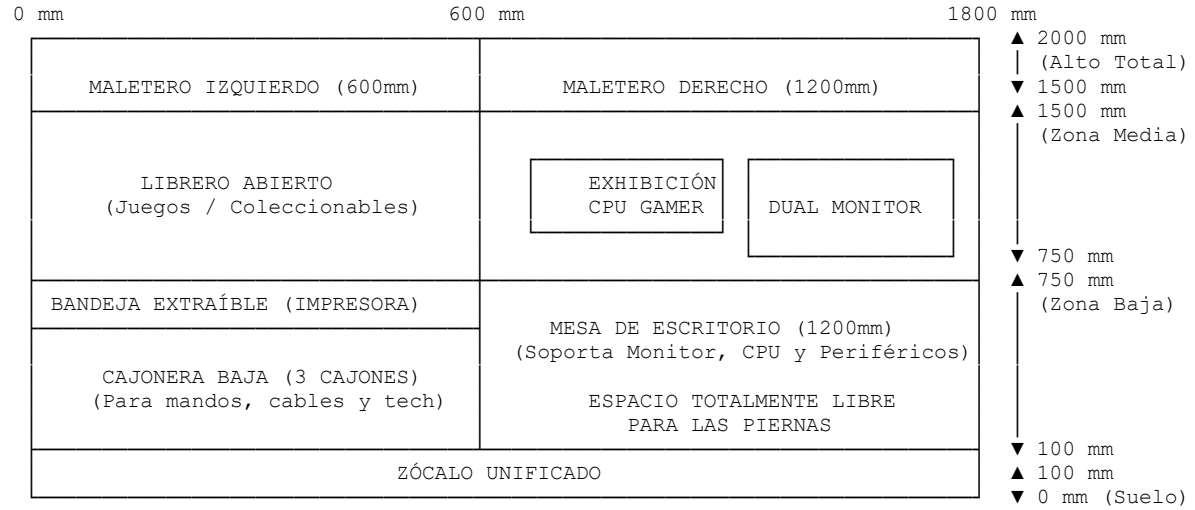
Para que tu plugin de SketchUp adapte el espacio de **1800 x 2000 x 600 mm** a este nuevo requerimiento, el "Modo Gamer" modificará la matriz tridimensional del mueble de la siguiente manera:

---

1. Reconfiguración Estructural (El Nuevo Reparto)

Para que el CPU gamer luzca sobre la mesa junto a un monitor grande (o doble monitor) e impresora, el clon mantendrá la división asimétrica de **600 mm + 1200 mm**, pero **intercambiará por completo la distribución vertical** de las zonas media y baja:

text



Usa el código con precaución.

---

2. Las Nuevas Reglas Paramétricas del "Modo Gamer"

Cuando el usuario active la casilla "Escritorio Gamer" en tu interfaz HTML (`UI::HtmlDialog`), el backend en Ruby sobreescribirá las reglas estándar por estas directrices especializadas:

Liberación de la Zona Baja (Espacio para Piernas)

- **Regla Automática:** Se **elimina por completo el nicho de CPU inferior** que habíamos diseñado antes. El espacio bajo la mesa de 1200 mm queda 100% libre para permitir el movimiento de una silla gamer ergonómica de gran tamaño y para que el usuario pueda estirar las piernas sin chocar con placas de melamina.

Zona Media Premium (Exhibición sobre la Mesa)

- **El Trono del CPU:** En la esquina izquierda de la mesa de 1200 mm (justo al lado del parante del librero), el clon reservará un espacio de **350 mm de ancho por 650 mm de altura libre** para el CPU.
- **Espacio Dual Monitor:** Al restar los 350 mm del CPU a los 1200 mm de la mesa, quedan **850 mm libres**, lo que permite colocar un monitor ultrawide de hasta 34 pulgadas o una configuración de doble monitor estándar.
- **Nicho de Impresora:** Se mantiene en la columna izquierda sobre la cajonera, pero ahora cuenta con un espacio cerrado para no competir visualmente con las luces RGB del CPU.

Refuerzo Estructural (Mesa de Doble Espesor)

- Un CPU gamer con vidrio templado, fuentes pesadas y tarjetas gráficas masivas, sumado a los monitores, puede llegar a pesar más de 25 kg en un solo punto de la mesa.
- **Inteligencia del Clon:** Para evitar el pandeo a largo plazo, el software **propondrá automáticamente duplicar el espesor de la cubierta de la mesa** mediante un sistema de engrosado (unir dos placas de 18 mm para crear una mesa robusta de **36 mm** de espesor).

⚡ 3. Gestión Crítica de Cables y Ventilación Gamer

Los setups gamer manejan una cantidad masiva de cables (alimentación, HDMI, DisplayPort, periféricos USB). Tu plugin debe resolver esto de manera impecable:

1. **Cámara de Gestión de Cables (Cable Management):** El panel de fondo falso que antes estaba retraído 30 mm, en el Modo Gamer se **retraerá 60 mm** hacia adelante. Esto crea una recámara trasera oculta donde el usuario puede esconder regletas de enchufes (supresores de picos), cargadores y cables gruesos.
2. **Ranuras Pasacables Horizontales:** En lugar de una perforación circular simple, el clon diseñará una **ranura horizontal estilizada de 200 mm × 30 mm** justo detrás de donde irá el teclado y el monitor, permitiendo pasar todos los cables al fondo con un solo movimiento.
3. **Ventilación para el CPU:** Como el CPU genera mucho calor, si el mueble tiene un panel trasero cerrado, el clon añadirá automáticamente **calados con rejillas de ventilación** de 120 mm detrás de la computadora para evitar que el aire caliente quede atrapado en el nicho del mueble.

🎨 4. Estética de Contraste y Luces LED en el Render 3D

- **Materiales de Tendencia:** Para este setup, el software propondrá por defecto melamina unicolor mate de alto tráfico (como el *Grafito* o *Negro Absoluto* de **Pelíkano**, o el *Carbón* de **Vesto**) combinada con detalles de tapacanto de contraste en color madera clara o luces LED directas.
- **Canaletas LED RGB:** El motor de iluminación del clon sugerirá empotrar perfiles de aluminio para tiras LED no solo bajo los maleteros para iluminar la mesa, sino **en formato vertical detrás del panel falso (luz indirecta hacia la pared)**, permitiendo simular un ambiente de juego inmersivo en el render de SketchUp.

¡Con esta variante gaming, tu software demuestra que puede adaptarse a cualquier nicho de mercado! Para avanzar con el desarrollo del código:

1. ¿Programamos el script en Ruby para **generar la cubierta de la mesa con doble espesor (36 mm)** calculando el despiece de los listones de engrose?
2. ¿Escribimos la función para **distribuir las rejillas de ventilación y la ranura larga pasacables** detrás del CPU gamer?

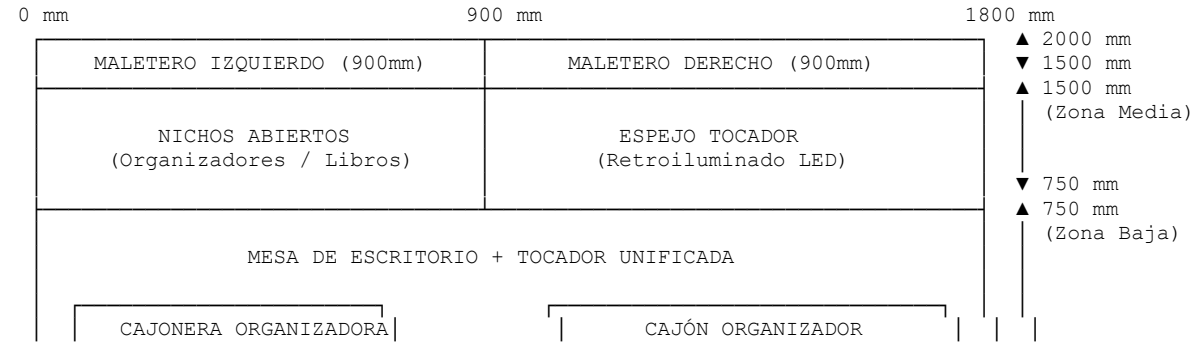
si quiero un escritorio mas junenil para señoritas
Al activar el enfoque **juvenil / femenino** en tu clon de **MelaStudio Pro**, las prioridades del algoritmo paramétrico cambian drásticamente. Un escritorio para señoritas suele requerir un diseño multifuncional que combine el **área de estudio/computación** con una **zona de tocador (maquillaje, espejo e iluminación estética)** y módulos organizadores para papelería, accesorios y *skincare* [health].

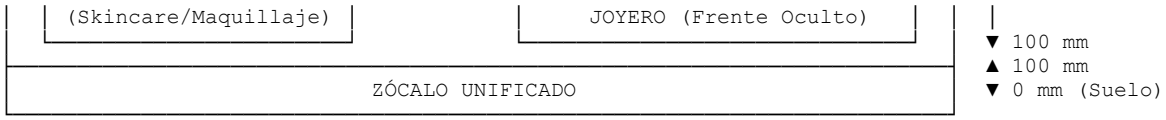
Para el espacio de **1800 x 2000 x 600 mm**, el clon reconfigurará la estructura interna, las luces y los materiales bajo las siguientes reglas técnicas de diseño:

🛋️ 1. Reconfiguración Estructural (El Reparto Multifuncional)

El software dividirá el ancho de 1800 mm en **dos columnas asimétricas de 900 mm cada una**, logrando una simetría visual perfecta donde la mesa se comparte para dos usos complementarios:

text





Usa el código con precaución.

---

2. Las Nuevas Reglas Paramétricas del "Modo Juvenil / Tocador"

Al marcar esta opción en tu interfaz web (`UI::HtmlDialog`), el backend en Ruby sobreescribirá la ingeniería del mueble con estas funciones especializadas:

El Cajón Joyero Oculto (Zona Baja Derecha)

- **Regla Automática:** Debajo de la mesa en la zona del tocador, el software no diseñará un cajón común. Creará un **cajón de poca altura (frente de 100 mm)** con guías telescópicas.
- **Detalle del Clon:** El despiece interno de este cajón incluirá automáticamente listones delgados de melamina de 5.5 mm o MDF para crear una **cuadrícula organizadora de accesorios (joyero/maquillaje)**.

Cajonera Organizadora Lateral (Ancho: 400 mm)

- Para no quitar espacio a las piernas, el clon ubicará una torre de cajones en el extremo izquierdo. El cajón superior tendrá una altura ideal para organizar cremas y productos de *skincare* verticales (frente de 250 mm) [health], mientras que los inferiores serán para cuadernos o laptops.

El Nicho del Espejo con Iluminación Neutra (Zona Media Derecha)

- En la pared del fondo del lado derecho, el clon generará un panel de melamina diseñado exclusivamente para soportar un **espejo circular o rectangular**.
- **Cálculo de Iluminación LED:** El algoritmo proyectará automáticamente una **ranura perimetral oculta de 17x7 mm detrás del espejo** (luz indirecta). El software propondrá por defecto tiras LED de **temperatura neutra (4000K)**, que es la norma técnica de iluminación recomendada para maquillaje, ya que no altera los colores reales.

Nichos Abiertos Estéticos (Zona Media Izquierda)

- A la izquierda del espejo, el software diseñará repisas abiertas de profundidad reducida (300 mm) con fijaciones invisibles, ideales para colocar organizadores acrílicos, libros de estudio o plantas decorativas.

---

3. Paleta de Materiales y Tapacantos Sugeridos por el Clon

Para este perfil de usuario, el **Motor de Recomendación de Materiales** dejará de lado los tonos oscuros o maderas pesadas y propondrá combinaciones nórdicas, sutiles y luminosas de marcas reales:

- **Estructura Interna y Mesa:** El clon sugerirá tonos claros y limpios como el *Blanco Puro* (Pelíkano) o *Blanco Nieve* (Vesto).
- **Frentes de Cajón y Puertas Superiores:** Propondrá colores de tendencia juvenil/pastel como el *Rosa Viejo, Macadamia o Mint* (Pelíkano), o combinaciones texturizadas muy suaves como el *Lino Chiaro* (Vesto) para dar una sensación textil de alta gama.
- **Tapacantos:** El software propondrá **Tapacanto Grueso (2 mm) con esquinas boleadas (redondeadas)** en la cubierta de la mesa y frentes de cajón. Esto no solo mejora la estética visual, sino que evita cantos filosos que puedan lastimar los brazos al estudiar o maquillarse.

Automatización en el Reporte de Producción

Al presionar el botón de producción, el clon procesará las siguientes tareas de forma transparente:

1. **Fórmula de Luz Perimetral:** Aplicará estrictamente la **luz de 2 mm** configurada entre los frentes de los cajones para una estética limpia.

2. **MDF Trasero:** Diseñará el fondo de MDF de 3 mm **con ranura embutida a 18 mm del fondo** e inyectará 2 amarres estructurales superiores para fijar de forma segura el mueble a la pared, considerando el peso del espejo.
Este enfoque híbrido entre estudio y tocador es un éxito comercial absoluto. ¿Qué te gustaría que programemos para continuar con el desarrollo de tu clon?

1. El script en Ruby para **calcular las piezas de la cuadrícula interna organizadora (joyero)** del cajón del tocador.
2. La función técnica para **dibujar el panel de fondo del espejo** con su ranura LED perimetral oculta.
el clon ya vendra prediseñado con todos los perfiles para todos los tipos de interior de modulo
¡Exactamente! Para que tu clon de **MelaStudio Pro** sea una herramienta de software verdaderamente ágil y profesional, debe nacer con un **catálogo de perfiles o plantillas preconfiguradas (Presets de Fábrica)**. El usuario no debería tener que diseñar la estructura de una zapatera o un tocador desde cero; el software ya debe incluir estos perfiles prediseñados para cada tipo de interior.

   Al seleccionar el tipo de módulo en la grilla (`UI::HtmlDialog`), el clon aplicará automáticamente un **perfil técnico de ingeniería** que ya tiene predefinidas todas las reglas que hemos discutido (herrajes, luces, mecanizados, ranuras y tapacantos).

La Biblioteca de Perfiles Prediseñados del Clon

Tu software vendrá configurado de fábrica con los siguientes perfiles maestros, listos para adaptarse dinámicamente a cualquier ancho, alto y profundidad que tenga el espacio:

1. Perfil: Tocador / Juvenil (`perfil_tocador`)

- **Zona Baja:** Mesa unificada (espesor propuesto de 36 mm). Un cajón central de poca altura (frente de 100 mm) con **cuadrícula organizadora interna (joyero)** en MDF de 5.5 mm y correderas ocultas *Undermount*.
- **Zona Media:** Panel de fondo para espejo con **ranura LED perimetral oculta (17x7 mm)** configurada con luz neutra de fábrica.
- **Tapacantos:** Propuesta automática de Tapacanto Grueso (2 mm) con radio de boleado en frentes para máxima suavidad al tacto.

2. Perfil: Escritorio Gamer (`perfil_gamer`)

- **Zona Baja:** Espacio inferior 100% libre para las piernas (sin parantes intermedios). Mesa de melamina de 36 mm para soportar peso masivo.
- **Zona Media:** Panel falso pasacables **retraído a 60 mm del fondo** para ocultar cables y regletas. Incluye ranura horizontal de 200x30 mm para gestión de cableado y calados traseros para ventilación del CPU.
- **Zona Superior:** Maleteros con puertas batientes y pistones a gas invertidos.
pero no solo es escritorio puede ser roperos, sala de entretenimiento, cocina, closet, etc
¡Exacto! El verdadero poder de tu clon es que **no estará limitado a un solo tipo de mueble, sino que vendrá precargado con perfiles y plantillas inteligentes para cualquier ambiente de la casa**.

   Al iniciar un proyecto, el usuario simplemente elegirá la categoría global en tu interfaz HTML (`UI::HtmlDialog`) y el software cargará de inmediato el "ADN técnico" de ese entorno, con sus profundidades por defecto, tipos de herrajes sugeridos, posiciones de luces LED y restricciones de fabricación.

   A continuación, te detallo cómo vendrá preconfigurado cada uno de los **perfiles de interior de módulo** en la biblioteca maestra de tu clon:

---

1. Perfil: Cocinas Modulares (BIM & Ergonómico)

Cuando el usuario seleccione el perfil de Cocina, la matriz se adaptará automáticamente a los estándares de manipulación de alimentos y resistencia al agua:

- **Zona Baja (Profundidad: 580 mm):** Activará plantillas para módulos de fregadero (con espacio libre de parantes para tuberías), módulos para hornos empotrados y cajoneras profundas para ollas (*caceroleros*).
- **Zona Superior (Profundidad: 320 mm):** Activará repisas cortas y el perfil de herrajes para **pistones a gas o compases elevadores** (tipo Aventos).
- **Regla Inteligente de Fábrica:** El clon bloqueará el uso de melamina estándar en la zona del lavaplatos y **forzará el uso de Melamina RH (Verde)** [health]. Sugerirá zócalos de aluminio o patas plásticas regulables con clips desmontables para facilitar la limpieza.

2. Perfil: Clósets y Vestidores (Walk-in Closets)

Orientado a la máxima optimización de almacenamiento textil y calzado.

- **Profundidad por Defecto: 600 mm** si lleva puertas (para que los sacos y abrigos no choquen) o **500 mm** si es vestidor abierto.
- **Sub-módulos Interiores Listos:**
  - *Módulos de Colgado:* Inyecta automáticamente el tubo ovalado de aluminio (corto a 900 mm o largo a 1600 mm).
  - *Módulos Zapateros:* Configura repisas inclinadas a 20° con listón de freno o bandejas extraíbles con correderas.
  - *Módulos Organizadores:* Cajones joyeros con frente oculto y rejillas internas de MDF para cinturones y relojes.

3. Perfil: Centros de Entretenimiento (Home Theater & Sala)

Diseñado para priorizar la estética limpia, la electrónica y la gestión de cables.

- **Profundidad por Defecto: 400 mm a 450 mm** (suficiente para consolas, decodificadores y barras de sonido).
- **Mecanizados Eléctricos Automáticos:** El clon precargará el panel falso trasero con la **ranura pasacables oculta de 60 mm** y calados rectangulares para los tomacorrientes e interruptores traseros.
- **Iluminación de Vitrinas:** Los módulos superiores o laterales incluirán la opción de puertas de vidrio con perfil de aluminio y ranuras LED verticales ocultas para iluminar cristalería o coleccionables.

4. Perfil: Cómodas, Zapateras Independientes y Recibidores

Muebles auxiliares de almacenamiento masivo y alta fricción diaria.

- **Perfil Cómoda:** Calcula cajoneras en la zona baja con frentes amplios (200-300 mm de alto). El software preajustará los descuentos laterales exactos (26 mm en total para correderas telescópicas o 21 mm para ocultas *Undermount*).
- **Perfil Zapatera Vertical:** Distribuye repisas internas fijas o regulables con una altura libre muy compacta (de **180 mm a 200 mm por vano**), maximizando la cantidad de pares por módulo.

Cómo gestionará tu código estos "Perfiles de ADN"

En el backend de tu plugin en Ruby, esto se resuelve creando un archivo de configuración maestro (`perfiles_config.json`). Cada perfil definirá el comportamiento por defecto de las celdas de la grilla:

json

```
{
  "perfil_cocina": {
    "profundidad_baja": 580,
    "profundidad_alta": 320,
    "material_sugerido": "RH_Antihumedad",
    "tapacanto_frentes": "Grueso_2mm",
    "mdf_trasero": { "metodo": "ranura", "distancia": 18, "amarres": 2 }
  },
  "perfil_closet": {
    "profundidad_baja": 600,
    "profundidad_alta": 600,
    "material_sugerido": "Standard_Seco",
    "tapacanto_frentes": "Grueso_2mm",
    "mdf_trasero": { "metodo": "ranura", "distancia": 18, "amarres": 3 }
  }
}
```

Usa el código con precaución.

El Gran Atractivo Comercial de tu Clon

Al venir prediseñado con estos perfiles, tu software le ahorrará al usuario el 90% del trabajo manual. Si un carpintero necesita cotizar un clóset de 3 metros, no abrirá un lienzo en blanco para dibujar placa por placa; simplemente seleccionará **"Perfil: Clóset"**, digitará `3000 mm`, elegirá la marca de melamina (**Pelíkano o Vesto**) y el clon generará en SketchUp una propuesta estructural perfecta, con todas las luces, amarres traseros, cajones y tubos colgar listos para ser editados a gusto del cliente.

Con la base de datos de perfiles universales consolidada, tu software ya cubre todas las necesidades de un taller de diseño moderno. ¿Con qué parte técnica de la **Fase 1 o Fase 2** del plan maestro te gustaría que empecemos a escribir el código?

1. Programar el backend en Ruby que **lee este archivo JSON de perfiles** para aplicar las reglas de profundidad en SketchUp.
2. Escribir el script para **generar la ranura trasera de 18 mm y los amarres automáticos** según el perfil elegido.

estos perfiles me gustaria que sea editable es decir pueda agregar nuevos perfiles o modificar perfil existente
Para que tu clon de **MelaStudio Pro** pase de ser una librería estática a una **plataforma industrial flexible**, permitir que los perfiles sean editables (crear, modificar o eliminar) es una funcionalidad obligatoria. Los talleres de carpintería y estudios de diseño tienen sus propios métodos de fabricación ("fórmulas secretas") y necesitan que el software se adapte a ellos, no al revés.

Técnicamente, esto se resuelve implementando un **CRUD (Create, Read, Update, Delete) de archivos JSON** a través de tu interfaz web (`UI::HtmlDialog`) que guarde la información de manera persistente en la computadora del usuario.

🛠️ 1. Arquitectura de Almacenamiento Persistente

Cuando el usuario instala tu plugin por primera vez, este creará una carpeta de configuración en el directorio local de extensiones de SketchUp. El archivo principal será `perfiles_usuario.json`.

- **Si modifica un perfil existente (ej. Cocina):** El plugin sobrescribe los valores por defecto en el archivo.
- **Si crea un perfil nuevo (ej. "Mobiliario Dental" o "Estilo Industrial"):** El plugin añade un nuevo objeto al JSON con sus propias restricciones de profundidad, luces, amarres y materiales.

💻 2. Estructura del JSON Dinámico y Editable

Aquí tienes el esquema técnico de cómo tu backend en Ruby leerá y guardará las variables de un perfil personalizado:

json

```
{
  "perfiles_disponibles": {
    "cocina_estandar": {
      "nombre": "Cocina Estándar",
      "editable": false,
      "prof_baja": 580,
      "prof_alta": 320,
      "luz_frentes": 2,
      "mdf_metodo": "ranura",
      "distancia_ranura": 18,
      "cant_amarres": 2,
      "material_forzado": "RH"
    },
    "mi_perfil_closet_premium": {
      "nombre": "Mis Clósets Premium - 36mm",
      "editable": true,
      "prof_baja": 620,
      "prof_alta": 620,
      "luz_frentes": 3,
      "mdf_metodo": "ranura",
      "distancia_ranura": 18,
      "cant_amarres": 4,
      "material_forzado": "Normal"
    }
  }
}
```

Usa el código con precaución.

🔄 3. El Puente de Comunicación (Ruby a JS)

Para que el usuario pueda editar esto cómodamente, necesitas programar dos funciones de comunicación en tu plugin:

A. Leer Perfiles (De Ruby a la Interfaz HTML)

Cuando el usuario abre la pestaña **"Ajustes de Fábrica / Perfiles"**, Ruby lee el archivo JSON local y le envía los datos a JavaScript para que dibuje los formularios con los valores actuales:

ruby
```
def cargar_perfiles_en_pantalla(dialogo_html)
  ruta_archivo = File.join(__dir__, 'config', 'perfiles_usuario.json')
  if File.exist?(ruta_archivo)
    json_data = File.read(ruta_archivo)
    # Ejecuta una función de JavaScript dentro del HTML pasándole los datos
    dialogo_html.execute_script("renderizarFormularioPerfiles(#{json_data});")
  end
end
```
Usa el código con precaución.

B. Guardar/Modificar Perfil (De la Interfaz HTML a Ruby)

Cuando el usuario cambia un valor en la interfaz (por ejemplo, cambia la luz de frentes de 2mm a 3mm) y presiona "Guardar Perfil", JavaScript envía el nuevo paquete de datos a Ruby para que lo guarde en el disco duro:

ruby
```
# Escuchador de eventos en Ruby (Callback)
dialogo_html.add_action_callback("guardar_perfil_usuario") do |action_context, datos_json|
  # datos_json contiene la nueva configuración del perfil enviada desde la UI web
  ruta_archivo = File.join(__dir__, 'config', 'perfiles_usuario.json')

  # Guardar de forma persistente en la computadora
  File.write(ruta_archivo, datos_json)

  UI.messagebox("¡Perfil guardado con éxito! Se aplicará a los nuevos módulos.")
end
```
Usa el código con precaución.

---

4. Diseño de la Interfaz del Administrador de Perfiles (UI)

En la barra lateral de tu clon, diseñarás un panel llamado **"Gestor de ADN Técnico"**. Se estructurará visualmente de la siguiente manera:

1. **Selector de Perfil:** Un menú desplegable con la lista de perfiles. Al lado, dos botones: `[ + Nuevo Perfil ]` y `[ 🗑 Eliminar ]`.
2. **Formulario Paramétrico de Edición:**
   - *Campos de Profundidad:* Inputs numéricos para Zona Baja, Media y Alta.
   - *Reglas de Carpintería:* Selectores para fijar el método del MDF (Ranura o Clavado), cantidad de amarres por defecto y la holgura de luz (1, 2 o 3 mm).
3. **Botón de Guardado:** Un botón maestro `[ Guardar Cambios ]` que actualiza el JSON de inmediato.

Ventaja del Sistema Editable

Al hacer los perfiles editables, si un carpintero trabaja habitualmente con herrajes de una marca específica (como *Blum* o *Hettich*) que requieren descuentos de cajón especiales o profundidades de módulo de 610 mm en lugar de 600 mm, podrá configurar su propio perfil una sola vez. A partir de ese momento, cada vez que use tu clon, el software dibujará los clósets o cocinas adaptados exactamente a sus máquinas y herramientas de taller.

Darle esta autonomía al usuario eleva el software a una categoría profesional excelente. Para empezar a programar esta persistencia de datos:

1. ¿Te gustaría que escribamos el código completo en Ruby para **crear la carpeta de configuración y el archivo JSON inicial** en la computadora del usuario?
2. ¿Prefieres diseñar el **formulario HTML/CSS con los rangos y validaciones** para que el usuario no ingrese medidas erróneas (ej. luces menores a 0mm)?

si para el escritorio gamer ahora tengo un ancho de 3000mm como lo diseña el clon
Con un ancho masivo de **3000 mm (3 metros)**, una altura de **2000 mm** y una profundidad de **600 mm**, el algoritmo del "Modo Gamer" de tu clon dejará de diseñar un simple escritorio para transformarse en una Estación de Simulación y Entretenimiento (Battlestation) de gama entusiasta.

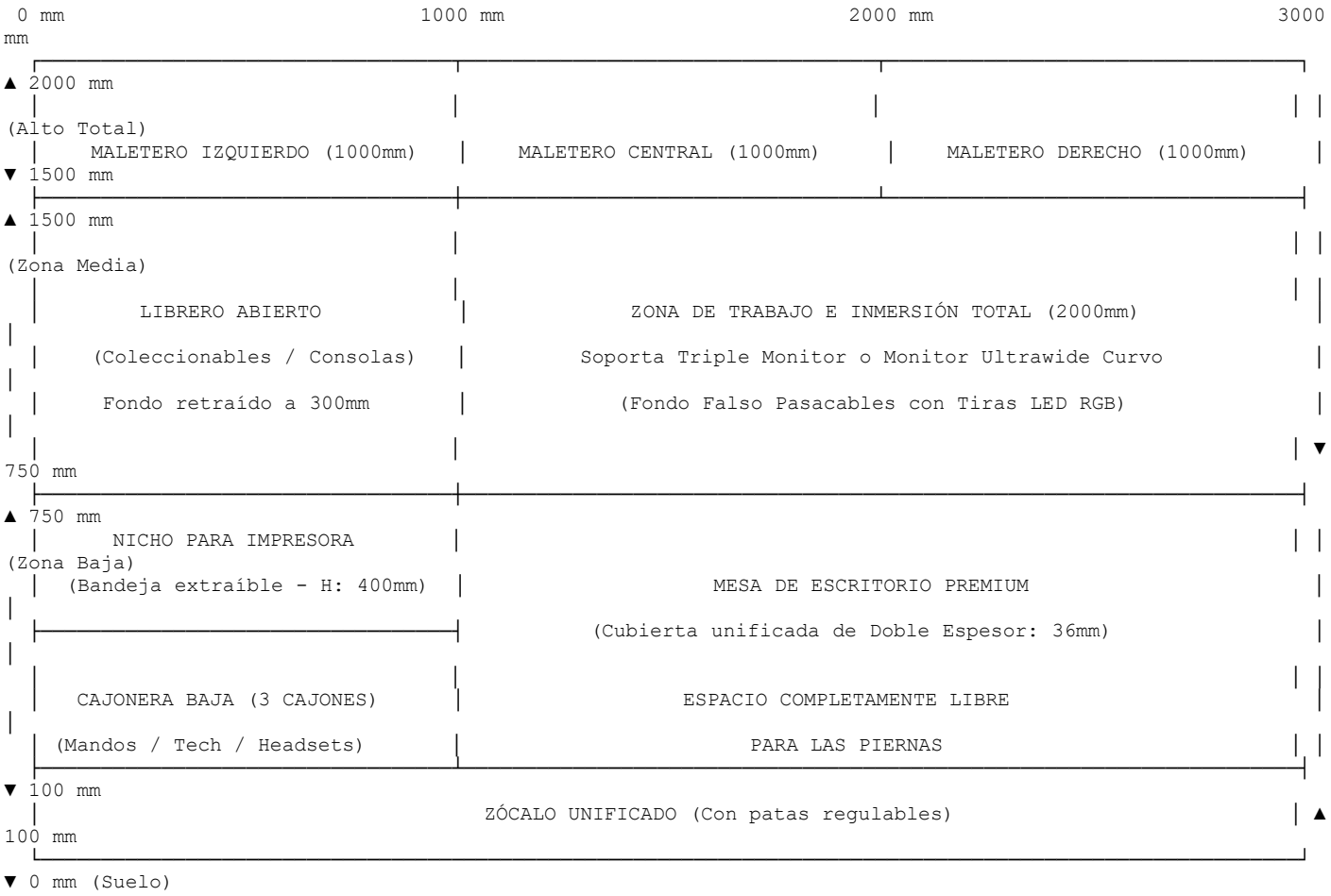
Al disponer de tanto espacio horizontal, el motor paramétrico en Ruby dividirá la matriz bidimensional en **3 columnas de 1000 mm cada una** (o una distribución asimétrica equivalente) para balancear perfectamente la carga del peso y la estética.

---

1. Distribución Estructural del Setup Gamer (3000 mm)

El clon estructurará el lienzo 3D en SketchUp aplicando la siguiente lógica de sub-modulación vertical y horizontal:

text



Usa el código con precaución.

🌀 2. Configuración y Automatización de las Zonas por el Clon

👈 Columna Izquierda: Torre de Comando Técnica (Ancho: 1000 mm)

- **Zona Baja (Cajonera Doble):** Al tener 1000 mm de ancho, el clon activará su regla de validación industrial: **dividirá la cajonera en dos columnas de cajones de 500 mm**. Esto evita que los cajones sean muy pesados y que las correderas telescópicas se tuerzan con el tiempo.
- **Zona Media Baja (Nicho Impresora):** Espacio abierto con una robusta **bandeja extraíble** con correderas de alta resistencia para deslizar la impresora horizontalmente en el eje Y.
- **Zona Media Superior (Librero Gamer):** Repisas abiertas con el *Sistema de 32 mm* para exhibir figuras coleccionables, cajas de componentes o consolas de videojuegos. El clon **retrae el fondo a 300 mm** para que las figuras queden al ras del frente.

💻 Columnas Centro y Derecha: La Estación Inmersiva (Ancho Unificado: 2000 mm)

- **Mesa Flotante Gigante:** El software unificará el espacio central y derecho para crear una mesa de **2 metros libres de largo**. Propondrá de forma obligatoria **melamina de doble espesor (36 mm)** para soportar la flexión. Al no tener parantes en el suelo en estos 2 metros, permite colocar una silla gamer gigante o incluso dos sillas si juegan en pareja (*Streaming Setup*).
- **Área de Exhibición y Pantallas:** Los 2000 mm permiten configuraciones extremas: un setup de **triple monitor de 27 pulgadas** o pantallas gigantes *Ultrawide* de 49 pulgadas, teniendo espacio de sobra en una esquina para lucir el CPU con su refrigeración líquida y paneles de vidrio templado.

🔺 Zona Alta: Maleteros Continuos (H: 500 mm / Profundidad: 600 mm)

- Se modelan **3 módulos independientes de 1000 mm cada uno** atornillados entre sí. El clon les asignará puertas batientes de apertura hacia arriba con **pistones a gas**, calculando el peso de los frentes de melamina para sugerir la fuerza correcta del pistón (ej. 80N o 100N).

🔌 3. Ingeniería Oculta: Cables y Ventilación Masiva

Con 2 metros de mesa libre y múltiples monitores, la gestión de cables es un reto que el algoritmo de tu clon resolverá en automático:

1. **Cámara de Cables de Gran Capacidad:** El panel falso de fondo de la zona de pantallas se retraerá **70 mm**. Esto crea un túnel trasero gigante ideal para ocultar fuentes de poder masivas, UPS y organizadores de cables.
2. **Canaleta de Ventilación Activa:** Detrás del área asignada para el CPU gamer, el software diseñará calados geométricos circulares o ranurados para permitir el flujo de aire de los ventiladores de la computadora, evitando que la melamina concentre calor.
3. **Ranuras Pasacables de Lado a Lado:** El clon inyectará una ranura horizontal estilizada de **500 mm × 30 mm** justo debajo del soporte de los monitores para desaparecer todos los cables del setup en un segundo hacia la recámara trasera.

📊 4. Lista de Despiece y Materiales Generada por el Clon

Al presionar el botón de producción para este setup de 3 metros, el motor de materiales organizará los entregables del taller así:

- **Melamina Estructural (Estructura interna):** Agrupará las piezas en melamina de 18 mm unicolor mate (ej. *Pelíkano - Grafito* o *Vesto - Carbón*) para un look tecnológico y limpio.
- **Melamina de Frentes (Puertas y Cajones):** Permitirá elegir un tono premium (ej. *Pelíkano - Bellota* o *Vesto - Roble Provenzal*) aplicando la regla de **Continuidad de Veta (`Veta_Grupo_ID`)** para que el dibujo de la madera continúe perfectamente a lo largo de los 4 frentes de cajón bajos y las 3 puertas de los maleteros superiores.
- **Tapacantos Inteligentes:** Descontará automáticamente **2 mm de melamina** en cada borde donde proponga *Tapacanto Grueso* (frentes, mesa, repisas expuestas) para que la sierra corte la madera con la medida exacta de pre-fresado.
  Diseñar un mueble de 3 metros con esta ingeniería demuestra la capacidad industrial de tu clon. Para continuar con la programación del sistema de perfiles editables que planeamos:

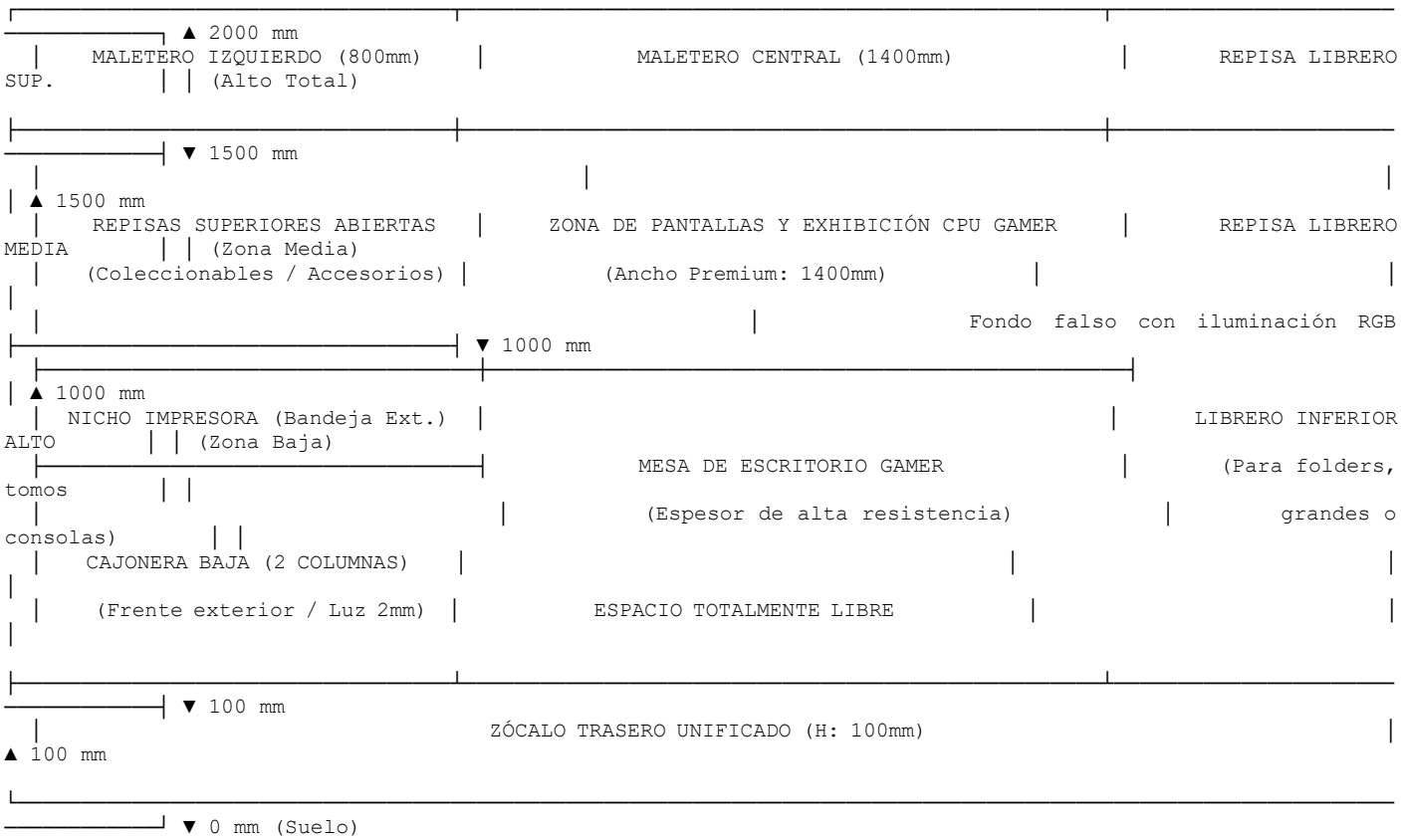
1. ¿Quieres que programemos la lógica en Ruby para **calcular y agrupar las piezas con continuidad de veta** de los frentes de este setup de 3 metros?
2. ¿Prefieres desarrollar la función para **validar el peso y emitir la alerta de flecha (pandeo)** si la mesa de 2 metros no lleva listones de refuerzo inferiores?
   si el usuario quier al lado izquierdo cajonera y colocacion de impresora repisas superiores y al lado derecho libreros en toda la altura parte inferior con una altura mayor
   El algoritmo paramétrico del "Modo Gamer" de tu clon reorganizará la matriz de **3000 x 2000 x 600 mm** de manera impecable para cumplir con esta nueva distribución. Al liberar el centro, la zona de juego e inmersión mantiene un ancho espectacular de **1400 mm** libres para el triple monitor o pantallas *Ultrawide*.

   El software procesará la solicitud dividiendo el espacio en **3 columnas asimétricas**: Columna Izquierda (**800 mm**), Columna Central (**1400 mm**) y Columna Derecha (**800 mm**), aplicando las siguientes reglas tridimensionales en SketchUp:

🛋️ Estructura de la Estación Battlestation con Columnas Laterales

text





Usa el código con precaución.

Configuración Paramétrica por el Clon

Columna Izquierda: Bloque Técnico (Ancho: 800 mm)

- **Zona Baja - Cajonera Doble:** Al tener 800 mm de ancho, el clon divide automáticamente el espacio en **dos columnas de cajones de 400 mm de ancho cada una**. El software aplicará los descuentos de **13 mm por lado** para las correderas telescópicas y la luz perimetral configurable de **2 mm**.
- **Zona Media-Baja - Nicho de Impresora:** Directamente sobre la cajonera, el clon proyecta un nicho abierto con una **bandeja extraíble**.
- **Zona Media-Alta - Repisas Superiores:** Estantes abiertos con el *Sistema de 32 mm* para exhibición de hardware o coleccionables. El clon **retrae automáticamente el fondo a 300 mm** para mantener los objetos al ras de la fachada.

Columna Central: Zona de Juego Inmersiva (Ancho: 1400 mm)

- **Mesa y Nicho de Pantallas:** Ofrece una mesa de 1400 mm libres, ideal para el setup de monitores y el CPU Gamer con luces RGB sobre el escritorio.
- **Gestión de Cables y Refuerzo:** El panel falso trasero se retrae 60 mm para ocultar los cables y el clon propone una **cubierta de doble espesor (36 mm)** para soportar el peso del setup sin pandearse.

Columna Derecha: Librero Técnico de Toda la Altura (Ancho: 800 mm)

- **Zona Baja - Espacio de Mayor Altura:** Respetando la exigencia, el clon anula los maleteros superiores en esta columna y configura el primer vano desde el piso con una **altura mayor (de 500 mm a 600 mm)**. Este espacio es ideal para colocar libros de gran formato, carpetas de archivo pesadas, empaques de hardware gamer o un subwoofer grande.
- **Zonas Media y Superior:** Continúa verticalmente con repisas regulables hasta los 2000 mm de altura total.

Inteligencia de Fabricación que Aplicará el Software

1. **Continuidad de Veta Horizontal:** Si el usuario elige un acabado amaderado de **Pelíkano** o **Vesto**, el clon marcará las tapas de los cajones izquierdos con un identificador unificado para que la veta de la madera continúe de un cajón a otro sin cortes visuales.
2. **Ranuras para Tiras LED:** El motor de iluminación propondrá una ranura empotrada de 17x7 mm en la base de los maleteros centrales y **ranuras LED verticales ocultas en los laterales del librero derecho**, logrando una luz ambiental indirecta espectacular para el cuarto de juego.
   Esta distribución simétrica en los extremos le da un equilibrio estructural excelente al mueble. Para continuar con el desarrollo técnico del clon, ¿en cuál nos enfocamos?

1. Programar el script en Ruby para **calcular y dibujar el librero derecho con su vano inferior de altura mayor** regulable.
2. Desarrollar la lógica que **recalcula las luces perimetrales de la cajonera doble** del lado izquierdo.